

- **DS Tutorial**

- 1. Data Structures and Algorithms (DSA) Tutorial**

Data structures and algorithms (DSA) are two fundamental components of any programming language. Understanding them is crucial for effective programming and problem-solving.

Our Data Structures and Algorithms (DSA) tutorial provides basic and advanced concepts of Data Structures. It is designed for beginners and professionals. It includes all topics of Data Structure such as [Array](#), [Linked List](#), [Stack](#), [Queue](#), [Graph](#), [Tree](#), [Hashing](#) and [Heap](#), [Searching](#), [Sorting](#), etc.

What is Data Structure and Algorithm?

The **data structure** name indicates that it organizes the data in memory. It is a fundamental of computer programming. Data structures are ways to organize, store, and manage data efficiently. It helps in structuring data so that it can be accessed and modified effectively. In programming, choosing the right data structure can make a huge difference in the performance and scalability of an application.

Every programming language provides its own data structures and different types of algorithms to handle these data structures. So, it is very important to know data structures for organizing large-scale data efficiently.

An **algorithm** is a set of rules that must be followed when solving a particular problem. In short, it is a step-by-step procedure to solve the problem. These steps describe a set of instructions to be executed in a certain order to get the expected output.

Why Data Structures and Algorithms?

In programming, data structures and algorithms are the backbone of efficient programming and problem-solving. It is important to learn data structures and algorithms because:

- **Optimized Performance:** Choosing the right data structure can significantly enhance speed and memory usage, making programs run faster and more efficiently
- **Scalability:** Well-designed algorithms enable software to manage larger data volumes without running into performance issues.
- **Problem-Solving:** It helps in breaking down complex problems into logical steps, which makes solutions more structured.
- **Competitive Programming:** Mastering DSA is crucial for success in coding competitions, where both speed and efficiency are key.
- **Real-World Applications:** From databases utilizing trees and hash maps to artificial intelligence employing graph algorithms, data structures, and algorithms are integral to nearly every software domain.

Applications of Data Structures and Algorithms

Data structure is used everywhere, from databases to web applications. They help to organize data for storage and retrieval, as well as address computational problems using established procedures.

1. Organizing and Storing Data

- **[Arrays](#):** It is used to store collections of the same data type.
- **[Linked Lists](#):** It is used for allocating and deallocating memory dynamically.
- **[Stacks](#):** It is used for tasks like function calls and undo/redo features.
- **[Queues](#):** It is used to store tasks and manage resources.
- **[Trees](#):** It is used to organize hierarchical data and search efficiently.
- **[Graphs](#):** It is used to represent networks and associations between data.

2. Solving Computational Problems

- **[Sorting](#):** Algorithms like [bubble sort](#), [quicksort](#), and [merge sort](#) are used to arrange data in a sorted order.

- **Searching:** Algorithms like [binary search](#) and [linear search](#) are used to find specific data within a structure.
- **Pathfinding:** Algorithms like Dijkstra's algorithm and A* are used to find the shortest path between two points in a graph.
- **Dynamic Programming:** This approach is used to solve optimization problems by breaking them down into smaller subproblems.

3. Efficiency and Performance

- **Time and Space Complexity:** DSA helps programmers to understand how algorithms and data structures affect the performance of code.
- **Optimized Code:** By choosing the right data structures and algorithms, programmers can write more efficient and faster code.

4. Real-World Applications

- **Social Networks:** Managing large amounts of user data and connections.
- **Search Engines:** Indexing and retrieving information from vast amounts of data.
- **Operating Systems:** Managing system resources and scheduling tasks.
- **Databases:** Organizing and querying data efficiently.
- **Networking:** Designing and implementing communication protocols.
- **Machine Learning and AI:** Training and running models.

Advantages of Data Structures and Algorithms

Advantages of Data Structures

The following are the advantages of a data structure:

- **Efficiency:** If the choice of a data structure for implementing a particular ADT is proper, it makes the program very efficient in terms of time and space.
- **Reusability:** The data structure provides reusability means that multiple client programs can use the data structure.
- **Scalability:** Ensures programs can handle large amounts of data without performance degradation.
- **Optimized Performance:** Choosing the right data structure improves speed and memory usage.

Advantages of Algorithms

- **Improved Efficiency:** Algorithms optimize operations like searching, sorting, and graph traversal.
- **Reduced Time Complexity:** It helps to minimize execution time by selecting the most efficient approach.
- **Better Problem-Solving:** It provides structured solutions to computational problems.
- **Resource Optimization:** It minimizes memory and CPU usage with optimized techniques.

2. What is Data Structure?

The basic building blocks of computer programming are data structures. They define how the organization, storage, and manipulation are done within a program. A good understanding of data structures is very important for developing effective and efficient algorithms.

Data Structure

The **data structure** indicates how data is organized in memory. Data structures are ways to organize, store, retrieve, and manage data efficiently. It helps in structuring data so that it can be accessed and modified effectively. In programming, choosing the right data structure can make a huge difference in the performance and scalability of an application.

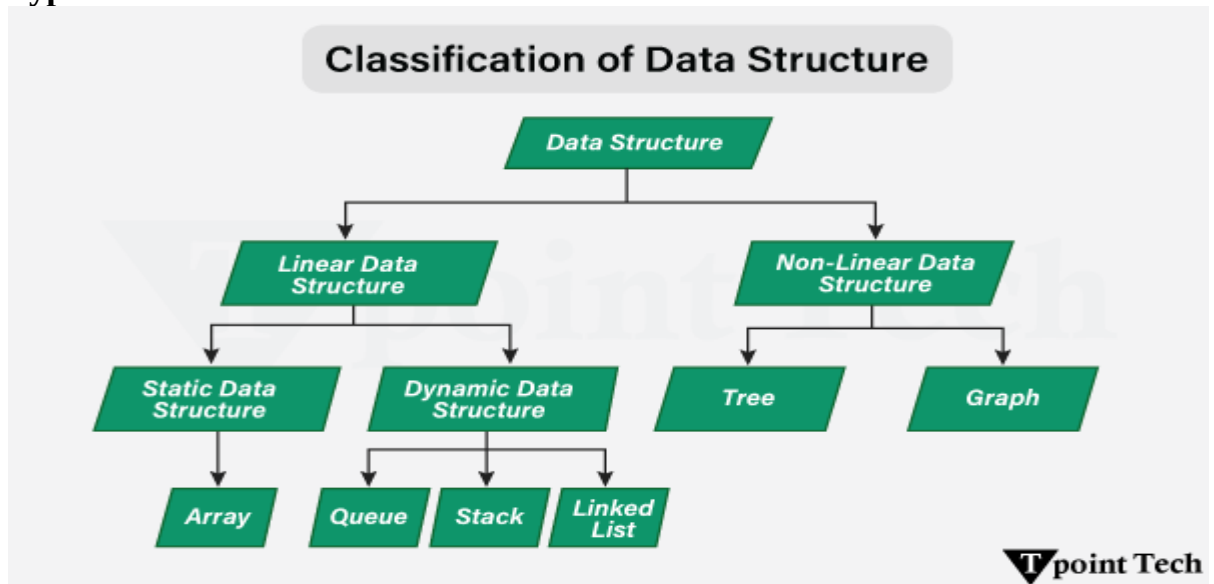
Example of Data Structure

The most common example of a data structure is the Array. It stores elements of the same data type in contiguous memory locations, which allows efficient indexing and retrieval.

Key Features of Data Structure

- **Efficient Data Storage:** It enables efficient management and retrieval of data, making operations such as searching, inserting, and deleting faster.
- **Memory Utilization:** It optimizes memory usage by structuring data to minimize space complexity.
- **Data Manipulation:** Structures such as stacks, queues, and trees facilitate specific operations like push/pop, enqueue/dequeue, and hierarchical traversal.
- **Data Organization:** Organizing data logically enables better data processing and operations.
- **Ease of Access:** Data structures such as arrays and linked lists offer quick access to elements, depending on their design.
- **Flexibility:** Various structures are appropriate for different scenarios, providing flexibility in programming.
- **Optimization:** Selecting the appropriate data structure can greatly enhance the speed and efficiency of an algorithm.
- **Dynamic Size:** Some data structures, such as linked lists and dynamic arrays, can dynamically adjust their size to accommodate varying amounts of data.

Types of Data Structure



Data structures can be mainly categorized into the following:

1. Linear Data Structure
2. Non-Linear Data Structure

Linear Data Structure

A type of data structure where data elements are arranged linearly or sequentially, where each element is attached to its next as well as previous adjacent elements, is called a linear data structure.

Note that the order of elements is also preserved in linear data structures. It means that the first element added will be the first one to be accessed or removed. Linear data structures can have fixed or dynamic sizes.

Example of Linear Data Structure

[Array](#), [Stack](#), [Queue](#), [Linked List](#), etc., are examples of linear data structures.

The linear data structure can be further categorized into the following:

1. Static Data Structure
2. Dynamic Data Structure

Static Data Structure

A type of data structure that has a fixed size of memory is called a static data structure. Accessing elements is easier in a static data structure. An [Array](#) is an example of a static data structure.

Dynamic Data Structure

A type of data structure where the size is not fixed is known as a dynamic data structure. The size can be updated randomly during runtime, which is considered efficient concerning the space (memory) complexity of the code. [Queue](#), [Stack](#), Linked List, and Tree are examples of dynamic data structures.

Non-linear Data Structure

A type of data structure where the placement of data elements is not done linearly or sequentially is called a non-linear data structure. Traversing all the elements in a single run is not possible in a non-linear data structure.

Example of Non-linear Data Structure

[Graphs](#), [Trees](#), Heap, and HashTable are examples of non-linear data structures.

Basic Operations of Data Structure

In data structures, basic operations are essential for manipulating and managing data efficiently. Here are the following common operations that can be used across various data structures.

1. **Insertion:** Adding a new element to the data structure.
2. **Deletion:** Removing an element from the data structure.
3. **Traversal:** Accessing each element of the data structure to perform some operation.
4. **Searching:** Finding an element in the data structure.
5. **Sorting:** Arranging elements in a specific order (ascending or descending).
6. **Updation:** Modifying an existing element in the data structure.
7. **Merge:** Combining two or more sorted data structures into a single sorted data structure.
8. **Access:** Accessing or retrieving the value of an element based on its position (index) or key.
9. **Reversing:** Reversing the order of elements in the data structure.

Note: Each data structure implements these operations differently based on its characteristics.

For example, inserting an element in an array requires shifting elements, while the same in a linked list can be done by updating pointers.

Advantages of Data Structures

The following are the advantages of a data structure:

1. **Efficiency:** If the choice of a data structure for implementing a particular ADT is proper, it makes the program very efficient in terms of time and space.
2. **Reusability:** The data structure provides reusability means that multiple client programs can use the data structure.
3. **Scalability:** Ensures programs can handle large amounts of data without performance degradation.
4. **Optimized Performance:** Choosing the right data structure improves speed and memory usage.

Use Cases of Data Structures

1. **Indexing:** Indexing of information can be achieved by mapping data values to their corresponding data items in a database with the help of data structures. Thus, making it easier to access to locate these data records.
2. **Scalability:** Data structures provide support to system scalability by helping computer programs process large data sets, solve complex problems, and efficiently use resources.

3. **Searching:** By organizing data with the help of data structures, it becomes easier for the applications and end users to comprehend. Locating and searching for data becomes easier using data structures.
4. **Data Exchange:** With the help of data structures, the organization of data can be done easily so that it can be shared between applications.
5. **Data Storage and Organization:** Data structures can provide storage of data efficiently and logically with high levels of persistence of data so that data can be accessed easily from other applications and databases.

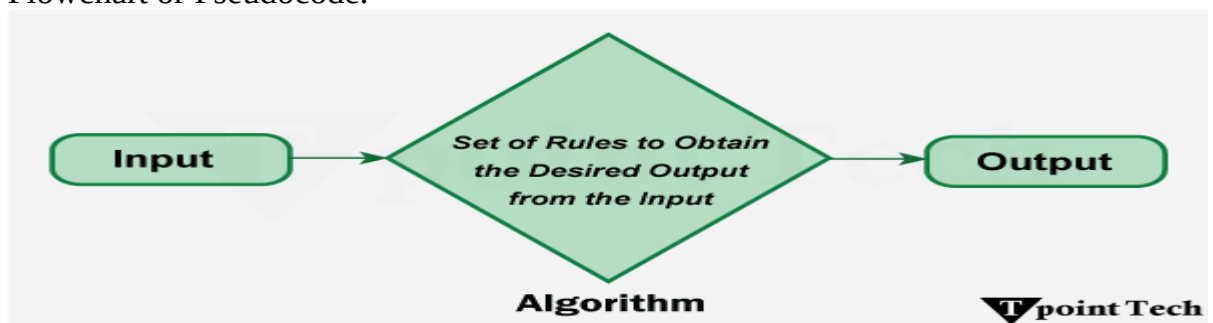
Applications of Data Structures

Data structure is the backbone of efficient programming, organizing, and managing data effectively. Data structure is used in the following domains:

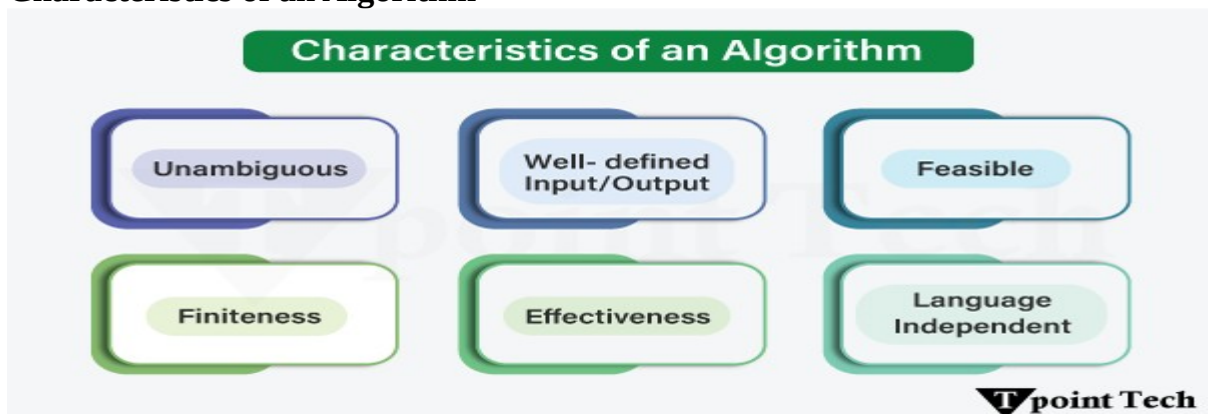
1. **Database Management Systems:** It is used to store and organize large amounts of structured data efficiently.
2. **Operating Systems:** It helps in memory allocation, process scheduling, and resource management.
3. **Computer Networks:** It is used in routing algorithms and network topology representation.
4. **AI & Machine Learning:** It helps in storing and processing large datasets for training models.
5. **Search Engines:** It is used in indexing and ranking web pages for fast retrieval.
6. **Cybersecurity:** It helps in encryption, hashing, and secure data storage.
7. **Gaming & Simulation:** It is used in pathfinding algorithms, physics simulations, and rendering.
8. **Bioinformatics:** It helps in analyzing DNA sequences and protein structures.
9. **Geographic Information Systems (GIS):** It is used for mapping and spatial data analysis.

3. What is an Algorithm? Definition, Characteristics & Examples

An algorithm is a set of rules or a sequence of instructions that one must perform in order to solve a well-formulated problem. The formal definition of an algorithm is that it contains a finite set of instructions that are carried out in a specific order to perform a particular task. It is not the complete program or code; it is just a solution (logic) to a problem, which can be represented either as an informal description using a Flowchart or Pseudocode.



Characteristics of an Algorithm



The following are the characteristics of an algorithm:

- **Well-defined Input:** An algorithm has some input values. We can pass zero or some input value to an algorithm.
- **Well-defined Output:** We will get one or more outputs at the end of an algorithm.
- **Unambiguous:** An algorithm should be unambiguous, which means that the instructions in an algorithm should be clear and simple.
- **Finiteness:** An algorithm should have finiteness. Here, finiteness means that the algorithm should contain a limited number of instructions, i.e., the instructions should be countable.
- **Effectiveness:** An algorithm should be effective, as each instruction in an algorithm affects the overall process.
- **Feasible:** The algorithm must be practical, simple, and generic so that it can be executed with the available resources. It must not have any dependency on some future technology or anything.
- **Language-Independent:** An algorithm must be language-independent so that the instructions in an algorithm can be implemented in any of the languages with the same output.

Stages of an Algorithm

Typical stages involved in an algorithm's functioning:

- **Problem:** A problem can be a real-world problem or any instance of a real-world problem for which we need to create a program or a set of instructions. The set of instructions is known as an algorithm.
- **Algorithm:** An algorithm will be designed for a problem, which is a step-by-step procedure.
- **Input:** After designing an algorithm, the required and the desired inputs are provided to the algorithm.
- **Processing and Decision Making:** The processing unit processes the provided input according to the algorithm and produces the desired output.
- **Output:** The output is the outcome or the result of the program.
- **Termination:** Once all the steps are executed successfully and the output is produced, the algorithm reaches its termination point.

Why do we need Algorithms?

We need algorithms for the following reasons:

- It is necessary because it solves complex problems effectively and efficiently.
- It provides a well-defined solution for a problem.
- It helps to solve problems quickly and with minimal resources (memory and time).
- It processes data, performs operations, and makes decisions without human involvement.
- It is used in various fields such as mathematics, computer science, engineering, finance, and many others to optimize processes.

Real World Example of Algorithm

Suppose we want to make lemon juice. The following are the steps required to make lemon juice:

Step 1: First, we will cut the lemon in half.

Step 2: Squeeze the lemon as much as you can and pour out its juice into a container.

Step 3: Add two tablespoons of sugar to it.

Step 4: Stir the container until the sugar dissolves.

Step 5: When the sugar gets dissolved, add some water and ice to it.

Step 6: Store the juice in the fridge for 5 to minutes.

Step 7: Now, it's ready to drink.

The above real-world example can be directly compared to the definition of the algorithm. We cannot perform Step 3 before Step 2; we need to follow the specific order to make lemon juice. An algorithm also says that every instruction should be followed in a particular order to perform a specific task.

Properties of Algorithm

- It should complete its work after a finite amount of time.
- A minimum of one output must be produced.
- It should accept zero or more inputs.
- It should be deterministic, meaning that for the same input, it will yield the same output.
- Each step of an algorithm must be effective, implying that some work should be done at every step.

Features of an Algorithm

The following are the factors that we need to consider for designing an algorithm.

- **Modularity:** If any problem is given and we can break that problem into small modules or small steps, which is a basic definition of an algorithm, it means that this feature has been perfectly designed for the algorithm.
- **Correctness:** The correctness of an algorithm is defined as when the given inputs produce the desired output, which means that the algorithm has been designed algorithm. The analysis of the algorithm has been done correctly.
- **Maintainability:** Here, maintainability means that the algorithm should be designed in a very simple, structured way so that when we redefine the algorithm, no major change will be made to the algorithm.
- **Functionality:** It considers various logical steps to solve the real-world problem.
- **Robustness:** Robustness means how clearly an algorithm can define our problem.
- **User-friendly:** If the algorithm is not user-friendly, then the designer will not be able to explain it to the programmer.
- **Simplicity:** If the algorithm is simple, then it is easy to understand.
- **Extensibility:** If any other algorithm designer or programmer wants to use your algorithm, then it should be extensible.

Importance of Algorithms

1. **Theoretical Importance:** When faced with any real-world problem, we break it down into smaller modules. To deconstruct the problem, we must understand all theoretical aspects.
2. **Practical Importance:** As we know, theory cannot be completed without practical implementation. Therefore, the importance of algorithms can be considered both theoretical and practical.

Issues of Algorithms

The following are the issues that come up while designing an algorithm:

- **How to design algorithms:** As we know, an algorithm is a step-by-step procedure, so we must follow some steps to design an algorithm.
- **How to analyze algorithm efficiency:** Analyzing algorithm efficiency is essential for understanding its performance, especially as the input size grows.

Approaches of the Algorithms

The following are the approaches used after considering both the theoretical and practical importance of designing an algorithm:

- **Brute Force Algorithm:** The general logic structure is applied to design an algorithm. It is also known as an exhaustive search algorithm that searches all the possibilities to provide the required solution. Such algorithms are of two types:

1. **Optimizing:** Finding all the solutions to a problem and then selecting the best solution, or if the value of the best solution is known, then it will terminate if the best solution is known.
 2. **Sacrificing:** As soon as the best solution is found, it will stop.
- **Divide and Conquer:** It is a very effective implementation of an algorithm. It allows you to design an algorithm in a step-by-step variation. It breaks down the algorithm to solve the problem in different methods. It enables us to break down the problem into different methods, and a valid output is produced for the valid input. This valid output is passed to some other function.
 - **Greedy Algorithm:** It is an algorithm paradigm that makes an optimal choice on each iteration with the hope of getting the best solution. It is easy to implement and has a faster execution time. But there are very rare cases in which it provides the optimal solution.
 - **Dynamic Programming:** It makes the algorithm more efficient by storing the intermediate results. It follows five different steps to find the optimal solution for the problem:
 1. It breaks down the problem into a subproblem to find the optimal solution.
 2. After breaking down the problem, it finds the optimal solution out of these subproblems.
 3. Stores the result of the subproblems is known as memorization.
 4. Reuse the result so that it cannot be recomputed for the same subproblems.
 5. Finally, it computes the result of the complex program.
 - **Branch and Bound Algorithm:** The branch and bound algorithm can be applied only to integer programming problems. This approach divides all the sets of feasible solutions into smaller subsets. These subsets are further evaluated to find the best solution.
 - **Randomized Algorithm:** As we have seen in a regular algorithm, we have predefined input and required output. Those algorithms that have some defined set of inputs and required output and follow some described steps are known as deterministic algorithms. What happens when the random variable is introduced in the randomized algorithm? In a randomized algorithm, some random bits are introduced by the algorithm and added to the input to produce the output, which is random in nature. Randomized algorithms are simpler and more efficient than deterministic algorithms.
 - **Backtracking:** Backtracking is an algorithmic technique that solves the problem recursively and removes the solution if it does not satisfy the constraints of a problem.

Categories of Algorithm

The major categories of algorithms are given below:

- **Sort:** An algorithm developed for sorting the items in a certain order.
- **Search:** An algorithm developed for searching the items inside a data structure.
- **Delete:** Algorithm developed for deleting the existing element from the data structure.
- **Insert:** Algorithm developed for inserting an item inside a data structure.
- **Update:** Algorithm developed for updating the existing element inside a data structure.

Algorithm Analysis

The process of comparing algorithms' rate of growth with respect to time and space complexity. The algorithm can be analyzed on two levels, i.e., the first is before creating the algorithm, and the second is after creating the algorithm. The following are the two analyses of an algorithm:

- **Priori Analysis:** Here, priori analysis is the theoretical analysis of an algorithm, which is done before implementing the algorithm. Various factors can be considered before implementing the algorithm, like processor speed, which does not affect the implementation part.
- **Posterior Analysis:** Here, posterior analysis is a practical analysis of an algorithm. The practical analysis is achieved by implementing the algorithm using any programming language. This analysis basically evaluates how much running time and space are taken by the algorithm.

The following are the differences between Priori Analysis and Posterior Analysis.

Aspect	Priori Analysis	Posterior Analysis
Analysis Time	Analysis is done before execution. For Example, Algorithm	Analysis is done after execution. For example, Program.
Hardware Dependency	This analysis is independent of the operating system and system architecture of the CPU.	This analysis is dependent on the operating system and system architecture of the CPU.
Cost	This analysis is cheaper because there is no additional requirement for software and hardware for execution.	This analysis is costly because of the additional requirements of software and hardware for execution.
Uniformity	It provides uniform values.	It provides non-uniform values.

Algorithm Complexity

The performance of the algorithm can be measured in two factors:

Time complexity

The time complexity of an algorithm is the amount of time required to complete its execution. The Big O notation denotes the time complexity of an algorithm. Here, big O notation is the asymptotic notation to represent the time complexity. The time complexity is mainly calculated by counting the number of steps to finish the execution. Let's understand the time complexity through an example.

1. sum=0;
2. //Suppose we have to calculate the sum of n numbers.
3. For i=1 to n
4. sum=sum+i;
5. //When the loop ends, then sum holds the sum of the n numbers
6. return sum;

In the above code, the time complexity of the loop statement will be at least n, and if the value of n increases, then the time complexity also increases. While the complexity of the code, i.e., return sum, will be constant as its value is not dependent on the value of n, and will provide the result in one step only. We generally consider the worst-case complexity as it is the maximum time taken for any given input size.

Space Complexity

An algorithm's space complexity is the amount of space required to solve a problem and produce an output. Similar to the time complexity, space complexity is also expressed in big O notation.

For an algorithm, the space is required for the following purposes:

1. To store program instructions
2. To store constant values
3. To store variable values
4. To track the function calls, jumping statements, etc.

Auxiliary Space

The extra space required by the algorithm, excluding the input size, is known as auxiliary space. The space complexity considers both the spaces, i.e., auxiliary space and the space used by the input. Therefore, we can calculate the space complexity by using the following formula:

1. Space Complexity = Auxiliary Space + Input Size

Types of Algorithms

The following are the types of algorithms:

- [Search Algorithm](#)
- [Sort Algorithm](#)

Search Algorithm

In case of computers, huge amounts of data are stored in a computer that, whenever the user asks for any data, then, the computer searches for that data in the memory and provides that data to the user. There are mainly two techniques available to search the data in an array:

- [Linear search](#)
- [Binary search](#)

Linear Search

Linear search is a very simple algorithm that starts searching for an element or a value from the beginning of an array until the required element is found. It compares the element to be searched with all the elements in an array; if a match is found, then it returns the index of the element, else it returns -1. This algorithm can be implemented on the unsorted list.

Binary Search

A Binary algorithm is the simplest algorithm that searches for an element very quickly. It is used to search for the element in the sorted list. The elements must be stored in sequential order or a sorted manner to implement the binary algorithm. Binary search cannot be implemented if the elements are stored randomly. It is used to find the middle element of the list.

Sorting Algorithms

[Sorting algorithms](#) are used to rearrange the elements in an array or a given data structure, either in ascending or descending order. The comparison operator decides the new order of the elements.

Why do we need a sorting algorithm?

1. An efficient sorting algorithm is required for optimizing the efficiency of other algorithms, like the binary search algorithm, as a binary search algorithm requires an array to be sorted in a particular order, mainly in ascending order.
2. It produces information in a sorted order, which is a human-readable format.
3. Searching for a particular element in a sorted list is faster than in an unsorted list.

How to express an algorithm?

- **Natural Language:** The algorithm is expressed in natural English. It is too difficult to comprehend the algorithm from it.
- **Flowchart:** The algorithm is expressed by making a pictorial/graphical representation of it. It is easier to understand as compared to natural language.
- **Pseudocode:** The algorithm is expressed in the form of informative and annotation text written in plain English, which is similar to the actual code, but it has no syntax like any of the programming languages.

It cannot be interpreted or compiled by the computer. It is the best way to express an algorithm. This is because it can be understood even by a layman with some school-level knowledge.

Example

Let's take an example of adding two numbers. We will express its algorithm in natural language, flowchart, and pseudocode.

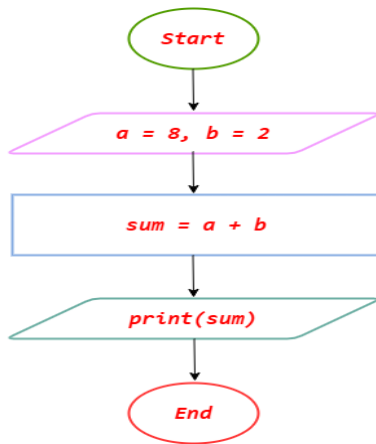
Natural Language

In simple terms, adding two numbers follows a straightforward process:

1. Ask the user for the first number (a).
2. Ask the user for the second number (b).
3. Perform the addition by summing both numbers (a+b).

4. Show the result (sum) to the user.

Flowchart



Pseudocode

1. BEGIN
2. PRINT "Enter first number:"
3. READ A
4. PRINT "Enter second number:"
5. READ B
6. $SUM \leftarrow A + B$
7. PRINT "The sum is:", SUM
8. END

Advantages of Algorithms

- It is easy to understand.
- An algorithm provides a step-by-step representation of a solution to a given problem, making it easy to grasp.
- In an algorithm, the problem is divided into smaller steps or components. Therefore, it becomes easier for a programmer to write an actual program based on the algorithm.

Disadvantages of Algorithms

- Writing an algorithm can be a time-consuming process. Since writing an algorithm takes a long time, it is indeed time-consuming.
- Understanding complex logic with the help of algorithms isn't easy.
- Displaying looping and branching statements in an algorithm is challenging.

4. What is Asymptotic Notation (Data Structure)?

Asymptotic Notations are mathematical tools that are used for analyzing the performance of algorithms by comprehending the changes in their working efficiency as the size of input varies. These notations give a clear and concise way of expressing the behaviour of an algorithm's time or space complexity as the size of input reaches infinity.

Instead of directly comparing the algorithms, the focus of asymptotic analysis is on understanding the relative rates of growth of complexities of algorithms. The comparison of algorithms is done by abstracting away the details of machine-specific constants and implementation and focusing on fundamental trends.

How to find the Time Complexity or running time for performing the operations?

The measuring of the actual running time is not practical at all. The running time to perform any operation depends on the size of the input. Let's understand this statement through a simple example.

Suppose we have an array of five elements, and we want to add a new element at the beginning of the array. To achieve this, we need to shift each element towards the right and suppose each element takes one unit of time. There are five elements, so five

units of time would be taken. Suppose there are 1000 elements in an array; then it takes 1000 units of time to shift. It concludes that time complexity depends upon the input size.

Therefore, if the input size is n , then $f(n)$ is a function of n that denotes the time complexity.

How to calculate $f(n)$?

Calculating the value of $f(n)$ for smaller programs is easy but for bigger programs, it's not that easy. We can compare the data structures by comparing their $f(n)$ values. We can compare the data structures by comparing their $f(n)$ values. We will find the growth rate of $f(n)$ because there might be a possibility that one data structure for a smaller input size is better than the other one but not for the larger sizes. Now, how to find $f(n)$.

Let's look at a simple example.

$$f(n) = 5n^2 + 6n + 12$$

where n is the number of instructions executed, and it depends on the size of the input.

When $n=1$

$$\% \text{ of running time due to } 5n^2 = (5/5+6+12) * 100 = 21.74\%$$

$$\% \text{ of running time due to } 6n = (6/5+6+12) * 100 = 26.09\%$$

$$\% \text{ of running time due to } 12 = (12/5+6+12) * 100 = 52.17\%$$

From the above calculation, it is observed that most of the time is taken by 12. But we have to find the growth rate of $f(n)$, we cannot say that the maximum amount of time is taken by 12. Let's assume the different values of n to find the growth rate of $f(n)$.

n	$5n^2$	$6n$	12
1	21.74%	26.09%	52.17%
10	87.41%	10.49%	2.09%
100	98.79%	1.19%	0.02%
1000	99.88%	0.12%	0.0002%

As we can observe in the above table, with the increase in the value of n , the running time of $5n^2$ increases while the running time of $6n$ and 12 also decreases. Therefore, we observed that for larger values of n , the squared term consumes almost 99% of the time as the n^2 term contributes most of the time, so we can eliminate the rest two terms.

Therefore,

$$f(n) = 5n^2$$

Here, we are getting the approximate time complexity whose result is very close to the actual result. This approximate measure of time complexity is known as an Asymptotic complexity. Here, we are not calculating the exact running time; we are eliminating the unnecessary terms, and we are just considering the term that is taking most of the time.

In mathematical analysis, asymptotic analysis of an algorithm is a method of defining the mathematical foundation of its run-time performance. Using the asymptotic analysis, we can easily conclude the average-case, best-case and worst-case scenario of an algorithm.

It is used to mathematically calculate the running time of any operation inside an algorithm.

Example: The running time of one operation is $x(n)$, and for another operation, it is calculated as $f(n^2)$. It refers to running time will increase linearly with an increase in ' n ' for the first operation, and running time will increase exponentially for the second operation. Similarly, the running time of both operations will be the same if n is significantly small.

Usually, the time required by an algorithm comes under three types:

Worst case: It defines the input for which the algorithm takes a huge amount of time.

Average case: It takes an average time for the program to execute.

Best case: It defines the input for which the algorithm takes the lowest time

Asymptotic Notations

The commonly used asymptotic notations used for calculating the running time complexity of an algorithm are given below:

- Big O Notation (O)
- Omega Notation (Ω)
- Theta Notation (θ)

Big O Notation (O)

- Big O notation is an asymptotic notation that measures the performance of an algorithm by simply providing the order of growth of the function.
- This notation provides an upper bound on a function, which ensures that the function never grows faster than the upper bound. So, it gives the least upper bound on a function so that the function never grows faster than this upper bound.

Omega Notation (Ω)

- It basically describes the best-case scenario which is opposite to the big o notation.
- It is the formal way to represent the lower bound of an algorithm's running time. It measures the best amount of time an algorithm can take to complete or the best-case time complexity.
- It determines what is the fastest time that an algorithm can run.

Theta Notation (θ)

- The theta notation mainly describes the average case scenarios.
- It represents the realistic time complexity of an algorithm. Every time an algorithm does not perform worst or best in real-world problems, algorithms mainly fluctuate between the worst-case and best-case, and this gives us the average case of the algorithm.
- Big theta is mainly used when the value of the worst-case and the best-case is the same.
- It is the formal way to express both the upper bound and lower bound of an algorithm's running time.

Common Asymptotic Notations

constant	$O(1)$
linear	$O(n)$
logarithmic	$O(\log n)$
$n \log n$	$O(n \log n)$
exponential	$2^{O(n)}$
cubic	$O(n^3)$
polynomial	$n^{O(1)}$
quadratic	$O(n^2)$

Difference Between Asymptotic Notations

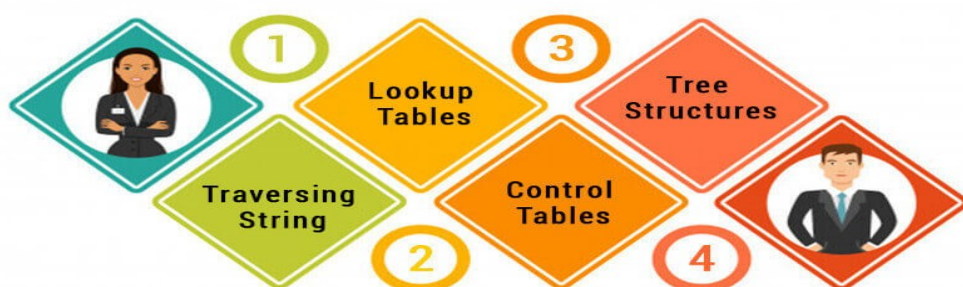
Aspect	Big-O Notation (O)	Omega Notation (Ω)	Theta Notation (Θ)
Meaning	Upper Bound (maximum growth rate)	Lower Bound (minimum growth rate)	Tight Bound (exact growth rate)
Represents	Worst-case performance	Best-case performance	Average-case / realistic performance
Symbol	$O(g(n))$	$\Omega(g(n))$	$\Theta(g(n))$
Usefulness	Ensures algorithm will never grow faster than	Ensures algorithm will never grow slower than	Ensures algorithm will always grow within these

	this.	this.	bounds.
Focus	Guarantees upper performance limit.	Guarantees lower performance limit.	Guarantees tight performance limit.
Example	Linear Search $\rightarrow O(n)$ in worst case.	Linear Search $\rightarrow \Omega(1)$ in best case.	Linear Search $\rightarrow \Theta(n)$ in average case.
Advantage	Predicts maximum running time.	Predicts guaranteed minimum time.	Gives realistic estimate.
Limitation	May overestimate performance.	May underestimate performance.	Not always applicable (upper $\neq$ lower bounds).

5. Pointers in Data Structure

A variable that stores the address of another variable is known as a pointer. Normal variables store the value. However, pointers hold memory locations in which values are stored. Pointers allow us to change and access the system memory directly. Pointer improves the performance for repetitive processes.

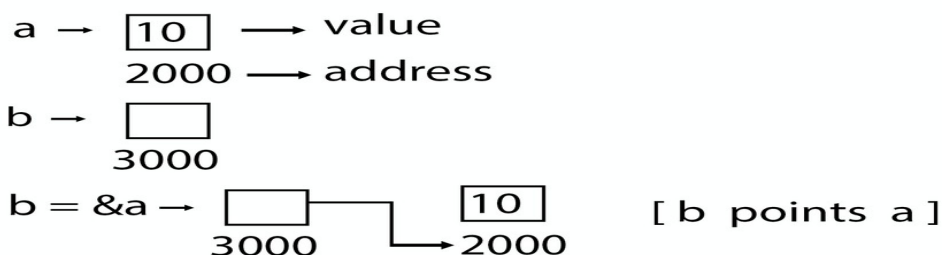
Pointer Improves Performance for



Languages such as C and C++ use pointers a lot. However, other languages can also have pointers, though they may not be the same.

Pointer Example

In the following figure, the variable *a* holds the value 10 at address 2000, and the variable *b* holds nothing at address 3000. But in the next statement (*b*=&*a*) *b* points to the address of *a*, which means the variable *b* holds the address of *a*, i.e., 2000.



Why do We Need Pointers in Data structures?

The pointer helps to reduce the time needed by an algorithm to copy data from one location to another. Therefore, pointer uses the memory locations directly; any change made to the value will be reflected at all the locations. For example,

- **Call-by-value:** needs the value of arguments to be copied every time any operation needs to be performed.
- **Call-by-reference:** makes this task easier by using its memory location to update the value at the memory location.
- **Control Program Flow:** Pointers can also be used to control program flow. This is achieved through control tables that utilize these pointers. The pointers

are stored in a table, pointing to the entry point of each subroutine, which is then executed one after the other.

- **Dynamic Memory Allocation:** Most programming languages use dynamic memory allocation for run-time variables. This type of memory allocation relies on the heap instead of the stack, which uses pointers. Pointers store the address of these dynamically created data blocks or arrays of objects.
- **Dereferencing:** To access the value stored at the location is known as dereferencing the pointer using an operator (e.g., * in C/C++). This retrieves the content at the pointed-to location.

Types of Pointers

There are various types of pointers that are used in programming; each is used for different purposes.

1. Basic Pointer
2. Null Pointer
3. Pointer to Pointer (Double Pointer)
4. Smart Pointer
5. Void Pointer
6. Integer Pointer
7. Float Pointer
8. Array Pointer
9. Function Pointer
10. Pointer to Structure (Struct Pointer)
11. Wild Pointer
12. Dangling Pointer

1. Basic Pointer

A basic or primary pointer is a variable that keeps the memory address of another variable. A **primary pointer** is a **variable** that is a memory address of another variable. Instead of having the data itself physically present, pointers only connect to the memory location where the data is kept. Therefore, the program will get a chance to use it to read and write with the value located at the given **memory address**.

Implementation in C++

Example

```
#include <iostream>
#include <cstdlib>
#include <cstdlib>
using namespace std;

int main(int argc, char** argv)
{
    int ip;
    ip = atoi(argv[1]);
    int* p;
    p = &ip;
    cout << "Value of ip :- " << ip << endl;
    cout << "Address of ip :- " << &ip << endl;
    cout << "Value of pointer (Address it holds) :- " << p << endl;
    cout << "Value pointed to by pointer :- " << *p << endl;
    return 0;
}
```

Output:

./basic_pointer 19

Value of ip :- 19

Address of ip :- 0x7ffd87aef6dc

Value of pointer (Address it holds) :- 0x7ffd87aef6dc

Value pointed to by pointer :- 19

2. Null Pointer

A pointer that does not point to a legitimate memory location. It is used in many programming languages where the pointer does not point to an object, or one can say it is not initialized.

Implementation in C++

Example

```
#include <iostream>
#include <cstdio>
#include <cstdlib>
using namespace std;

int main()
{
    int *p = nullptr;
    if(p == nullptr)
    {
        cout << "This is null pointer." << endl;
    }
    return 0;
}
```

Output:

```
./null_pointer
This is null pointer.
```

3. Pointer to Pointer (Double Pointer)

A pointer to a pointer is also known as a **double pointer**. It is a type of pointer that keeps the memory address of another pointer. Data can be indirectly referenced using a double pointer if required.

Implementation in C++

Example

```
#include <iostream>
#include <cstdlib>
#include <cstdio>
using namespace std;

int main(int argc, char** argv)
{
    int x = atoi(argv[1]);
    int* p = &x;
    int** pp = &p;

    cout << "Value of x: " << x << endl;
    cout << "Address of x: " << &x << endl;

    cout << "Value of p (address of x): " << p << endl;
    cout << "Value pointed by p (*p): " << *p << endl;

    cout << "Value of pp (address of p): " << pp << endl;
    cout << "Value pointed by pp (*pp, which is p): " << *pp << endl;
    cout << "Value pointed by the pointer pointed by pp (**pp, which is x): " << **pp
    << endl;
}
```

```

    **pp = 99;
    cout << "New value of x after modifying through double pointer: " << x << endl;
    return 0;
}

```

Output:

```

./double_pointer 15
Value of x: 15
Address of x: 0x7ffd9cfae264
Value of p (address of x): 0x7ffd9cfae264
Value pointed by p (*p): 15
Value of pp (address of p): 0x7ffd9cfae268
Value pointed by pp (*pp, which is p): 0x7ffd9cfae264
Value pointed by the pointer pointed by pp (**pp, which is x): 15
New value of x after modifying through double pointer: 99

```

4. Smart Pointers

A **smart pointer** acts as a pointer and is a wrapper class over a pointer. These pointers automatically manage the memory they point to. It makes sure that the deallocation of memory is done properly when it is no longer needed. It is available in C++.

Implementation in C++

Example

```

#include <iostream>
#include <memory>
using namespace std;

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    if (n <= 0) {
        cout << "Invalid size!" << endl;
        return 1;
    }
    // unique_ptr for dynamic array
    unique_ptr<int[]> arr = make_unique<int[]>(n);
    for (int i = 0; i < n; ++i) {
        cout << "Enter element " << i + 1 << ": ";
        cin >> arr[i];
    }
    cout << "Array elements are: ";
    for (int i = 0; i < n; ++i)
        cout << arr[i] << " ";
    cout << endl;
    // shared_ptr pointing to first element of array (without deleting it)
    shared_ptr<int> sp(arr.get(), [](int*){});
    cout << "Shared pointer value (first element): " << *sp << endl;
    shared_ptr<int> sp2 = sp;
    cout << "Use count of shared pointer: " << sp.use_count() << endl;
    // Modify first element via shared pointer
    *sp2 = 999;
    cout << "Modified array first element via shared pointer: " << arr[0] << endl;
    cout << "Array elements are: ";
    for (int i = 0; i < n; ++i)
        cout << arr[i] << " ";
}

```

```

    cout << endl;
    return 0;
}

```

Output:

```

./smart_pointer
Enter the number of elements: 5
Enter element 1: 1
Enter element 2: 3
Enter element 3: 4
Enter element 4: 5
Enter element 5: 6
Array elements are: 1 3 4 5 6
Shared pointer value (first element): 1
Use count of shared pointer: 2
Modified array first element via shared pointer: 999
Array elements are: 999 3 4 5 6

```

5. Void Pointers

A void pointer is a pointer that does not have any **associated data type**. It can store the address of any data type and can be typecasted to any type.

Implementation in C++

Example

```

#include <iostream>
#include <stdio.h>
#include <stdlib.h>
using namespace std;

int main(int argc, char** argv)
{
    float in = stof(argv[1]);
    void* vp;
    vp = &in;
    cout << "Value using void pointer in int :- " << static_cast<int>(*(float*)vp) <<
endl;
    cout << "Value using void pointer in float :- " << *(float*)vp << endl;
    return 0;
}

```

Output:

```

./void_pointer 3.14
Value using void pointer in int :- 3
Value using void pointer in float :- 3.14

```

6. Integer Pointer

As the name suggests, an integer pointer points to the address of the memory where an integer is stored.

Implementation in C++

Example

```

#include <iostream>
#include <stdio.h>
#include <stdlib.h>
using namespace std;

```

```

int main(int argc, char** argv)
{

```

```

int n = atoi(argv[1]);
int* p = &n;
cout << "Value of v: " << n << endl;
cout << "Value of n: " << &n << endl;
cout << "Value of p: " << p << endl;
cout << "Address of p: " << &p << endl;
cout << "The value pointed by p: " << *p << endl;
return 0;
}

```

Output:

```

./integer_pointer 10
Value of v: 10
Value of n: 0x7fff0fa17e9c
Value of p: 0x7fff0fa17e9c
Address of p: 0x7fff0fa17ea0
The value pointed by p: 10

```

7. Float Pointer

A float pointer is a pointer that points to the address of the memory where a float is stored.

Implementation in C++

Example

```

#include <iostream>
#include <stdio.h>
#include <stdlib.h>
using namespace std;

int main(int argc, char** argv)
{
    float fl = stof(argv[1]);
    float* fp = &fl;
    int* it = reinterpret_cast<int*>(&fl); //Only for Education
    cout << "Value of fl: " << fl << endl;
    cout << "Value of fp: " << fp << endl;
    cout << "Value of it: " << it << endl;
    cout << "The value pointed by fp: " << *fp << endl;
    cout << "The value pointed by it: " << (*it) << endl;
    return 0;
}

```

Output:

```

./float_pointer 10.34
Value of fl: 10.34
Value of fp: 0x7ffde5efad3c
Value of it: 0x7ffde5efad3c
The value pointed by fp: 10.34
The value pointed by it: 1092972708

```

8. Array Pointer

An array pointer is a type of pointer that can be used to point to the first element of the array. Along with storing the memory address of the first element of the associated array, the array pointer is also tied to the element type of the array.

Implementation in C++

Example

```

#include <iostream>

```

```

#include <stdio.h>
#include <stdlib.h>
using namespace std;

int main(int argc, char** argv)
{
    int n = atoi(argv[1]);
    int arr[n];
    for(int i=0;i<n;i++)
    {
        cin>> arr[i];
    }
    cout << endl;
    int* ptr = arr;
    cout << "Elements of array:- ";
    for(int k=0;k<n;k++)
    {
        cout << *(ptr+k) << " ";
    }
    cout << endl;
    return 0;
}

```

Output:

```

./array_pointer 5
2 1 3 45 7
Elements of array:- 2 1 3 45 7

```

9. Function Pointer

A function pointer is a pointer that keeps the address of a function. Using a function pointer, one can call the function to which the function points. This is because the function pointer has the address of the function.

Implementation in C++

Example

```

#include <iostream>
#include <stdio.h>
#include <stdlib.h>
using namespace std;

int add(int a, int b)
{
    return a+b;
}

int main(int argc, char** argv)
{
    int n = atoi(argv[1]);
    int m = atoi(argv[2]);
    int (*fp)(int, int);
    fp = add;
    cout << "Output of function ptr:- " << fp(n, m) << endl;
    return 0;
}

```

Output:

```

./function_pointer 5 7

```

10. Pointer to Structure (Struct Pointer)

A struct pointer is a pointer that keeps the structure address. Struct pointers are necessary for dynamic memory allocation. Function parameters are passed to big structures with less overhead.

Implementation in C++

Example

```
#include <iostream>
#include <stdlib.h>
#include <string>
using namespace std;

struct data {
    int id;
    string name;
    int age;
};

int main(int argc, char** argv)
{
    int in = atoi(argv[1]);
    // Allocate array of structures
    data* p = new data[in];
    int argIndex = 2;
    for (int i = 0; i < in; i++)
    {
        p[i].id = atoi(argv[argIndex++]);
        p[i].name = argv[argIndex++];
        p[i].age = atoi(argv[argIndex++]);
    }
    for (int k = 0; k < in; k++)
    {
        cout << "data id:- " << p[k].id << endl;
        cout << "data name:- " << p[k].name << endl;
        cout << "data age:- " << p[k].age << endl;
        cout << endl;
    }
    delete[] p;
    return 0;
}
```

Output:

```
./structure_pointer 3 1 "help1" 11 2 "help2" 22 3 "help3" 33
```

```
data id:- 1
```

```
data name:- help1
```

```
data age:- 11
```

```
data id:- 2
```

```
data name:- help2
```

```
data age:- 22
```

```
data id:- 3
```

```
data name:- help3
```

```
data age:- 33
```

11. WILD Pointer

It is a type of pointer that does not hold the address of any variable.

A pointer that is **declared but not initialized** and points to an unknown memory location.

How to avoid wild pointers?

- Initialize the pointer when declaring.
- Set the unused pointer to null.
- Use dynamic memory allocation carefully.

Implementation in C++

Example

```
#include <iostream>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    printf("Line - %d\n", __LINE__);
```

```
    int* ptr;
```

```
    printf("Line - %d\n", __LINE__);
```

```
    *ptr = 11;
```

```
    printf("Line - %d\n", __LINE__);
```

```
    cout << "ptr value:- " << *ptr << endl;
```

```
    printf("Line - %d\n", __LINE__);
```

```
    return 0;
```

```
}
```

Output:

```
./wild_pointer
```

```
Line - 9
```

```
Line - 11
```

```
Segmentation fault (core dumped)
```

12. Dangling Pointer

A **dangling pointer** is a pointer that continues to reference a memory location after the memory has been **freed or gone out of scope**. Accessing such a pointer leads to undefined behavior—anything from garbage values to program crashes.

Causes of Dangling Pointer

- Memory deallocation
- The returning address of a local variable
- Variable going out of scope

How to avoid dangling pointers?

- Set freed pointers to null
- Avoid returning addresses of local variables
- Use static variables if needed:

Implementation in C++

Example

```
#include <iostream>
```

```
#include <stdio.h>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int x = 0;
```

```

int* ptr = &x;
*ptr = 11;
delete ptr; // Memory freed
// ptr now becomes a dangling pointer
// Dangerous!
cout << "ptr value:- " << *ptr << endl;
return 0;
}

```

Output:

```

./dangling_pointer
free(): invalid pointer
Aborted (core dumped)

```

Note: Languages like Python, Java, and JavaScript do not have direct support for pointers, but they use similar concepts like objects and references.

Applications of Pointers

Pointers are essential for various reasons:

- **Dynamic Memory Allocation:** In languages like C++ and C, dynamic memory allocation on the heap is done by pointers. This is required for creating data structures like graphs, trees, and linked lists.
- **Efficiency:** Pointers enhance code efficiency. One can avoid copying large data by passing pointers to functions.
- **Function Pointers:** Pointers can even point to functions. This allows for dynamic function callbacks and calls.

Advantages of Pointers

- **Efficiency:** Pointers can access memory locations directly, allowing for efficient memory management.
- **Dynamic Memory Allocation:** Pointers allow dynamic memory allocation. Thus, memory can be allocated in the programs as needed during runtime.
- **Passing Parameters:** Pointers allow functions to change variables outside of their scope by sending memory addresses instead of values. It is beneficial for functions that require changing variables or returning multiple values.
- **Reduced Data Duplicity:** Instead of copying large data, pointers can point to the same data in memory. Thus, it avoids duplication and conserves memory.

Disadvantages of Pointers

- **Complexity:** Introducing pointers can increase complexity and can also make code harder to maintain and comprehend.
- **Memory Management:** Dealing with pointers requires memory management to be handled manually in languages like C++ and C. This can lead to Segmentation Faults or memory leaks (unreleased memory).
- **Security Vulnerabilities:** We know that pointers have direct access to memory addresses. Therefore, it may introduce security issues in the program if not handled properly. It can also lead to the program crashing.

Important Points

- **Pointer arithmetic:** Four arithmetic operators can be used with pointers: ++, --, +, and -.
- **Array of pointers:** We can define arrays to hold a number of pointers.
- **Pointer to pointer:** C allows us to have a pointer to a pointer and so on.
- **Passing pointers to functions in C:** Passing an argument by reference or by address enables the passed argument to be changed in the calling function by the called function.
- **Return pointer from functions in C:** C allows a function to return a pointer to the local variable, static variable, and dynamically allocated memory as well.

Conclusion

Pointers in programming are useful tools that keep the memory addresses of variables. Different types of pointers, like Null, Void, etc., have unique uses and properties. They play an important role in memory management, making programs more flexible and efficient. On the other hand, the use of pointers is also risky, as the incorrect use of them can lead to crashes and errors.

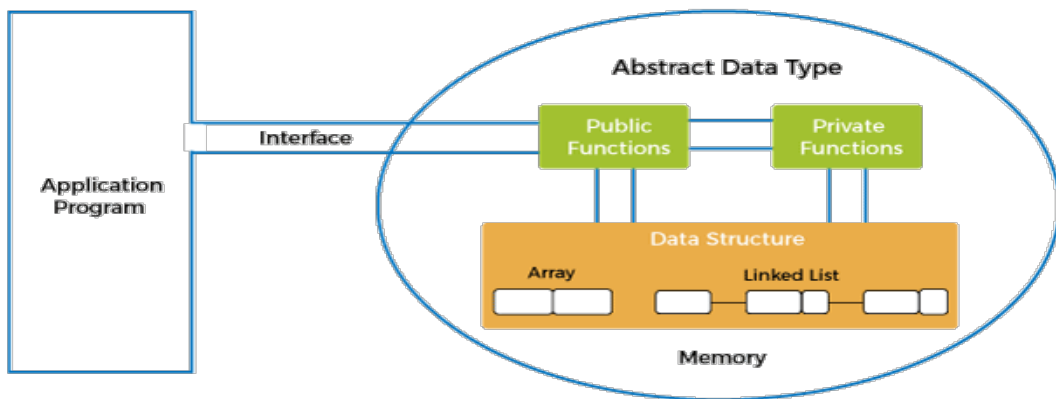
6. Abstract Data Type (ADT) in Data Structure

An **abstract data type** is an abstraction of a data structure that provides only the interface to which the data structure must adhere. The interface does not give any specific details about something that should be implemented or in what programming language.

In other words, we can say that abstract data types are the entities that are definitions of data and operations, but do not have implementation details. In this case, we know the data that we are storing and the operations that can be performed on the data, but we don't know about the implementation details. The reason for not having implementation details is that every programming language has a different implementation strategy. For example, a C data structure is implemented using structures, while a C++ data structure is implemented using objects and classes.

For example, a List is an abstract data type that is implemented using a dynamic array and a linked list. A queue is implemented using a linked list-based queue, an array-based queue, and a stack-based queue. A Map is implemented using a TreeMap, HashMap, or HashTable.

Abstract Data Type Model



The above figure shows the ADT model. There are two types of models in the ADT model, i.e., the public function and the private function. The ADT model also contains the data structures that we are using in a program. In this model, the first encapsulation is performed, i.e., all the data is wrapped in a single unit, i.e., ADT. Then, the abstraction is performed means showing the operations that can be performed on the data structure and the data structures that we are using in a program.

Let's understand the abstract data type with a real-world example.

If we consider the smartphone. We look at the high specifications of the smartphone, such as:

- 4 GB RAM
- Snapdragon 2.2ghz processor
- 5-inch LCD screen
- Dual camera
- Android 8.0

The above specifications of the smartphone are the data, and we can also perform the following operations on the smartphone:

- **call()**: We can call through the smartphone.
- **text()**: We can text a message.

- **photo():** We can click a photo.
- **video():** We can also make a video.

The smartphone is an entity whose data or specifications, and operations are given above. The abstract/logical view and operations are the abstract or logical views of a smartphone.

The implementation view of the above abstract/logical view is given below:

```

1. class Smartphone
2. {
3.     private:
4.         int ramSize;
5.         string processorName;
6.         float screenSize;
7.         int cameraCount;
8.         string androidVersion;
9.     public:
10.        void call();
11.        void text();
12.        void photo();
13.        void video();
14.}
```

The above code is the implementation of the specifications and operations that can be performed on the smartphone. The implementation view can differ because the syntax of programming languages is different, but the abstract/logical view of the data structure would remain the same. Therefore, we can say that the abstract/logical view is independent of the implementation view.

Note: We are aware of the operations that can be performed on predefined data types, such as int, float, char, etc., but we do not know the implementation details of these data types. Therefore, we can state that an abstract data type is considered a hidden box that conceals all the internal details of the data type.

Data Structure Example

Suppose we have an index array of size 4. We have an index location starting from 0, 1, 2, 3. An array is a data structure where the elements are stored in a contiguous location. The memory address of the first element is 1000, the second element is 1004, the third element is 1008, and the fourth element is 1012. Since it is of integer type so it will occupy 4 bytes, and the difference between the addresses of each element is 4 bytes. The values stored in an array are 10, 20, 30 and 40. These values, index positions and the memory addresses are the implementations.

The abstract or logical view of the integer array can be stated as:

- It stores a set of elements of integer type.
- It reads the elements by position, i.e., index.
- It modifies the elements by index
- It performs sorting

The implementation view of the integer array:

```

1. a[4]={10,20,30,40}
2. cout<<a[2]
3. a[3]=50
```

Common ADTs

Now, let's understand three common ADTs: List, Stack, and Queue ADTs.

1. List ADT

The List ADT (Abstract Data Type) is a collection of elements in a sequential manner that provides a set of operations **without giving details of the internal implementation**. It provides an ordered way to store, access, and modify data.

12	22	32	42	52	62
----	----	----	----	----	----

View of List

Operations of List

The List ADT has to store the data sequentially and must have the following operations:

- **insert():** For inserting an element at any position in the given list.
- **get():** For returning an element at any given position in the list.
- **remove():** For removing any element that occurs first from the non-empty list.
- **removeAt():** For removing the element from a non-empty list at the specified location.
- **replace():** For replacing an element with another element at any position.
- **size():** For returning the number of elements in the list.
- **isEmpty():** For returning a true value if the list is empty; otherwise, return a false value.
- **isFull():** For returning a true value if the list is full; otherwise, return a false value. Only applied to the fixed-size implementations (for example, array-based lists).

2. Stack ADT

The stack ADT follows the LIFO (Last In, First Out) principle. It is a linear data structure. In a stack ADT, the addition and removal of elements is done from only one end, which is the top of the stack.

51
41
31
21
11

View of Stack

Operations of the Stack

The order of deletion and insertion should be as per the LIFO or FILO Principle in the stack ADT. It must also support the following operations:

- **push():** For inserting an element at one end of the stack, called the top.
- **pop():** For removing and returning the element at the top of the stack, provided the stack is not empty.
- **peek():** For returning the element at the top of the stack without removing it, provided the stack is not empty.
- **size():** For returning the total number of elements present in the stack.
- **isEmpty():** For returning a true value if the stack is empty; otherwise, return a false value.
- **isFull():** For returning a true value if the stack is full; otherwise, return a false value. It is only relevant for the fixed-capacity stacks (e.g., array-based).

3. Queue ADT

A linear data structure that follows the FIFO (First In, First Out) principle is the queue ADT. It allows elements to be removed at the other end(front) and inserted at one end (rear).

	11	21	31	41	51	61	
--	----	----	----	----	----	----	--

View of Queue

Operations of the Queue

The design of the queue ADT is similar to the stack ADT. It should give support to the following operations:

- **enqueue():** For inserting an element at the end of the queue.
- **dequeue():** For removing and returning the first element of the queue, provided the queue is not empty.
- **peek():** For returning the element of the queue without removing it, provided the queue is not empty.
- **size():** For returning the total number of elements present in the queue.
- **isEmpty():** For returning a true value if the queue is empty; otherwise, return a false value.

Why Use ADTs?

The reasons for using ADTs in Java are mentioned below:

- **Encapsulation:** Hiding details of complex implementation behind a clean interface.
- **Reusability:** Allowing various internal implementations (e.g., linked list or array) without changing external usage.
- **Modularity:** Simplifying updates and maintenance by the separation of logic.
- **Security:** Protecting data by stopping direct access and minimizing unintended changes and bugs.

Advantages of ADTs

- **Encapsulation:** ADTs facilitate a way for encapsulating data and operations into a single unit. Hence, making it easier to modify and manage the data structure.
- **Abstraction:** ADTs allow users to use data structures without knowing the implementation details, which can do simplification programming and reduction in errors.
- **Data Structure Independence:** Implementation of ADTs can be done using different data structures. Thus, making it easier to adapt to the changing requirements and needs.
- **Information Hiding:** ADTs can protect the data integrity by taking control of access and stopping unauthorized modifications.
- **Modularity:** One ADT can be combined with the other ADT to generate more complex data structures, which can increase modularity and flexibility in programming.

Disadvantages of ADTs

- **Overhead:** Implementation of ADTs can put some overhead in terms of processing and memory, which can affect performance.
- **Complexity:** Implementation of ADTs can be complex, especially for complex and large data structures.
- **Learning Curve:** To use ADTs, there is a requirement for knowledge of their implementation and usage, which can take time and effort to learn.
- **Limited Flexibility:** There can be limitations in the functionality of some ADTs, or they may not be apt for all types of data structures.
- **Cost:** Implementation of ADTs may require additional investment and resources, which can increase the development cost.

Conclusion

ADTs are a basic concept in object-oriented programming and data structures. ADTs allow developers to create more maintainable, robust, and reusable software. They promote modularity, abstraction, and flexibility. Hence, it makes it easier to solve complex problems.

- **DS Array**

- 1. **Array in Data Structure**

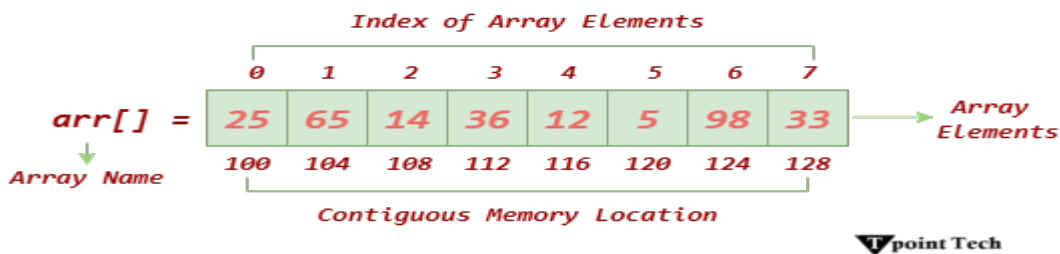
Arrays are a type of data structure that is a collection of data items of similar kinds kept at contiguous memory locations. It is one of the simplest data structures that is used in computer programming. In an array, each data element can be accessed randomly with the help of its index number.

Basic Terminologies

- **Array Element:** Items of an array are known as its elements, which are kept in an array and can be accessed randomly with the help of their index number.
- **Array Index:** In an array, elements are recognized by their indices. Array index begins from 0. In other words, zero-based indexing occurs in an array.
- **Array Length:** The number of elements in an array determines the array length.
- **Contiguous Memory:** Array elements are stored back-to-back in memory for fast access

Memory Representation of Array

All the elements are kept in contiguous memory locations in an array. So, if we do the initialization of an array, the elements of the array will be kept sequentially in memory. It allows for efficient manipulation and access of elements.



Memory Calculation

It is the **total memory** (physical size) used by the array. The total memory occupied by an array can be calculated by using the following formula:

1. Total Memory = Number of Elements × Size of Each Element

Note: The size of array elements depends on the data type. For example, if array is of type int, then each element is 4 bytes.

If an array has 5 elements of type int, then total memory occupied by an array will be 20 bytes because char has 4 bytes.

Similarly, if an array has 100 elements of type char, then total memory occupied by an array will be 200 bytes because char has 2 bytes.

Size or Length of Array

It is the **count of elements** (logical size) the array holds. For example, if an array holds 10 elements, the size or length of array will be 10.

Array Indexing

1. **0 (zero-based indexing):** The first element of the array will be `arr[0]`.
2. **1 (one-based indexing):** The first element of the array will be `arr[1]`.
3. **n (n - n-based indexing):** The first element of the array can reside at any random index number.

Declaration of Array

The declaration of arrays can be done in different ways in different languages. For better understanding, the following are some array declarations in other languages.

Implementation in C++

1. //The integer type element is kept in this array
2. `int arr[4];` //array is of size 4
3. //The char type element is kept in this array
4. `char arr[9];` //array is of size 9
5. //The float type element is kept in this array
6. `float arr[15];` // array is of size 15

Initialization of Array

Initialization of arrays can be done in different ways in different languages. For better understanding, the following are some array initializations in other languages.

Implementation in C++

1. `int arr[]={11,12,13,14,15};` //integer type
2. `char arr[5]={'d','e','f','g','h'};` //character type
3. `float arr[5]={2.4f,3.0f,23.0f,6.0f,7.0f};` //float type

Why do we Need Arrays?

Suppose we have to keep track of the marks of a subject (say English) of a class (say class 10th). Also, assume there are 30 students in the 10th class.

Then, to keep track of the elements, we have to make 30 variables, one each for the student.

1. `Class MarksManagementEnglish`
2. `{`
3. `int marksOfStu1;`
4. `int marksOfStu2;`
5. `int marksOfStu3;`
6. `.....`
7. `int marksOfStu30;`
8. `}`

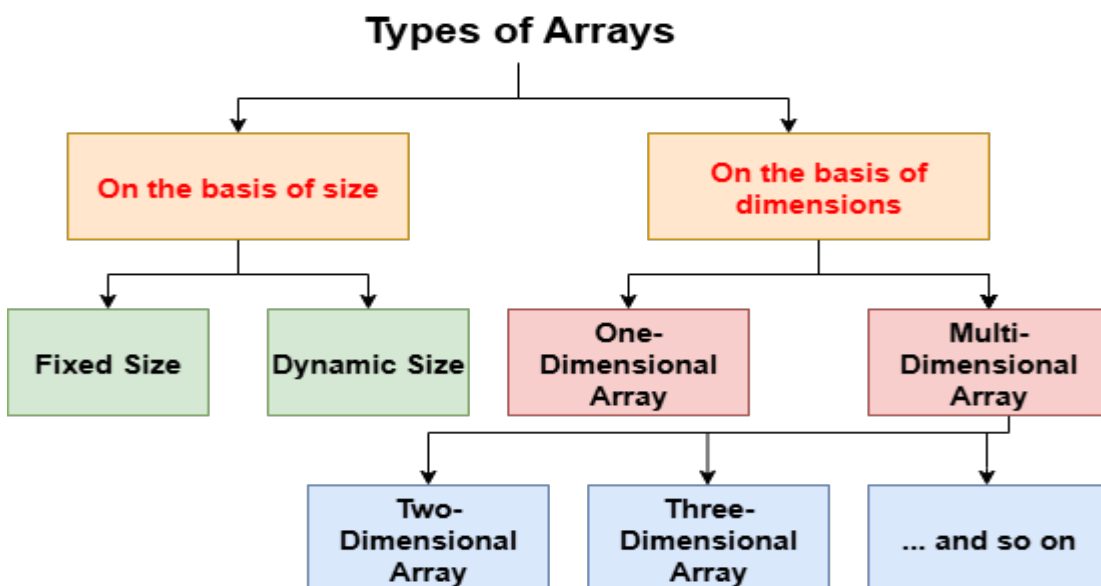
We see that we have to write code for 30 variables, which not only takes more time and writing space, but it is also difficult to manage 30 variables. What it means is that, if we want to store a large number of instances, then it becomes a tedious task to manage them with 30 normal variables. Here, an array can become very handy.

1. `Class MarksManagementEnglish`
2. `{`
3. `int marksEngArr[30];` // an array of size 30
4. `}`

Compare this code snippet with the previous one; we see that we have written just one line instead of 30 lines in the last code snippet. It not only reduces the writing time, but also makes it easy to manage. Here, each element of the array represents the marks of the subject English of each student.

Types of Arrays

Arrays can be classified based on its **size** and **dimension**



Types of Arrays: Based on Size

1. Fixed-Sized Array

Fixed-sized arrays are those arrays whose size cannot be changed. The drawback in these arrays is that in case one doesn't know the array size and declares a larger-sized

array and uses a smaller number of array elements, then it will result in a memory wastage, or if an array is declared of a lesser size, then there might be a scarcity of space to keep all of the elements. In such cases, it is not preferred to have a static memory allocation.

Implementation in C++

1. // A way of creating a fixed-sized array.
2. // Here, the memory allocation is done at compile-time.
3. `int ar1[4];` // size of the array is 4
4. // Another way (Array creation as well as initialization both)
5. `int ar2[4] = {1, 2, 4, 3};` // size of the array is 4
6. // Another way of creating a fixed-sized array
7. // Here, allocation of memory is done at run-time (It is also called dynamically allocated arrays)
8. `int *arr = new int[4];` // size of the array is 4

2. Dynamic-Sized Array

In these arrays, allocation and deallocation of memory are mostly done dynamically. The array size changes as per the requirements of the user during the execution of the code, so programmers do not have to worry about array sizes. Addition and removal of the elements can be done as per the requirement.

Implementation in C++

1. `#include<vector>`
- 2.
3. // The following array is a dynamic array. It is of type Integer.
4. `vector<int> vec;`

Types of Arrays: Based on Dimensions

1. **One-dimensional Array (1D Array):** One can imagine a one-dimensional array as a single row, where elements are stored sequentially, one after the other. It has a single dimension.
2. **Multi-dimensional Array:** An array containing more than one dimension is called a two-dimensional array. Data in the form of tables can be stored in a multi-dimensional array. In a multi-dimensional array, we can have 2-d arrays, 3-d arrays, and so on.

Array Operations

In an array, we can perform the following operations:

Operation	Description
Traversal	Visiting each element (for printing or processing)
Insertion	Adding an element (typically at end; at index requires shifting)
Deletion	Removing an element (may require shifting others)
Searching	Finding a value (linear or binary search)
Sorting	Rearranging elements in order (various algorithms: Bubble, Merge, etc.)
Updating	Changing the value at a specific index

1. Array Traversal

Sequentially traversing the array to access and process elements is referred to as array traversal. Array traversal is one of the most basic operations in computer programming. This is because arrays are commonly and frequently used data structures for keeping more than one element in a single variable. In array traversal, each element is accessed from start to end, or sometimes in reverse order (i.e., from end to start). Array traversal is usually done with the help of a loop.

Working of Array Traversal

A contiguous block of memory is allocated whenever an array is created, where elements are stored in an indexed manner. In an array, each element can be accessed with the help of its index, which begins from 0 in most of the computer programming languages.

For example, observe the following array containing six integers:

```
1. arr = [10, 30, 50, 70, 90]
```

Here, 10 is located at the 0th index, and can be accessed by arr[0]. 30 is located at the 1st index, and can be accessed by arr[1]. 50 is located at the 2nd index, and can be accessed by arr[2]. Similarly, the last element 90 is located at the index 5, and can be accessed by arr[5].

Types of Array Traversal

Linear (Sequential) Traversal: It involves traversing the elements of the array from beginning to end. It is the most common array traversal, and is used for searching elements, printing elements, and doing calculations.

Implementation in C++

Example

```
#include <iostream>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
using namespace std;
```

```
int main(int argc, char** argv)
{
    int n = atoi(argv[1]);
    int arr[n];
    for(int i=0;i<n;i++)
    {
        cin >> arr[i];
    }
    cout << "Elements of the array: ";
    for(int j=0;j<n;j++)
    {
        cout << arr[j] << " ";
    }
    cout << endl;
    return 0;
}
```

Output:

```
./array_traversal 5
```

```
10 30 50 70 90
```

```
Elements of the array: 10 30 50 70 90
```

Reverse Traversal: It involves traversing the elements of the array from beginning to end. It is used for processing elements in the reverse order.

Implementation in C++

Example

```
#include <iostream>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
using namespace std;
```

```
int main(int argc, char** argv)
{
    if(argc < 2)
    {
        cout << "Invalid Input.\n";
        return 0;
    }
}
```

```

int n = atoi(argv[1]);
int arr[n];
for(int i=0;i<n;i++)
{
    cin>>arr[i];
}
cout << "Array elements before reverse: ";
for(int i=0;i<n;i++)
{
    cout << arr[i] << " ";
}
cout << endl;
cout << "Array elements after reverse: ";
for(int i=n-1;i>=0;i--)
{
    cout << arr[i] << " ";
}
cout << endl;
return 0;
}

```

Output

```

./array_reversal 5
10 30 50 70 90
Array elements before reverse: 10 30 50 70 90
Array elements after reverse: 90 70 50 30 10

```

2. Insertion in Array

The process adding a new element in an array while maintaining the order of the other elements that are present in the array is referred to as insertion in array. Since arrays are kept in contiguous memory locations, therefore insertion of a new element requires shifting of the existing elements of the array.

Working of Insertion in an Array

Elements of array are present right after one another. Therefore, following happens when the insertion of a new element occurs in array.

1. **Spotting the position (index):** One must know where the new element is going to be inserted.
2. **Shifting the elements:** After finding the index, all the elements after the index must be shift to the immediately next memory location in order to make a space for the new element.
3. **Inserting the New Element:** Now, we have to put the new element at its appropriate location.

For example, observe the following array containing six integers:

1. arr = [40, 50, 60, 70, 80, 90]

If we have to insert element 55 at the index 2, then after insertion the updated array will be:

1. arr = [40, 50, 55, 60, 70, 80, 90]

We see that, elements, 60, 70, 80, and 90 are shifted right to make the space for the new element 55.

Types of Insertion

Inserting at the Index 0 (at the beginning): Every existing element of the array must be shift to right by one position. Note that this case is not efficient case as it affects all the existing elements of the array.

Example

```
#include <iostream>
```

```

#include <stdio.h>
#include <stdlib.h>
using namespace std;

int main(int argc, char** argv)
{
    if(argc < 2)
    {
        cout << "Invalid Input.\n";
        return 0;
    }
    int n = atoi(argv[1]);
    int size = n+1;
    int arr[size];
    for(int i=0;i<n;i++)
    {
        cin >> arr[i];
    }
    cout << "Array elements before insertion: ";
    for(int i=0;i<n;i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    int m;
    cout << "Enter new element to insert into array: ";
    cin >> m;
    for (int i = n; i > 0; i--)
    {
        arr[i] = arr[i - 1];
    }
    arr[0] = m;
    cout << "Array elements after insertion: ";
    for(int i=0;i<size;i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}

```

Output

```

./array_insert_at_beg 5
1 2 3 4 5
Array elements before insertion: 1 2 3 4 5
Enter new element to insert into array: 7
Array elements after insertion: 7 1 2 3 4 5

```

Insertion at a Specific Index: After insertion at a specific index in the array, all the existing elements after the index shift rightwards.

Example

```

#include <iostream>
#include <stdio.h>
#include <stdlib.h>

```

using namespace std;

```
int main(int argc, char** argv)
{
    if(argc < 2)
    {
        cout << "Invalid Input.\n";
        return 0;
    }
    int n = atoi(argv[1]);
    int s = n+1;
    int arr[s];
    for(int i=0;i<n;i++)
    {
        cin >> arr[i];
    }
    cout << "Array elements before insertion: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    int idx, ele;
    cout << "Enter index where want to add new element: ";
    cin >> idx;
    if (idx < 0 || idx > n)
    {
        cout << "Invalid index\n";
        return 0;
    }
    cout << "Enter new element to add to array: ";
    cin >> ele;
    for(int i=n;i>idx;i--)
    {
        arr[i] = arr[i-1];
    }
    arr[idx] = ele;
    cout << "Array elements after insertion: ";
    for (int i = 0; i < s; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}
```

Output

./array_insert_at_specific 5

1 2 3 4 5

Array elements before insertion: 1 2 3 4 5

Enter index where want to add new element: 3

Enter new element to add to array: 12

Array elements after insertion: 1 2 3 12 4 5

Insertion at the End: It is the simplest case, as shifting of elements is not required.

Example

```
#include <stdio.h>
#include <stdlib.h>
using namespace std;

int main(int argc, char** argv)
{
    if(argc < 2)
    {
        cout << "Invalid Input.\n";
        return 0;
    }
    int n = atoi(argv[1]);
    int s = n+1;
    int arr[s];
    for(int i=0;i<n;i++)
    {
        cin >> arr[i];
    }
    cout << "Array elements before insertion at end: ";
    for(int i=0;i<n;i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    int m;
    cout << "Enter new element to add to array: ";
    cin >> m;
    arr[n] = m;
    cout << "Array elements after insertion at end: ";
    for(int i=0;i<s;i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}
```

Output

```
./array_insert_at_end 5
1 2 3 4 5
Array elements before insertion at end: 1 2 3 4 5
Enter new element to add to array: 99
Array elements after insertion at end: 1 2 3 4 5 99
```

3. Deletion in Array

In an array, the deletion is the process of eliminating an element from a given position while maintaining the order of the rest of the elements. Eliminating an element from the array requires the shifting of elements to fill the gap.

1. **Spotting the position (index):** One must know from which index the element is going to be removed.
2. **Shifting the elements:** After the removal of the element, all the elements after the index (from where the deletion has occurred) must be shifted to the left.

3. **Updating the size (if required):** In case of dynamic memory allocation, the size is reduced after the deletion of elements.

For example, observe the following array containing six integers:

1. arr = [40, 50, 60, 70, 80, 90]

If we have to delete element 60 at index 2, then after the deletion, the updated array will be:

1. arr = [40, 50, 70, 80, 90]

We see that, elements, 70, 80, and 90 are shifted to left.

Types of Deletion

Deleting an Element at the Index 0 (at the beginning): Every existing element of the array must be shift to the left by one position.

Example

```
#include <iostream>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
using namespace std;
```

```
int main(int argc, char** argv)
```

```
{
```

```
    if(argc < 2)
```

```
    {
```

```
        cout << "Invalid Input.\n";
```

```
        return 0;
```

```
    }
```

```
    int n = atoi(argv[1]);
```

```
    int arr[n];
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        cin >> arr[i];
```

```
    }
```

```
    cout << "Array elements before delete at start: ";
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        cout << arr[i] << " ";
```

```
    }
```

```
    cout << endl;
```

```
    for(int i=0;i<n-1;i++)
```

```
    {
```

```
        arr[i] = arr[i+1];
```

```
    }
```

```
    n--;
```

```
    cout << "Array elements after delete at start: ";
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        cout << arr[i] << " ";
```

```
    }
```

```
    cout << endl;
```

```
    return 0;
```

```
}
```

Output

```
./array_delete_at_beg 5
```

```
3 4 5 6 7
```

```
Array elements before delete at start: 3 4 5 6 7
```

```
Array elements after delete at start: 4 5 6 7
```

Deletion at a Specific Index: After the deletion at a specific index in the array, all the existing elements after the index shift leftwards. In the following programs, we will consider zero-indexing for the deletion of an element.

Example

```
#include <iostream>
#include <stdio.h>
#include <stdlib.h>
using namespace std;

int main(int argc, char** argv)
{
    if(argc < 2)
    {
        cout << "Invalid Input.\n";
        return 0;
    }
    int n = atoi(argv[1]);
    int arr[n];
    for(int i=0;i<n;i++)
    {
        cin >> arr[i];
    }
    cout << "Array elements before remove element from specific index: ";
    for(int i=0;i<n;i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    int idx;
    cout << "Enter the index where want to remove element: ";
    cin >> idx;
    if(idx < 0 || idx > n)
    {
        cout << "Invalid Index no.\n";
        return 0;
    }

    for(int i=idx;i<n;i++)
    {
        arr[i] = arr[i+1];
    }
    n--;
    cout << "Array elements after remove element from specific index: ";
    for(int i=0;i<n;i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}
```

Output

```
./array_delete_at_specific 5
1 2 3 4 5
```

Array elements before remove element from specific index: 1 2 3 4 5

Enter the index where want to remove element: 2

Array elements after remove element from specific index: 1 2 4 5

Deletion at the End: Since the deletion is happening at the end, no element shift is required.

Example

```
#include <iostream>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
using namespace std;
```

```
int main(int argc, char** argv)
```

```
{
```

```
    if(argc < 2)
```

```
    {
```

```
        cout << "Invalid Input.\n";
```

```
        return 0;
```

```
    }
```

```
    int n = atoi(argv[1]);
```

```
    int arr[n];
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        cin >> arr[i];
```

```
    }
```

```
    cout << "Array elements before remove element from end: ";
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        cout << arr[i] << " ";
```

```
    }
```

```
    cout << endl;
```

```
    n--;
```

```
    cout << "Array elements after remove element from end: ";
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        cout << arr[i] << " ";
```

```
    }
```

```
    cout << endl;
```

```
    return 0;
```

```
}
```

Output

```
./array_delete_at_end 5
```

```
1 2 3 4 5
```

```
Array elements before remove element from end: 1 2 3 4 5
```

```
Array elements after remove element from end: 1 2 3 4
```

4. Searching

Searching in an array is the process of finding a specific element in a given array of elements. The task is to find out whether the element is present in the given array or not. If the element is found, return its position.

Types of Searching

1. Linear Search (Sequential Search)

Linear search, also known as sequential search, is the simplest searching algorithm. In linear search, the array is traversed generally using a loop. In each iteration, an element is compared with the target element.

If the element value matches the target value, the index of the element is returned; if not, then the next element is compared with the target element in the next iteration. In the worst-case scenario, each element of the input array is traversed only once, thus making the time complexity $O(n)$.

Example

```
#include <iostream>
#include <stdio.h>
#include <stdlib.h>
using namespace std;
```

```
int linear_search(int arr[], int size, int find)
{
    int out = -1;
    for(int i=0;i<size;i++)
    {
        if(arr[i] == find)
            return i;
    }
    return out;
}
```

```
int main(int argc, char** argv)
{
    if(argc < 2)
    {
        cout << "Invalid Input.\n";
        return 0;
    }
    int n = atoi(argv[1]);
    int arr[n];
    for(int i=0;i<n;i++)
    {
        cin >> arr[i];
    }
    cout << "Array elements are: ";
    for(int i=0;i<n;i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    int ele;
    cout << "Enter element to find in array: ";
    cin >> ele;
    int res = linear_search(arr, n, ele);
    if(res != -1)
        cout << "Element " << ele << " found at index: " << res << endl;
    else
        cout << "Element " << ele << " not found in array. " << endl;
    return 0;
}
```

Output

```
./array_linear_search 10
1 2 3 4 5 6 7 8 9 10
Array elements are: 1 2 3 4 5 6 7 8 9 10
Enter element to find in array: 5
Element 5 found at index: 4
```

2. Binary Search

Binary search works only with sorted arrays. In binary search, the middle element of the given array is found first. Then, it is compared with the target element. If the mid element is equal to the target element, then the index of the mid element is returned. If the target element is greater than the mid element, then reduce the given array to all the elements that come after the mid element, and repeat the process. If the target element is less than the mid element, then reduce the given array to all the elements that come before the mid element, and repeat the process.

It is better than linear search in terms of time complexity, which is $O(\log(n))$. Because the binary search works only the half of the array, and then makes half of half and so on. Therefore, not all the elements of the given array are traversed.

Example

```
#include <iostream>
#include <stdio.h>
#include <stdlib.h>
using namespace std;
```

```
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}
```

```
void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

```
int binary_search(int arr[], int size, int find)
{
    int low = 0, high = size - 1;
    while (low <= high)
    {
```

```

        int mid = low + (high - low) / 2;
        if (arr[mid] == find)
            return mid;
        else if (arr[mid] > find)
            high = mid - 1;
        else
            low = mid + 1;
    }
    return -1;
}

```

```

int main(int argc, char** argv)
{
    if(argc < 2)
    {
        cout << "Invalid Input.\n";
        return 0;
    }
    int n = atoi(argv[1]);
    int arr[n];
    for(int i=0;i<n;i++)
    {
        cin >> arr[i];
    }
    cout << "Array elements are: ";
    for(int i=0;i<n;i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    quickSort(arr, 0, n-1);
    cout << "Array elements after sorting are: ";
    for(int i=0;i<n;i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    int ele;
    cout << "Enter element to find in array: ";
    cin >> ele;
    int res = binary_search(arr, n, ele);
    if(res != -1)
        cout << "Element " << ele << " found at index: " << res << endl;
    else
        cout << "Element " << ele << " not found in array. " << endl;
    return 0;
}

```

Output

```
./array_binary_search 5
```

```
1 5 2 4 0
```

```
Array elements are: 1 5 2 4 0
```

```
Array elements after sorting are: 0 1 2 4 5
```

```
Enter element to find in array: 5
```

```
Element 5 found at index: 4
```

Array Operations Time Complexities

Operation	Best Case	Average Case	Worst Case
Traversal	$O(n)$	$O(n)$	$O(n)$
Access by Index	$O(1)$	$O(1)$	$O(1)$
Search (Unsorted)	$O(1)$	$O(n)$	$O(n)$
Search (Sorted)	$O(1)$	$O(\log n)$	$O(\log n)$
Insertion at End	$O(1)$	$O(1)$	$O(n)$
Insertion at Index	$O(1)$	$O(n)$	$O(n)$
Deletion at Index	$O(1)$	$O(n)$	$O(n)$
Sorting	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Update at Index	$O(1)$	$O(1)$	$O(1)$

Applications of Array Data Structure

The following are some of the applications of arrays.

- **Efficient Storage:** Arrays store multiple elements of the same type in contiguous memory, making access via index lightning-fast ($O(1)$ time for retrieval).
- **Index-Based Mapping:** Fast alternative to hash maps when working with small integer ranges (for example, frequency counters or bucket sort).
- **Accessing Data:** Arrays keep elements in a specific order. Therefore, it allows constant-time $O(1)$ for accessing any element.
- **Searching:** If elements of an array are sorted, we can do searching for an item in $O(\log n)$ time. We can also find `ceiling()`, `floor()`, `kth smallest`, `kth largest`, etc., efficiently.
- **Matrices:** Implementation of matrices can be achieved with the help of two-dimensional arrays.
- **Implementation of the Other Data Structures:** One can easily implement [stacks](#) and [queues](#) with the help of an array. Even a [deque](#) can also be implemented with the help of arrays.
- **Usage in [Dynamic programming](#):** The intermediate result in the dynamic programming algorithms is often stored in an array. These intermediate results are quite helpful in solving larger problems.
- **Data Buffers:** Arrays can also serve as data buffers, temporarily storing incoming data such as file streams, network packets, and database results before processing.
- **Multidimensional Modeling:** 2D arrays are great for matrix operations, graphs (adjacency matrix), and games (like Sudoku or chessboards).

Advantages of Array Data Structure

- **Quick and Efficient Access:** Arrays allow efficient and quick access to any element in the collection with constant access time. This is because the data is kept in memory locations that are contiguous.
- **Memory Efficiency:** Since elements of an array are kept in contiguous memory, fragmentation of memory is reduced.
- **Versatile Nature:** Arrays can be used to store integers, strings, floating-point numbers, characters, and even pointers and objects.
- **Hardware Compatibility:** The array data structure is generally compatible with most hardware architectures. Thus, making it a versatile tool in the world of computer programming.

Disadvantages of Array Data Structure

- **Fixed Size:** Generally, an array's size is fixed. Therefore, expanding an array will require more memory. Also, it will be more time-consuming because of the elements of the array being copied. Thus, array expansion is memory-intensive and time-consuming.

- **Memory Allocation Issues:** Allocation of large arrays can cause exhaustion of memory. This may lead to crashes, especially on those systems where resources are limited.
- **Challenges in Insertion and Deletion:** Since elements are stored in an array immediately next to each other, and removing or adding elements requires shifting the subsequent elements, making these insertion or deletion operations inefficient.
- **Limited Data Type Support:** Arrays only support elements of the same type, limiting their usage with complex data types.
- **Flexibility Issue:** Since arrays are fixed size and support elements of the type, therefore, arrays are less adaptable than structures like trees or [linked lists](#).

Conclusion

Even after many disadvantages (mentioned above), arrays remain indispensable in programming because they are efficient enough in handling collections of the same [data type](#). Understanding arrays is very important for making scalable software systems and efficient algorithms.

2. Multidimensional Array in Data Structure

Multidimensional arrays are basically arrays of arrays. It allows us to store data in a tabular or grid-like structure, making them useful for scenarios like matrices, game boards, or any data that requires multiple dimensions. It is useful when we want to store data in tabular form (rows and columns). A multidimensional array can be 2-dimensional, 3-dimensional and up to n-dimensional.

2D Array

In a 2D array, each row is a one-dimensional array. The syntax of declaring a two-dimensional array is very similar to that of a one-dimensional array, given as follows.

Syntax (Declaration, Initialization and Assigning)

1. `data_type[][] array_name = new data_type[r][c];` // here r is the number of rows, and c is the number of columns
2. // Assigning Value
3. `array_name[r][c] = val;`

For r = 3, c = 3, the following 2-dimensional array is generated.

	Column 0	Column 1	Column 2
Row 0	<code>arr[0][0]</code>	<code>arr[0][1]</code>	<code>arr[0][2]</code>
Row 1	<code>arr[1][0]</code>	<code>arr[1][1]</code>	<code>arr[1][2]</code>
Row 2	<code>arr[2][0]</code>	<code>arr[2][1]</code>	<code>arr[2][2]</code>

Since arrays follow zero-indexing for storing elements, the row ranges from 0 to r - 1, and the column ranges from 0 to c - 1. Let's see the implementation of two-dimensional arrays in various languages.

1. What is a 2D Matrix?

A 2D matrix is a two-dimensional array - a grid of elements organized in rows and columns.

Matrix[m][n] where:

- m = number of rows
- n = number of columns

Memory Layout:

- Stored in row-major order (row by row in contiguous memory)
- `matrix[i][j]` = element at row i, column j

Basic Indexing

```
j=0 j=1 j=2 j=3
i=0  1  2  3  4
i=1  5  6  7  8
i=2  9 10 11 12
```

- Access: matrix[i][j]
- Row i: All elements with same first index
- Column j: All elements with same second index

3. Traversal Patterns - Theory

A. Row-Major Traversal (Default)

- Visit all elements row by row, left to right
- Order: Process all columns in row 0, then row 1, then row 2...
- Use case: Most common, cache-friendly (matches memory layout)
- Complexity: $O(m \times n)$

B. Column-Major Traversal

- Visit all elements column by column, top to bottom
- Order: Process all rows in column 0, then column 1...
- Use case: Transpose operations, column-based operations
- Complexity: $O(m \times n)$
- Note: Less cache-friendly than row-major

C. Diagonal Traversal

- Primary diagonal: $i = j$ (top-left to bottom-right)
- Secondary diagonal: $i + j = n-1$ (top-right to bottom-left)
- Use case: Matrix properties, game boards (chess, tic-tac-toe)
- Complexity: $O(\min(m, n))$

D. Spiral Traversal

- Visit elements in spiral pattern: Right → Down → Left → Up
- Shrink boundaries after each direction
- Use case: Puzzles, image processing, special algorithms
- Complexity: $O(m \times n)$

E. Zigzag/Wave Traversal

- Alternate direction per row: Row 0 (L → R), Row 1 (R → L), Row 2 (L → R)...
- Use case: Snake games, serpentine scanning
- Complexity: $O(m \times n)$

4. Time & Space Complexity

Traversal Type	Time Complexity	Space Complexity
Row-wise	$O(m \times n)$	$O(1)$
Column-wise	$O(m \times n)$	$O(1)$
Diagonal	$O(\min(m,n))$	$O(1)$
Spiral	$O(m \times n)$	$O(1)$
Zigzag	$O(m \times n)$	$O(1)$

Space = $O(1)$ assuming in-place traversal (no extra arrays)

5. Boundary Conditions

Always check:

- $0 \leq i < \text{rows}$
- $0 \leq j < \text{columns}$
- Handle edge cases: empty matrix, single row/column

6. Common Applications

1. Image Processing

- Pixel manipulation (row-major)
- Filters (spiral, neighbor access)

2. Game Development

- Board games (diagonal checks)
- Pathfinding (neighbor traversal)

3. Matrix Operations

- Addition, multiplication (row/column-wise)
- Transpose (column-major)

4. Data Analysis

- Tables, spreadsheets (row/column access)
- Heat maps (various patterns)

7. Key Concepts

- Cache Locality: Row-major is faster (matches memory layout)
- Nested Loops: Outer loop = rows, Inner loop = columns (typically)
- Direction Control: Use variables to control traversal direction
- Boundary Management: Critical for spiral/complex patterns

8. Interview Perspective

Common Questions:

- Print matrix in spiral order
- Rotate matrix 90°
- Search in sorted 2D matrix
- Find saddle point
- Zigzag traversal

Key Skills:

- Index manipulation
- Boundary handling
- Loop control

Example

```
#include <iostream>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
using namespace std;
```

```
int main(int argc, char** argv)
```

```
{
```

```
    if(argc < 3)
```

```
    {
```

```
        cout << "Invalid Input.\n";
```

```
        return 0;
```

```
    }
```

```
    int n = atoi(argv[1]);
```

```
    int m = atoi(argv[2]);
```

```
    int arr[n][m];
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        for(int j=0;j<m;j++)
```

```
        {
```

```
            cin >> arr[i][j];
```

```
        }
```

```
    }
```

```
    cout << "Enter elements of matrix are: \n";
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        for(int j=0;j<m;j++)
```

```
        {
```

```
            cout << arr[i][j] << " ";
```

```
        }
```

```
        cout << endl;
```

```
    }
```

```

    cout << endl;
    return 0;
}

```

Output

```

./2d_array 3 3
1 2 3 4 5 6 6 7 8 9
Enter elements of matrix are:
1 2 3
4 5 6
6 7 8

```

1. Print matrix in spiral order

What is Spiral Order?

Spiral order means printing the elements of a matrix by moving:

Left to Right (top row)

Top to Bottom (right column)

Right to Left (bottom row)

Bottom to Top (left column)

...and then repeating this process for the inner matrix.

Example Matrix

```

1  2  3
4  5  6
7  8  9

```

Spiral Order Output

```

1 2 3 6 9 8 7 4 5

```

How to Print Matrix in Spiral Order (C++)

Idea / Logic

We maintain **four boundaries**:

- top → first row
- bottom → last row
- left → first column
- right → last column

We keep shrinking these boundaries after printing each side.

Example

```

#include <iostream>
#include <stdio.h>
#include <stdlib.h>

```

```

using namespace std;

```

```

int main(int argc, char** argv)
{
    if(argc < 3)
    {
        cout << "Invalid Input.\n";
        return 0;
    }
    int n = atoi(argv[1]);
    int m = atoi(argv[2]);
    int arr[n][m];
    for(int i=0;i<n;i++)
    {

```

```

        for(int j=0;j<m;j++)
        {
            cin >> arr[i][j];
        }
    }
    cout << "Elements of matrix are: \n";
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<m;j++)
        {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }
    cout << endl;
    int t = 0, b = n-1;
    int l = 0, r = m-1;
    cout << "Elements after spiral order are: ";
    while(t <= b && l <= r)
    {
        // Print top row
        for(int i=l;i<=r;i++)
        {
            cout << arr[t][i] << " ";
        }
        t++;
        // Print right column
        for(int i=t;i<=b;i++)
        {
            cout << arr[i][r] << " ";
        }
        r--;
        // Print bottom row
        if(t <= b)
        {
            for(int i=r;i>=l;i--)
            {
                cout << arr[b][i] << " ";
            }
            b--;
        }
        // Print left column
        if(l <= r)
        {
            for(int i=b;i>=t;i--)
            {
                cout << arr[i][l] << " ";
            }
            l++;
        }
    }
    cout << endl;
    return 0;
}

```

Output

```
./2d_array_spiral_order 3 4
1 3 5 7 9 11 13 15 17 19 21 23
Elements of matrix are:
1 3 5 7
9 11 13 15
17 19 21 23
```

Elements after spiral order are: 1 3 5 7 15 23 21 19 17 9 11 13

2. Rotate matrix 90°

Two steps:

1. **Transpose the matrix**
2. **Reverse each row**

Example [Clockwise Rotation 90° with n*n (Square Matrix)]

```
#include <iostream>
#include <stdio.h>
#include <stdlib.h>
using namespace std;

int main(int argc, char** argv)
{
    if(argc < 2)
    {
        cout << "Invalid Input.\n";
        return 0;
    }
    int n = atoi(argv[1]);
    int arr[n][n];
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<n;j++)
        {
            cin >> arr[i][j];
        }
    }
    cout << " Square Matrix before 90° clockwise rotation: \n";
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<n;j++)
        {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }
    cout << endl;
    for(int i=0;i<n;i++)
    {
        for(int j=i+1;j<n;j++)
        {
            swap(arr[i][j], arr[j][i]);
        }
    }
    for(int i=0;i<n;i++)
    {
```

```

    int left = 0, right = n-1;
    while(left < right)
    {
        swap(arr[i][left], arr[i][right]);
        left++;
        right--;
    }
}
cout << " Square Matrix after 90° clockwise rotation: \n";
for(int i = 0; i < n; i++)
{
    for(int j = 0; j < n; j++)
        cout << arr[i][j] << " ";
    cout << endl;
}
return 0;
}

```

Output

./2d_array_clock90d 3 3

1 2 3 4 5 6 7 8 9

Square Matrix before 90° clockwise rotation:

1 2 3

4 5 6

7 8 9

Square Matrix after 90° clockwise rotation:

7 4 1

8 5 2

9 6 3

Example [Anti-Clockwise Rotation 90° with n*n (Square Matrix)]

Two Steps:

1. **Transpose**
2. **Reverse each column**

```
#include <iostream>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
using namespace std;
```

```
int main(int argc, char** argv)
```

```

{
    if(argc < 2)
    {
        cout << "Invalid Input.\n";
        return 0;
    }
    int n = atoi(argv[1]);
    int arr[n][n];
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<n;j++)
        {
            cin >> arr[i][j];
        }
    }
}

```

```

}
cout << "Square Matrix before 90° anti-clockwise rotation: \n";
for(int i=0;i<n;i++)
{
    for(int j=0;j<n;j++)
    {
        cout << arr[i][j] << " ";
    }
    cout << endl;
}
cout << endl;
for(int i=0;i<n;i++)
{
    for(int j=i+1;j<n;j++)
    {
        swap(arr[i][j], arr[j][i]);
    }
}
for(int i=0;i<n;i++)
{
    int top = 0, bottom = n-1;
    while(top < bottom)
    {
        swap(arr[top][i], arr[bottom][i]);
        top++;
        bottom--;
    }
}
cout << "Square Matrix after 90° anti-clockwise rotation: \n";
for(int i = 0; i < n; i++)
{
    for(int j = 0; j < n; j++)
        cout << arr[i][j] << " ";
    cout << endl;
}
return 0;
}

```

Output

./2d_array_anticlock90d 3 3

1 2 3 4 5 6 7 8 9

Square Matrix before 90° anti-clockwise rotation:

1 2 3

4 5 6

7 8 9

Square Matrix after 90° anti-clockwise rotation:

3 6 9

2 5 8

1 4 7

Example [Clockwise Rotation 90° with n*m (Rectangular Matrix)]

- For a **rectangular matrix** ($n \times m$), rotation **cannot be done in-place**.

Two steps:

1. **Transpose the matrix**

2. Reverse each row

```
#include <iostream>
#include <stdio.h>
#include <stdlib.h>
using namespace std;

int main(int argc, char** argv)
{
    if(argc < 3)
    {
        cout << "Invalid Input.\n";
        return 0;
    }
    int n = atoi(argv[1]);
    int m = atoi(argv[2]);
    int arr[n][m];
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<m;j++)
        {
            cin >> arr[i][j];
        }
    }
    cout << "Rectangular Matrix before 90* clockwise rotation: \n";
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<m;j++)
        {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }
    cout << endl;
    int rot[m][n];
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<m;j++)
        {
            rot[j][n-1-i] = arr[i][j];
        }
    }
    cout << "Rectangular Matrix after 90* clockwise rotation: \n";
    for(int i = 0; i < m; i++)
    {
        for(int j = 0; j < n; j++)
            cout << rot[i][j] << " ";
        cout << endl;
    }
    return 0;
}
```

Output

./2d_array_clock90d_rect 2 3

1 2 3 4 5 6

Rectangular Matrix before 90* clockwise rotation:

1 2 3
4 5 6

Rectangular Matrix after 90° clockwise rotation:

4 1
5 2
6 3

Example [Anti-Clockwise Rotation 90° with n*m (Rectangular Matrix)]

Two Steps:

1. **Transpose**
2. **Reverse each column**

```
#include <iostream>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
using namespace std;
```

```
int main(int argc, char** argv)
```

```
{
```

```
    if(argc < 3)
```

```
    {
```

```
        cout << "Invalid Input.\n";
```

```
        return 0;
```

```
    }
```

```
    int n = atoi(argv[1]);
```

```
    int m = atoi(argv[2]);
```

```
    int arr[n][m];
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        for(int j=0;j<m;j++)
```

```
        {
```

```
            cin >> arr[i][j];
```

```
        }
```

```
    }
```

```
    cout << "Rectangular Matrix before 90° anti-clockwise rotation: \n";
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        for(int j=0;j<m;j++)
```

```
        {
```

```
            cout << arr[i][j] << " ";
```

```
        }
```

```
        cout << endl;
```

```
    }
```

```
    cout << endl;
```

```
    int rot[m][n];
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        for(int j=0;j<m;j++)
```

```
        {
```

```
            rot[m-1-j][i] = arr[i][j];
```

```
        }
```

```
    }
```

```
    cout << "Rectangular Matrix after 90° anti-clockwise rotation: \n";
```

```
    for(int i = 0; i < m; i++)
```

```

{
    for(int j = 0; j < n; j++)
        cout << rot[i][j] << " ";
    cout << endl;
}
return 0;
}

```

Output

./2d_array_anticlock90d_rect 2 3

1 2 3 4 5 6

Rectangular Matrix before 90* anti-clockwise rotation:

1 2 3

4 5 6

Rectangular Matrix after 90* anti-clockwise rotation:

3 6

2 5

1 4

3. Search in sorted 2D matrix

A **sorted 2D matrix** is a matrix where the elements follow **specific sorting rules**.

Type 1: Row-wise Sorted Matrix

Definition

Each **row is sorted in ascending order**, but rows are independent.

Example

1 3 5

2 4 6

7 8 9

Each row is sorted

Columns may NOT be sorted

Type 2: Row-wise & Column-wise Sorted Matrix (Most Important)

Definition

- Each **row is sorted left → right**
- Each **column is sorted top → bottom**

Example

1 4 7

2 5 8

3 6 9

Rows sorted

Columns sorted

This is the **classic “sorted 2D matrix”** used in searching problems.

Because **searching becomes faster**:

Matrix Type	Best Search Method	Time
Row-wise only	Linear / Binary per row	$O(n \log m)$
Row + column sorted	Staircase search	$O(n + m)$
Fully sorted	Binary search	$O(\log(n \times m))$

Example (Linear Search Approach)

```
#include <iostream>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
using namespace std;
```

```
int main(int argc, char** argv)
```

```

{
    if(argc < 3)
    {
        cout << "Invalid Input.\n";
        return 0;
    }
    int n = atoi(argv[1]);
    int m = atoi(argv[2]);
    int arr[n][m];
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<m;j++)
        {
            cin >> arr[i][j];
        }
    }
    cout << "2D array elements are: \n";
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<m;j++)
        {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }
    cout << endl;
    int key;
    cout << "Enter element to find 2D array: ";
    cin >> key;
    bool found = false;
    int r = -1, c = -1;
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<m;j++)
        {
            if(arr[i][j] == key)
            {
                found = true;
                r = i+1;
                c = j+1;
                break;
            }
        }
    }
    if(found)
        cout << "Element " << key << " found at row: " << r << " & column: " << c <<
endl;
    else
        cout << "Element " << key << " not found.\n";
    return 0;
}

```

Output

```
./search_2d_sorted_array 2 3
```

```
1 2 3 4 5 6
```

2D array elements are:

1 2 3

4 5 6

Enter element to find 2D array: 6

Element 6 found at row: 2 & column: 3

Example (Binary Search Approach)

```
#include <iostream>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
using namespace std;
```

```
int main(int argc, char** argv)
```

```
{
```

```
    if(argc < 3)
```

```
    {
```

```
        cout << "Invalid Input.\n";
```

```
        return 0;
```

```
    }
```

```
    int n = atoi(argv[1]);
```

```
    int m = atoi(argv[2]);
```

```
    int arr[n][m];
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        for(int j=0;j<m;j++)
```

```
        {
```

```
            cin >> arr[i][j];
```

```
        }
```

```
    }
```

```
    cout << "2D array elements are: \n";
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        for(int j=0;j<m;j++)
```

```
        {
```

```
            cout << arr[i][j] << " ";
```

```
        }
```

```
        cout << endl;
```

```
    }
```

```
    cout << endl;
```

```
    int key;
```

```
    cout << "Enter element to find 2D array: ";
```

```
    cin >> key;
```

```
    bool found = false;
```

```
    int r = -1, c = -1;
```

```
    int i = 0, j = m-1;
```

```
    while(i < n && j >= 0)
```

```
    {
```

```
        if(arr[i][j] == key)
```

```
        {
```

```
            found = true;
```

```
            r = i+1;
```

```
            c = j+1;
```

```
            break;
```

```
        }
```

```
        else if(arr[i][j] > key)
```

```

        j--;
    else
        i++;
    }
    if(found)
        cout << "Element " << key << " found at row: " << r << " & column: " << c <<
endl;
    else
        cout << "Element " << key << " not found.\n";
    return 0;
}

```

Output

```

./search_2d_sorted_array_binaryApproach 2 3
1 2 3 4 5 6
2D array elements are:
1 2 3
4 5 6
Enter element to find 2D array: 5
Element 5 found at row: 2 & column: 2

```

4. Find saddle point

Definition

A **saddle point** in a matrix is an element that is:

1. **The smallest element in its row, and**
2. **The largest element in its column**

Think of it like a "valley in the row but a peak in the column" — hence the name **saddle point**.

Matrix:

```

1  2  3
4  5  6
7  8  9

```

Check each element:

- Row 1: 1, 2, 3 → min = 1
 - Column of 1: 1, 4, 7 → max = 7 not equal
- Row 2: min = 4 → column max = 7
- Row 3: min = 7 → column max = 7 **saddle point found!**

Saddle point = 7

Important Notes

- There can be **zero, one, or multiple saddle points**.
- Only **row min & column max** counts.

C++ Approach

Steps:

1. For each row, find the **minimum element**.
2. Check if it is the **maximum in its column**.
3. If yes → **saddle point found**.

Example

```

#include <iostream>
#include <stdio.h>
#include <stdlib.h>
using namespace std;

```

```

int main(int argc, char** argv)
{

```

```

if(argc < 3)
{
    cout << "Invalid Input.\n";
    return 0;
}
int n = atoi(argv[1]);
int m = atoi(argv[2]);
int arr[n][m];
for(int i=0;i<n;i++)
{
    for(int j=0;j<m;j++)
    {
        cin >> arr[i][j];
    }
}
cout << "Elements of the matrix are: \n";
for(int i=0;i<n;i++)
{
    for(int j=0;j<m;j++)
    {
        cout << arr[i][j] << " ";
    }
    cout << endl;
}
cout << endl;
bool found = false;
for(int i=0;i<n;i++)
{
    int rm = arr[i][0];
    int cidx = 0;
    for(int j=1;j<m;j++)
    {
        if(arr[i][j] < rm)
        {
            rm = arr[i][j];
            cidx = j;
        }
    }
    bool saddle = true;
    for(int k=0;k<n;k++)
    {
        if(arr[k][cidx] > rm)
        {
            saddle = false;
            break;
        }
    }
    if(saddle)
    {
        cout << "Saddle point: " << rm << endl;
        found = true;
    }
}
if(!found)

```

```

        cout << "No saddle point found\n";
    return 0;
}

```

Output

```

./2d_matrix_saddle_point 2 3
1 2 3 4 5 6
Elements of the matrix are:
1 2 3
4 5 6
Saddle point: 4

```

5. Zigzag traversal

What is Zigzag Traversal in a Matrix?

Zigzag traversal means printing matrix elements **row by row**, but:

- **Even rows** → **left to right**
- **Odd rows** → **right to left**

So the direction **keeps changing (zig-zag)** as you move down the rows.

Matrix:

```

1  2  3
4  5  6
7  8  9

```

Zigzag Output:

```

1 2 3 6 5 4 7 8 9

```

Explanation:

- Row 0 → left → right → 1 2 3
- Row 1 → right → left → 6 5 4
- Row 2 → left → right → 7 8 9

Example

```

#include <iostream>

```

```

#include <stdio.h>

```

```

#include <stdlib.h>

```

```

using namespace std;

```

```

int main(int argc, char** argv)

```

```

{
    if(argc < 3)
    {
        cout << "Invalid Input.\n";
        return 0;
    }
    int n = atoi(argv[1]);
    int m = atoi(argv[2]);
    int arr[n][m];
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<m;j++)
        {
            cin >> arr[i][j];
        }
    }
    cout << "Elements of matrix are: \n";
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<m;j++)

```

```

    {
        cout << arr[i][j] << " ";
    }
    cout << endl;
}
cout << endl;
cout << "Elements of matrix after zigzag transform are: ";
for(int i=0;i<n;i++)
{
    if(i%2==0)
    {
        for(int j=0;j<m;j++)
        {
            cout << arr[i][j] << " ";
        }
    }
    else
    {
        for(int j=m-1;j>=0;j--)
        {
            cout << arr[i][j] << " ";
        }
    }
}
cout << endl;
return 0;
}

```

Output

./2d_matrix_zigzag_traversal 3 3

1 2 3 4 5 6 7 8 9

Elements of matrix are:

1 2 3

4 5 6

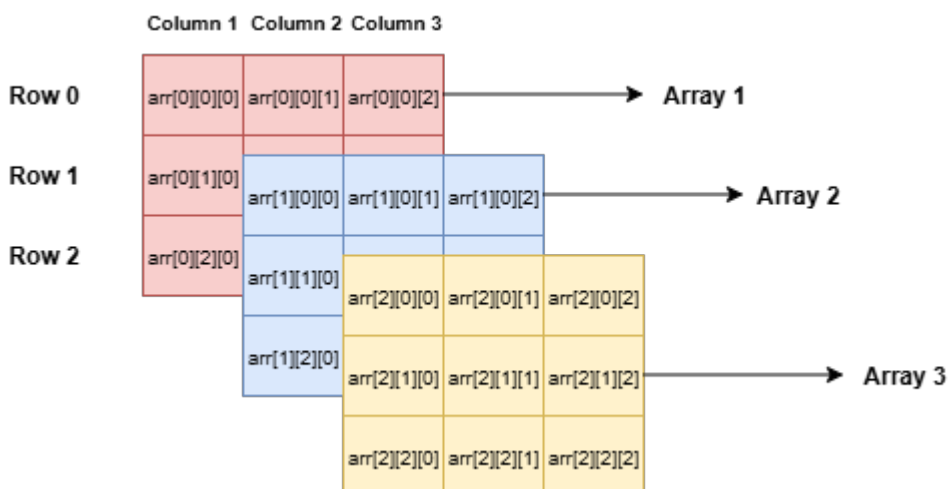
7 8 9

Elements of matrix after zigzag transform are: 1 2 3 6 5 4 7 8 9

3D Array

A 3D array is the extension of a 2D array. In a 3D array, multiple 2D arrays can be put together.

A three-dimensional array having three 2D arrays (or blocks), each 2D array having three rows and three columns, can be represented as:



Example of 3D Array

```
#include <iostream>
#include <stdio.h>
#include <stdlib.h>
using namespace std;

int main(int argc, char** argv)
{
    if(argc < 4)
    {
        cout << "Invalid Input.\n";
        return 0;
    }
    int x = atoi(argv[1]);
    int y = atoi(argv[2]);
    int z = atoi(argv[3]);
    int arr[x][y][z];
    for(int i=0;i<x;i++)
    {
        for(int j=0;j<y;j++)
        {
            for(int k=0;k<z;k++)
            {
                cin >> arr[i][j][k];
            }
        }
    }
    cout << "Elements of 3D Array are: \n";
    for(int i=0;i<x;i++)
    {
        for(int j=0;j<y;j++)
        {
            for(int k=0;k<z;k++)
            {
                cout << arr[i][j][k] << " ";
            }
            cout << endl;
        }
        cout << endl;
    }
    cout << endl;
    return 0;
}
```

Output

```
./3d_array_traversal 3 3 3
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27
Elements of 3D Array are:
1 2 3
4 5 6
7 8 9

10 11 12
13 14 15
16 17 18
```

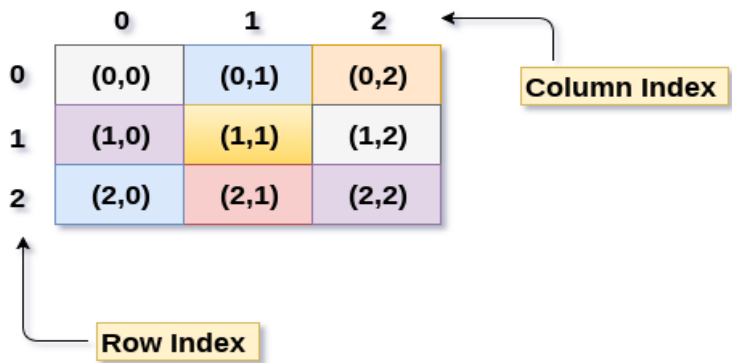
19 20 21
22 23 24
25 26 27

Mapping 2D Array to 1D Array

When it comes to mapping a 2-dimensional array, most of us might wonder why this mapping is required. However, 2D arrays exist from the user's point of view. 2D arrays are created to implement a relational database table-like data structure. In computer memory, the storage technique for a 2D array is similar to that of a one-dimensional array.

The size of a two-dimensional array is equal to the product of the number of rows and the number of columns present in the array. We do need to map a two-dimensional array to a one-dimensional array to store it in memory.

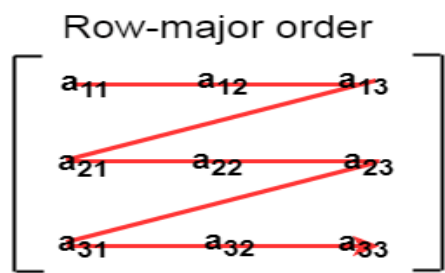
A 3 × 3 two-dimensional array is shown in the following image. However, this array needs to be mapped to a one-dimensional array to be stored in memory.



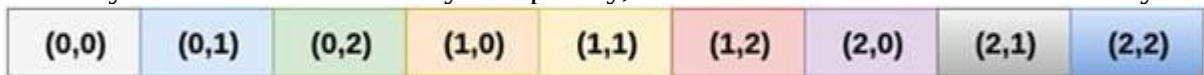
There are two main techniques for storing 2D array elements into memory are:

1. Row Major Ordering

In row-major ordering, all the rows of the 2D array are stored in memory contiguously. Considering the array shown in the above image, its memory allocation according to row-major order is shown as follows:



First, the 1st row of the array is stored in the memory completely, then the 2nd row of the array is stored in the memory completely, and so on till the last row of the array.



Transform 2D Array to 1D Array(Row Major Ordering)

Example

```
#include <iostream>
#include <stdio.h>
#include <stdlib.h>
using namespace std;
```

```
int main(int argc, char** argv)
{
    if(argc < 3)
    {
```

```

        cout << "Invalid Input.\n";
        return 0;
    }
    int n = atoi(argv[1]);
    int m = atoi(argv[2]);
    int arr[n][m];
    int ar[n*m];
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<m;j++)
        {
            cin >> arr[i][j];
        }
    }
    int k = 0;
    cout << "Elemets of 2D Array: \n";
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<m;j++)
        {
            cout << arr[i][j] << " ";
            ar[k] = arr[i][j];
            k++;
        }
        cout << endl;
    }
    cout << "1D Array elements after row major transform: ";
    for(int i=0;i<n*m;i++)
    {
        cout << ar[i] << " ";
    }
    cout << endl;
    return 0;
}

```

Output

```
./2d_to_1d_row_major 2 3
```

```
1 2 3 4 5 6
```

```
Elemets of 2D Array:
```

```
1 2 3
```

```
4 5 6
```

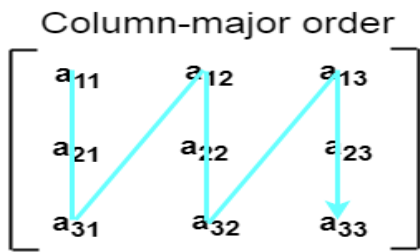
```
1D Array elements after row major transform: 1 2 3 4 5 6
```

2. Column Major ordering

According to the column major ordering, all the columns of the 2D array are stored in memory contiguously. The memory allocation of the array, which is shown in the above image, is given as follows.

(0,0)	(1,0)	(2,0)	(0,1)	(1,1)	(2,1)	(0,2)	(1,2)	(2,2)
-------	-------	-------	-------	-------	-------	-------	-------	-------

First, the 1st column of the array is stored in the memory completely, then the 2nd column of the array is stored in the memory completely, and so on till the last column of the array.



Transform 2D Array to 1D Array(Column Major Ordering)

Example

```
#include <iostream>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
using namespace std;
```

```
int main(int argc, char** argv)
```

```
{
```

```
    if(argc < 3)
```

```
    {
```

```
        cout << "Invalid Input.\n";
```

```
        return 0;
```

```
    }
```

```
    int n = atoi(argv[1]);
```

```
    int m = atoi(argv[2]);
```

```
    int arr[n][m];
```

```
    int ar[n*m];
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        for(int j=0;j<m;j++)
```

```
        {
```

```
            cin >> arr[i][j];
```

```
        }
```

```
    }
```

```
    cout << "Elements of 2D array are: \n";
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        for(int j=0;j<m;j++)
```

```
        {
```

```
            cout << arr[i][j] << " ";
```

```
        }
```

```
        cout << endl;
```

```
    }
```

```
    int k = 0;
```

```
    for(int j=0;j<m;j++)
```

```
    {
```

```
        for(int i=0;i<n;i++)
```

```
        {
```

```
            ar[k] = arr[i][j];
```

```
            k++;
```

```
        }
```

```
    }
```

```
    cout << "Elements of 1D array after transform from 2D to 1D are: ";
```

```
    for(int i=0;i<n*m;i++)
```

```
    {
```

```
        cout << ar[i] << " ";
```

```

}
cout << endl;
return 0;
}

```

Output

./2d_to_1d_column_major 2 3

1 2 3 4 5 6

Elements of 2D array are:

1 2 3

4 5 6

Elements of 1D array after transform from 2D to 1D are: 1 4 2 5 3 6

Note: The way a 2D array is converted into a 1D array, one can do the same for the 3D arrays too. Readers are required to try it out on their own.

Application of Multi-Dimensional Array

- The data can be arranged in a more manageable form (Tabular form) using a multidimensional matrix. For example, the arrangement of a student's data, such as the student's roll number, marks, and image of the student, using a 3D array or matrix.
- In dynamic programming questions, multidimensional arrays are very useful to store the intermediate results of the dynamic programming problems. Apart from these, they are also used in various standard problems like: Adjacency matrix representation in graphs, Matrix Multiplication, and Grid search problems.

3. Sparse Matrix (Data Structures)

In this article, we will discuss the sparse matrix.

Let's first see a brief description of the matrix.

What is a matrix?

A matrix can be defined as a two-dimensional array having 'm' rows and 'n' columns. A matrix with m rows and n columns is called m × n matrix. It is a set of numbers that are arranged in the horizontal or vertical lines of entries.

For example -

$$A = \begin{matrix} & \begin{matrix} \text{Row (m)} \end{matrix} & \\ \begin{matrix} \text{Columns (n)} \end{matrix} & \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} & \end{matrix}$$

What is a sparse matrix?

Sparse matrices are those matrices that have the majority of their elements equal to zero. In other words, the sparse matrix can be defined as the matrix that has a greater number of zero elements than the non-zero elements.

Now, the question arises: we can also use the simple matrix to store the elements, then why is the sparse matrix required?

Why is a sparse matrix required if we can use the simple matrix to store elements?

There are the following benefits of using the sparse matrix -

Storage - We know that a sparse matrix contains lesser non-zero elements than zero, so less memory can be used to store elements. It evaluates only the non-zero elements.

Computing time: In the case of searching in sparse matrix, we need to traverse only the non-zero elements rather than traversing all the sparse matrix elements. It saves computing time by logically designing a data structure traversing non-zero elements.

Representation of sparse matrix

Now, let's see the representation of the sparse matrix. The non-zero elements in the sparse matrix can be stored using triplets that are rows, columns, and values. There are two ways to represent the sparse matrix that are listed as follows -

- Array representation
- Linked list representation

Array representation of the sparse matrix

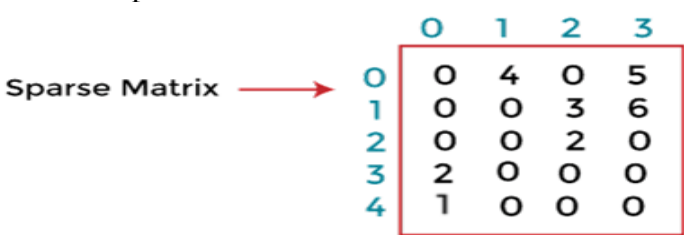
Representing a sparse matrix by a 2D array leads to the wastage of lots of memory. This is because zeroes in the matrix are of no use, so storing zeroes with non-zero elements is wastage of memory. To avoid such wastage, we can store only non-zero elements. If we store only non-zero elements, it reduces the traversal time and the storage space.
In 2D array representation of sparse matrix, there are three fields used that are named as -

ROW	COL	VALUE
-----	-----	-------

- **Row** - It is the index of a row where a non-zero element is located in the matrix.
- **Column** - It is the index of the column where a non-zero element is located in the matrix.
- **Value** - It is the value of the non-zero element that is located at the index (row, column).

Example -

Let's understand the array representation of sparse matrix with the help of the example given below -
Consider the sparse matrix -



In the above figure, we can observe a 5x4 sparse matrix containing 7 non-zero elements and 13 zero elements. The above matrix occupies 5x4 = 20 memory space. Increasing the size of matrix will increase the wastage space.
The tabular representation of the above matrix is given below -

Table Structure

Row	Column	Value
0	1	4
0	3	5
1	2	3
1	3	6
2	2	2
3	0	2
4	0	1
5	4	7

In the above structure, first column represents the rows, the second column represents the columns, and the third column represents the non-zero value. The first row of the table represents the triplets. The first triplet represents that the value 4 is stored at 0th row and 1st column. Similarly, the second triplet represents that the value 5 is stored at the 0th row and 3rd column. In a similar manner, all triplets represent the stored location of the non-zero elements in the matrix.

The size of the table depends upon the total number of non-zero elements in the given sparse matrix. Above table occupies $8 \times 3 = 24$ memory space which is more than the space occupied by the sparse matrix. So, what's the benefit of using the sparse matrix? Consider the case if the matrix is 8×8 and there are only 8 non-zero elements in the matrix, then the space occupied by the sparse matrix would be $8 \times 8 = 64$, whereas the space occupied by the table represented using triplets would be $8 \times 3 = 24$.

Implementation of array representation of the sparse matrix

Now, let's see the implementation of array representation of sparse matrix in C language.

In the program below, we will show the tabular representation of the non-zero elements of the sparse matrix stored in array.

Example

Output

In the output, first row of the table represent the row location of the value, second row represents the column location of the value, and the third represents the value itself. In the below screenshot, the first column with values 0, 2, and 6 represents the value 6 stored at the 0th row and 2nd column.

Linked List representation of the sparse matrix

In a linked list representation, the linked list data structure is used to represent the sparse matrix. The advantage of using a linked list to represent the sparse matrix is that the complexity of inserting or deleting a node in a linked list is lesser than the array.

Unlike the array representation, a node in the linked list representation consists of four fields. The four fields of the linked list are given as follows -

- **Row** - It represents the index of the row where the non-zero element is located.
- **Column** - It represents the index of the column where the non-zero element is located.
- **Value** - It is the value of the non-zero element that is located at the index (row, column).
- **Next node** - It stores the address of the next node.

The node structure of the linked list representation of the sparse matrix is shown in the below image -

Node Structure			
Row	Column	Value	Pointer to Next Node

Example -

Let's understand the linked list representation of sparse matrix with the help of the example given below -

Consider the sparse matrix -

		0	1	2	3
Sparse Matrix →	0	0	0	1	0
	1	3	0	0	0
	2	0	4	5	0
	3	0	6	0	0

In the above figure, we can observe a 4×4 sparse matrix containing 5 non-zero elements and 11 zero elements. Above matrix occupies $4 \times 4 = 16$ memory space. Increasing the size of matrix will increase the wastage space. The linked list representation of the above matrix is given below -



In the above figure, the sparse matrix is represented in the linked list form. In the node, the first field represents the index of the row, the second field represents the index of the column, the third field represents the value, and the fourth field contains the address of the next node.

In the above figure, the first field of the first node of the linked list contains 0, which means 0th row, the second field contains 2, which means 2nd column, and the third field contains 1 that is the non-zero element. So, the first node represents that element 1 is stored at the 0th row-2nd column in the given sparse matrix. In a similar manner, all of the nodes represent the non-zero elements of the sparse matrix.

Implementation of linked list representation of sparse matrix

Now, let's see the implementation of linked list representation of sparse matrix in Java.

Example

Output

Every row in the output represents the node of the linked list. In every row of the below screenshot, the first element represents the row index location of the non-zero element, the second element represents the column index location of the non-zero element, and the third element represents the non-zero element itself.

So, that's all about the article. In this article, we have first discussed the brief description of Matrix and Sparse Matrix. After that, we saw why the sparse matrix is useful, and at last, we have discussed the array and linked list representation of the sparse matrix. Hope, the article will be helpful and informative to you.

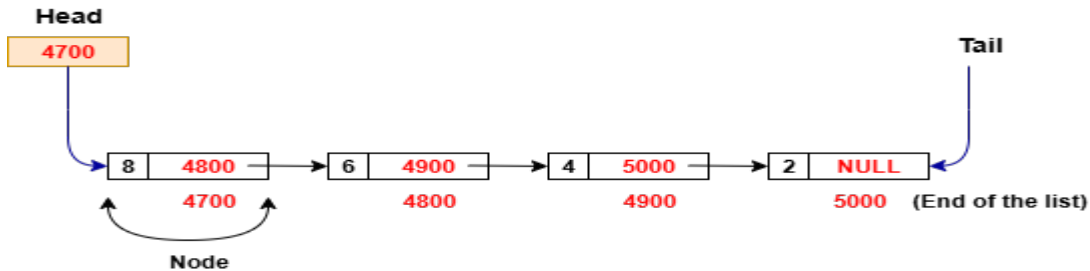
• DS Linked List

1. Linked List Data Structure

A linked list is a linear data structure that includes a series of connected nodes. A linked list can be defined as a set of nodes that are randomly stored in memory. A node in the linked list contains two parts, i.e., the first is the data part and the second is the address part. The last node of the list contains a pointer to null. After array, linked list is the second most used data structure. In a linked list, every link contains a connection to another link.

Representation of a Linked List

A linked list can be represented as a connection of nodes in which each node points to the next node of the list. The representation of the linked list is shown below:



Till now, we have been using an array data structure to organize a group of elements that are to be stored individually in the memory. However, Array has several advantages and disadvantages that must be known to decide the data structure that will be used throughout the program.

Why use a Linked List over an Array?

A linked list is a data structure that overcomes the limitations of arrays. Let's first see some of the limitations of arrays:

- The size of the array must be known in advance before using it in the program.
- Increasing the size of the array is a time-consuming process. It is almost impossible to expand the size of the array at run time.
- All the elements in the array need to be contiguously stored in the memory. Inserting an element in the array needs shifting of all its predecessors.

A linked list is useful because:

- It allocates the memory dynamically. All the nodes of the linked list are non-contiguously stored in the memory and linked together with the help of pointers.
- In a linked list, size is no longer a problem since we do not need to define its size at the time of declaration. The list grows as per the program's demand and is limited to the available memory space.

How to declare a Linked List?

It is simple to declare an array, as it is of a single type, while the declaration of a linked list is a bit more typical than that of an array. A linked list contains two parts, and both are of different types, i.e., one is a simple variable, while the other is a pointer variable. We can declare the linked list by using a user-defined data type **structure**.

The declaration of a linked list is given as follows:

```
struct node
{
    int data;
    struct node *next;
}
```

In the above declaration, we have defined a structure named as **node** that contains two variables, one is **data** that is of integer type, and another one is the **next**, which is a pointer that contains the address of the next node.

Implementation of Linked List

Example

```
#include <iostream>
```

```
#include <stdio.h>
```

```
using namespace std;
```

```
struct Node
```

```
{  
    int data;  
    Node *next;  
    Node(int val)  
    {  
        data = val;  
        next = nullptr;  
    }  
};
```

```
void insertAtHead(Node*& head, int val)
```

```
{  
    Node* newNode = new Node(val);  
    newNode->next = head;  
    head = newNode;  
}
```

```
void insertAtTail(Node*& head, int val)
```

```
{  
    Node* newNode = new Node(val);  
    if(head == nullptr)  
    {  
        head = newNode;  
        return;  
    }  
    Node* temp = head;  
    while(temp->next != nullptr)  
        temp = temp->next;  
    temp->next = newNode;  
}
```

```
void printData(Node* head)
```

```
{  
    while(head != nullptr)  
    {  
        cout << head->data << "->";  
        head = head->next;  
    }  
    cout << "NULL\n";  
}
```

```
int main()
```

```
{  
    Node* head = nullptr;  
    insertAtHead(head, 2);  
    insertAtHead(head, 5);  
    insertAtTail(head, 10);  
}
```

```
insertAtTail(head, 25);  
printData(head);  
return 0;  
}
```

Output

./linked_list

5->2->10->25->NULL

Linked List Operations

The basic operations that are supported by a list are mentioned as follows -

- **Insertion:** This operation is performed to add an element into the list.
- **Deletion:** It is performed to delete an operation from the list.
- **Traversal:** It is performed to traverse the elements of the list.
- **Search:** It is performed to search an element from the list using the given key.

Types of Linked List

A linked list is classified into the following types:

- **Singly-Linked List:** A singly linked list can be defined as the collection of an ordered set of elements. A node in the singly linked list consists of two parts: the data part and the link part. The data part of the node stores actual information that is to be represented by the node, while the link part of the node stores the address of its immediate successor.
- **Doubly Linked List:** A doubly linked list is a complex type of linked list in which a node contains a pointer to the previous as well as the next node in the sequence. Therefore, in a doubly-linked list, a node consists of three parts: node data, pointer to the next node in sequence (next pointer), and pointer to the previous node (previous pointer).
- **Circular Linked List:** In a circular linked list, the last node of the list contains a pointer to the first node of the list. We can have a circular singly linked list as well as a circular doubly linked list.
- **Circular Doubly Linked List:** A circular doubly linked list is a more complex type of data structure in which a node contains pointers to its previous node as well as the next node. A circular doubly linked list does not contain NULL in any of the nodes. The last node of the list contains the address of the first node of the list. The first node of the list also contains the address of the last node in its previous pointer.

Advantages of Linked List

The advantages of using the linked list are given as follows:

- **Dynamic data structure:** The size of the linked list may vary according to the requirements. Linked list does not have a fixed size.
- **Insertion and deletion:** Unlike arrays, insertion and deletion in a linked list are Array elements are stored in consecutive locations, whereas the elements in the linked list are stored at random location. To insert or delete an element in an array, we have to shift the elements to create space. Whereas, in a linked list, instead of shifting, we just have to update the address of the pointer of the node.
- **Memory efficient:** The size of a linked list can grow or shrink according to the requirements, so the memory consumption in a linked list is efficient.
- **Implementation:** We can implement both stacks and queues using a linked list.

Disadvantages of Linked List

The limitations of using the linked list are given as follows:

- **Memory usage:** In a linked list, a node occupies more memory than an Each node of the linked list occupies two types of variables, i.e., one is a simple variable, and the other is a pointer variable.

- **Traversal:** Traversal is not easy in a linked list. If we have to access an element in the linked list, we cannot access it randomly, while in the case of array, we can randomly access it by index. For example, if we want to access the 3rd node, then we need to traverse all the nodes before it. So, the time required to access a particular node is large.
- **Reverse traversing:** Backtracking or reverse traversing is difficult in a linked list. In a doubly-linked list, it is easier but requires more memory to store the back pointer.

Applications of Linked List

The applications of the linked list are given as follows:

- With the help of a linked list, the polynomials can be represented, as we can perform the operations on the polynomial.
- A linked list can be used to represent the sparse matrix.
- The various operations like student's details, employee's details, or product details can be implemented using the linked list as the linked list uses the structure data type that can hold different data types.
- Using a linked list, we can implement a stack, a queue, a tree, and other various data structures.
- The graph is a collection of edges and vertices, and the graph can be represented as an adjacency matrix and an adjacency list. If we want to represent the graph as an adjacency matrix, then it can be implemented as an array. If we want to represent the graph as an adjacency list, then it can be implemented as a linked list.
- A linked list can be used to implement dynamic memory allocation. The dynamic memory allocation is the memory allocation done at run-time.

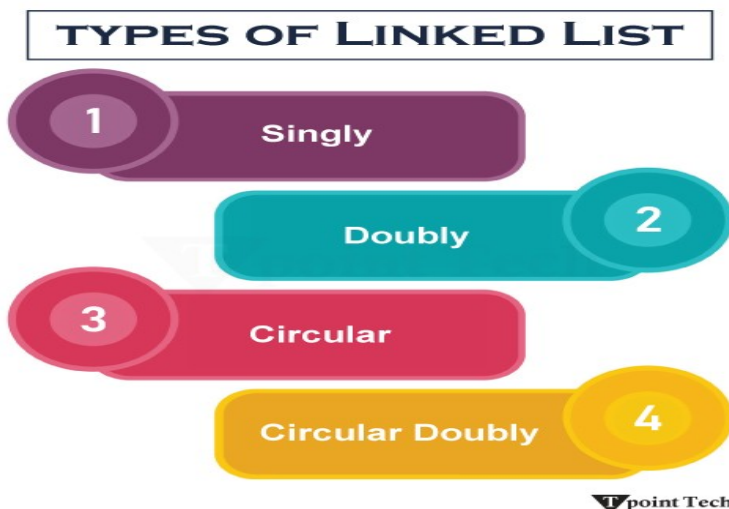
Conclusion

A linked list is a non-contiguous, flexible data structure good for data that is dynamic with frequent deletions and insertions, but is slow due to higher memory overhead and slower random access as compared to arrays. The nodes of a linked list, containing a pointer to the next node and data, are kept in non-sequential memory locations, enabling allocation of the dynamic memory.

2. Types of Linked List

There are four types of Linked List in Data Structures which are discussed below:

- [Singly Linked list](#)
- [Doubly Linked list](#)
- [Circular Linked list](#)
- [Doubly Circular Linked list](#)

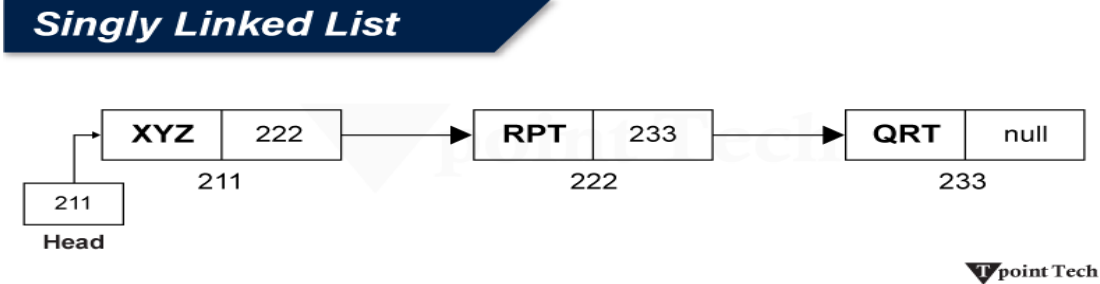


1. Singly Linked List

Singly linked lists contain nodes that have a data part as well as a next address part, which points to the next node in the sequence of nodes. The operations we can perform on singly linked lists are insertion, deletion, and traversal. The singly linked list is a type of data structure that has two parts: the address part, which contains the address of the node that comes after it, and the data part. Another name for a node's address portion is a pointer.

Implementation of Singly Linked List

Suppose, there are three nodes with the following addresses: XPZ, RPT, and QRT. The image below illustrates how three nodes are represented as a linked list:



Three distinct nodes with the addresses 211, 222, and 233, respectively, are visible in the above figure. The first node has the node's address, which is 211; the second node has the last node's address, which is 233; and the third node has the null value in its address section since it does not point to any nodes. A head pointer is the one that contains the address of the first node.

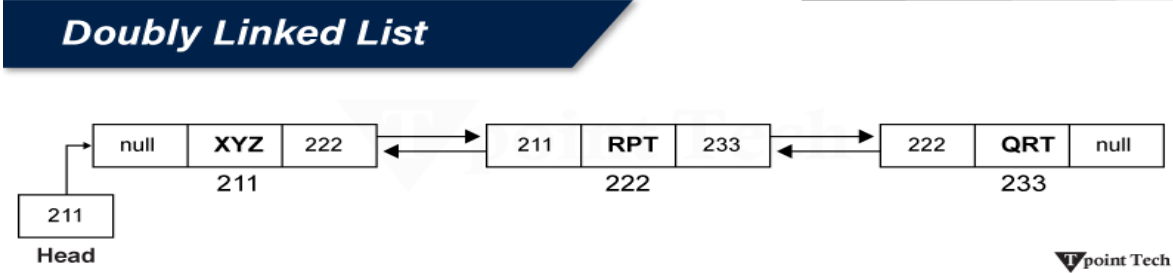
1. struct node
2. {
3. int data;
4. struct node *next;
5. }

In the above representation, we have defined a user-defined structure named node containing two members; the first one is an integer type (data), and the other one is a pointer (next) of the node type.

2. Doubly Linked List

The doubly linked list contains two pointers. It can be described as a linear data structure consisting of three components: the data component and the remaining two address components. To put it another way, a doubly linked list contains three parts in a single node: a pointer to the node before it, a pointer to the node after it, and a data part.

Here, we have three nodes; their respective addresses are XPZ, RPT, and QRT. The following illustrates how these nodes are represented in a doubly linked list:



As we can observe in the above figure, the node in a doubly-linked list has two address parts; one part stores the **address of the next** while the other part of the node stores the **previous node's address**. The initial node in the doubly linked list has the **NULL** value in the address part, which provides the address of the previous node.

Representation of the node in a Doubly Linked List

1. struct node
2. {

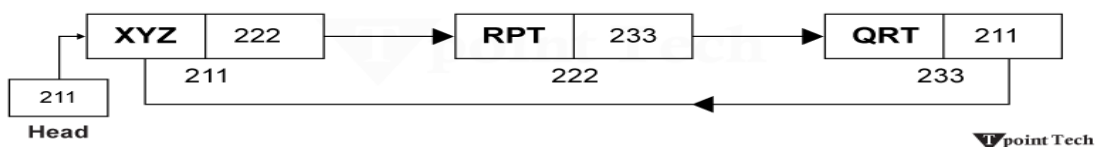
3. int data;
4. struct node *next;
5. struct node *prev;
6. }

A user-defined structure called a node with three members has been defined in the above representation. One member is integer-type data, and the other two are the node-type's pointers, next and prev. The address of the next node is stored in the next pointer variable, while the address of the previous node is stored in the prev pointer. The next and prev pointers have the type struct node * because they both store the address of a node of the struct node type.

3. Circular Linked List

One variant of a singly linked list is a circular linked list. The final node in a singly linked list does not point to any other nodes; therefore, its link part contains a null value. It is the only difference between a singly linked list and a circular linked list. In contrast, a circular linked list is one in which the last node is connected to the first node; therefore, the address of the first node is stored in the link part of the last node.

Circular Linked List



The representation of the node in a circular linked list is similar to that of a singly linked list:

1. struct node
2. {
3. int data;
4. struct node *next;
5. }

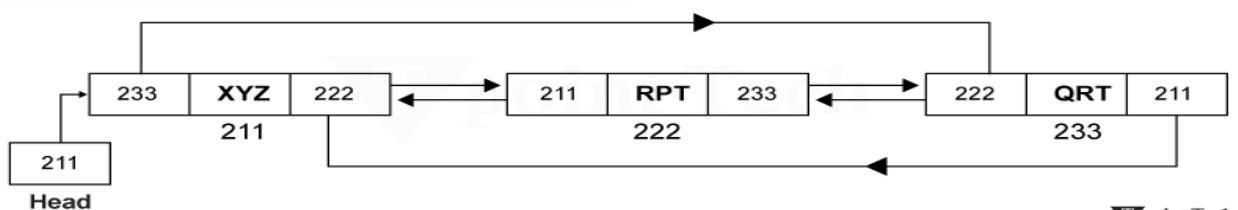
A circular linked list is a sequence of elements in which each node has a link to the next node, and the last node has a link to the first node.

4. Circular Doubly Linked List

The characteristics of both the circular linked list and the doubly linked list are present in the circular doubly linked list. The doubly circular linked list, where the last node's next pointer is connected to the first node and the first node's prev pointer is connected to the last node, forms a circle.

Additionally, each node contains the address of the node before it, making it a doubly linked list. The primary distinction between a doubly circular linked list and a doubly linked list is that the former does not have the null value in the first node's prev field or the last node's next field.

Circular Doubly Linked List



The doubly circular linked list is represented similarly to the doubly linked list because it consists of three parts: two address parts and one data part.

The structure of a node in a circular doubly linked list is:

1. struct node

```

2. {
3.   int data;
4.   struct node *next; // Points to the next node
5.   struct node *prev; // Points to the previous node
6. };

```

Conclusion

A basic linear data structure that provides flexibility in managing sets of data elements is a linked list. The node structure and connection patterns of the four primary types under discussion distinguish them from one another.

1. Singly Linked List in Data Structure

Linked lists may seem a bit complex at first, but they're quite handy. Essentially, you have a bunch of individual nodes, like separate building blocks. Each node stores a piece of data, say a number or some useful info. But that's not all - it also has a special pointer that points to the next node in the sequence.

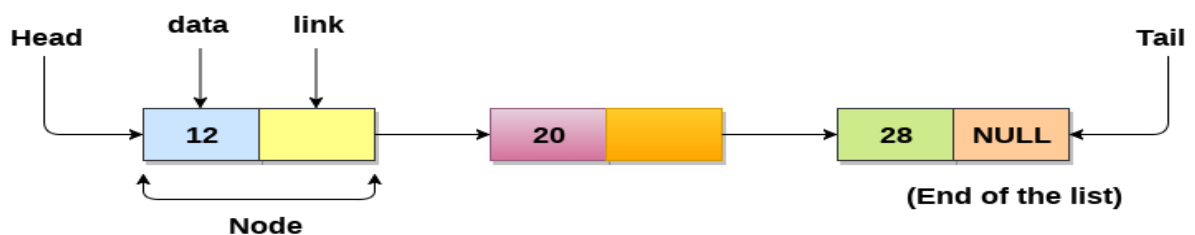
Now, unlike arrays where all the elements are lined up neatly in memory, these nodes can be scattered all over the place. The pointers are what tie them together in a chain. The first node is called the head, and the last node points to null, indicating the end of the list.

One major upside is that you don't need to pre-decide how much memory the list will need. It can keep growing organically by just adding new nodes wherever there's space available. No need to shift everything around or reallocate a bigger chunk of memory like with arrays.

Another perk is how quick and easy it is to insert or remove items. All you do is update some pointers to link or unlink a node. No large data shuffling required. This makes linked lists super handy when you're frequently adding or removing stuff.

Of course, there are some downsides too. Accessing a random element isn't as straightforward as with arrays since you have to follow the chain from the start. But for situations where you mostly work with the start or end, linked lists are awesome.

- Linked List can be defined as collection of objects called **nodes** that are randomly stored in the memory.
- A node contains two fields i.e. data stored at that particular address and the pointer which contains the address of the next node in the memory.
- The last node of the list contains pointer to the null.
- Linked lists are a way to store data in a flexible and dynamic manner
- Nodes are connected by references/pointers, not stored together in memory
- This allows the list to grow or shrink easily as needed
- No need to pre-allocate a fixed amount of memory like arrays
- Inserting and removing data is efficient like just updating pointers
- Linked lists offer versatility through different variations
- It can build doubly linked lists, circular lists, trees, graphs, etc.
- Widely used in algorithms, data processing, memory management



Uses of Linked List in Real World

Linked lists find numerous real-world applications due to their dynamic nature and efficiency in certain scenarios. Let's explore some of these practical uses:

- **Memory management:** Operating systems and programming language runtimes often utilize linked lists for dynamic memory allocation and deallocation. When you request memory, a new node is added to the linked list, and when you free memory, that node is removed.
- **Browser history and cache:** Modern web browsers use linked lists to keep track of visited URLs and cache recently visited pages for faster loading. Each visited page is added as a new node, and the cache management algorithm decides which nodes to keep or remove based on factors like frequency and recency of access.
- **Music playlists:** Streaming services and media players represent playlists as linked lists. Each song is a node, and the order of nodes defines the playback sequence. Adding, removing, or rearranging songs is efficient with linked lists.
- **Undo/Redo operations:** Similar to stacks, linked lists can be used to implement undo/redo functionality in text editors and other applications. Each editing operation is stored as a node, allowing users to traverse the list in either direction to undo or redo changes.
- **Routing algorithms:** Linked lists are used in routing algorithms for computer networks and transportation systems. Each node represents a network node or a location, and the links between nodes represent the connections or routes.
- **Online multiplayer games:** In real-time multiplayer games, linked lists are often used to manage the active player list. When a new player joins, a node is added to the list, and when a player leaves, their node is removed.
- **Blockchain technology:** The blockchain, which underpins cryptocurrencies like Bitcoin, is essentially a linked list. Each block (node) contains transaction data and is linked to the previous block, forming an immutable chain of records.
- **Image viewer applications:** Image viewer applications may use linked lists to store and navigate through a collection of images. Each node represents an image, and users can move forward or backward through the list.
- **Polynomial representation:** In computer algebra systems, polynomials can be efficiently represented using linked lists, where each node stores the coefficient and exponent of a term.
- **Implementing other data structures:** Linked lists can be used as building blocks for more complex data structures like stacks, queues, and hash tables, which are widely used in various applications.

These are just a few examples that highlight the versatility of linked lists in real-world scenarios. Their dynamic nature, efficient insertion and deletion operations, and ability to grow or shrink as needed make them a valuable tool in many domains.

Why use linked list over array?

We've been rocking arrays to organize grouped elements stored individually in memory. But arrays, as awesome as they are, come with their own set of limitations that make us reconsider our options in certain situations. It's good to know when linked lists can swoop in and save the day!

Array limitations we need to watch out for:

1. **Fixed size:** Before using an array, we need to know just how much space it'll need. Resizing is a serious pain once the party's started.
2. **Inflexible memory layout:** Array elements must be contiguously stored in memory. Inserting new ones requires reshuffling the whole squad, predecessor by predecessor.
3. **Rigid in Nature:** Arrays are stuck with the initial size they were created with. Expanding becomes an expensive reallocation operation at runtime.

Advantages of Linked Lists:

1. **Dynamic Size:** Unlike arrays, linked lists can grow or shrink dynamically during runtime as needed. This flexibility allows efficient memory utilization, especially when working with large or variable-sized data sets.
2. **Efficient Insertion and Deletion:** Adding or removing elements from a linked list is relatively simple and efficient. It only requires updating the next pointers of the surrounding nodes, without the need for costly operations like shifting or reallocating memory.
3. **Discontinuous Memory Allocation:** Linked lists can store their nodes in non-contiguous memory locations, allowing better memory management and utilization, especially in situations where contiguous blocks of memory are scarce.
4. **Versatility:** Linked lists can be easily adapted to implement other data structures like stacks, queues, and even advanced structures like trees and graphs. This versatility makes them a valuable tool in various algorithms and applications.
5. **Insertion Order Preservation:** Linked lists naturally maintain the order of insertion, which can be beneficial in certain applications where the order of elements is essential.

Disadvantages of Linked Lists:

1. **Random Access Inefficiency:** Accessing an element at a specific index in a linked list requires traversing from the beginning or the end, making random access operations relatively slow compared to arrays.
2. **Extra Memory Overhead:** Each node in a linked list requires additional memory to store the pointer(s) to the next (and possibly previous) node(s), leading to higher memory overhead than arrays.
3. **Lack of Cache Locality:** Linked list nodes are scattered in memory, which can lead to poor cache locality and performance issues, especially in operations that require frequent traversals.
4. **Null Termination Requirement:** Linked lists require a special terminating node (usually null) to indicate the end of the list, adding an extra step in certain operations.
5. **Complexity in Reverse Traversal:** In singly linked lists, reversing or traversing the list in reverse order is not straightforward and may require additional operations or a separate implementation.
6. **Risk of Losing Data:** If a node's pointer is inadvertently overwritten or corrupted, it can lead to a broken link and potentially lose access to a portion of the linked list, resulting in data loss.

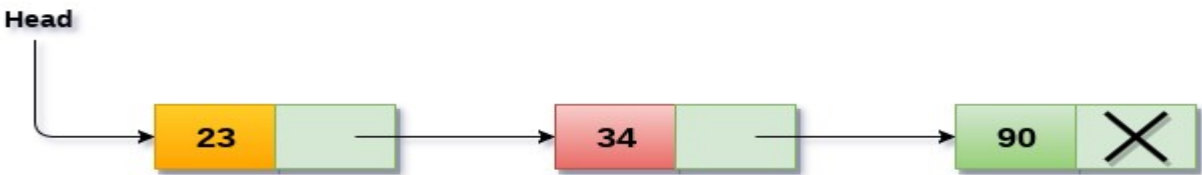
While linked lists have their advantages in terms of dynamic size, efficient insertion/deletion, and versatility, they also come with trade-offs in terms of random access performance, extra memory overhead, and potential complexities in certain operations. The choice between using a linked list or an array ultimately depends on the specific requirements of the application, such as the expected size of the data set, the frequency of insertions and deletions, and the importance of random access performance.

Singly linked list or One way chain

Singly linked list can be defined as the collection of ordered set of elements. The number of elements may vary according to need of the program. A node in the singly linked list consist of two parts: data part and link part. Data part of the node stores actual information that is to be represented by the node while the link part of the node stores the address of its immediate successor.

One way chain or singly linked list can be traversed only in one direction. In other words, we can say that each node contains only next pointer, therefore we can not traverse the list in the reverse direction.

Consider an example where the marks obtained by the student in three subjects are stored in a linked list as shown in the figure.



In the above figure, the arrow represents the links. The data part of every node contains the marks obtained by the student in the different subject. The last node in the list is identified by the null pointer which is present in the address part of the last node. We can have as many elements we require, in the data part of the list.

Complexity

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Singly Linked List	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$

Operations on Singly Linked List

There are various operations which can be performed on singly linked list. A list of all such operations is given below.

Node Creation

```
1. struct node
2. {
3.     int data;
4.     struct node *next;
5. };
6. struct node *head, *ptr;
7. ptr = (struct node *)malloc(sizeof(struct node *));
```

Insertion

The insertion into a singly linked list can be performed at different positions. Based on the position of the new node being inserted, the insertion is categorized into the following categories.

SN	Operation	Description
1	Insertion at beginning	It involves inserting any element at the front of the list. We just need to a few link adjustments to make the new node as the head of the list.
2	Insertion at end of the list	It involves insertion at the last of the linked list. The new node can be inserted as the only node in the list or it can be inserted as the last one. Different logics are implemented in each scenario.
3	Insertion after specified node	It involves insertion after the specified node of the linked list. We need to skip the desired number of nodes in order to reach the node after which the new node will be inserted.

Deletion and Traversing

The Deletion of a node from a singly linked list can be performed at different positions. Based on the position of the node being deleted, the operation is categorized into the following categories.

SN	Operation	Description
1	Deletion at beginning	It involves deletion of a node from the beginning of the list. This is the simplest operation among all. It just need a few adjustments in the node pointers.

- | | | |
|---|--|--|
| 2 | Deletion at the end of the list | It involves deleting the last node of the list. The list can either be empty or full. Different logic is implemented for the different scenarios. |
| 3 | Deletion after specified node | It involves deleting the node after the specified node in the list. we need to skip the desired number of nodes to reach the node after which the node will be deleted. This requires traversing through the list. |
| 4 | Traversing | In traversing, we simply visit each node of the list at least once in order to perform some specific operation on it, for example, printing data part of each node present in the list. |
| 5 | Searching | In searching, we match each element of the list with the given element. If the element is found on any of the location then location of that element is returned otherwise null is returned. |

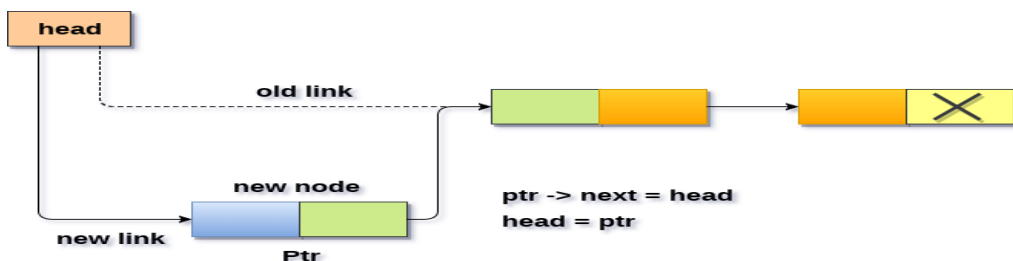
Insertion in singly linked list at beginning

Inserting a new element into a singly linked list at beginning is quite simple. We just need to make a few adjustments in the node links. There are the following steps which need to be followed in order to insert a new node in the list at beginning.

- Allocate the space for the new node and store data into the data part of the node. This will be done by the following statements.
 1. `ptr = (struct node *) malloc(sizeof(struct node *));`
 2. `ptr → data = item`
- Make the link part of the new node pointing to the existing first node of the list. This will be done by using the following statement.
 1. `ptr->next = head;`
- At the last, we need to make the new node as the first node of the list this will be done by using the following statement.
 1. `head = ptr;`

Algorithm

- **Step 1:** IF PTR = NULL
Write OVERFLOW
Go to Step 7
[END OF IF]
- **Step 2:** SET NEW_NODE = PTR
- **Step 3:** SET PTR = PTR → NEXT
- **Step 4:** SET NEW_NODE → DATA = VAL
- **Step 5:** SET NEW_NODE → NEXT = HEAD
- **Step 6:** SET HEAD = NEW_NODE
- **Step 7:** EXIT



Example

```
#include <iostream>
#include <stdio.h>
using namespace std;
```

```
struct Node
{
    int data;
```

```

    Node* next;
};
/*
//Pass by reference
void insertathead(Node* &head, int val)
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = head;
    head = newNode;
}
*/
//Pass by pointer
void insertathead(Node** head, int val)
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = *head;
    *head = newNode;
}

void displayNode(Node* head)
{
    if(head == nullptr)
    {
        cout << "Linked list is empty.\n";
        return;
    }
    Node* temp = head;
    while(temp != nullptr)
    {
        cout << temp->data << "->";
        temp = temp->next;
    }
    cout << "NULL\n";
}

int main()
{
    Node* head = nullptr;
    int n;
    cout << "Enter the size of linked list: ";
    cin >> n;
    int val;
    for(int i=0; i<n; i++)
    {
        cin >> val;
        //Pass by reference
        //insertathead(head, val);
        //Pass by Pointer
        insertathead(&head, val);
    }
    displayNode(head);
    return 0;
}

```

```
}
```

Output

```
./add_at_head_ll
```

Enter the size of linked list: 5

```
1 2 3 4 5
```

```
5->4->3->2->1->NULL
```

Insertion in singly linked list at the end

In order to insert a node at the last, there are two following scenarios which need to be mentioned.

1. The node is being added to an empty list
2. The node is being added to the end of the linked list

in the first case,

- The condition (`head == NULL`) gets satisfied. Hence, we just need to allocate the space for the node by using `malloc` statement in C. Data and the link part of the node are set up by using the following statements.

1. `ptr->data = item;`
2. `ptr->next = NULL;`

- Since, **ptr** is the only node that will be inserted in the list hence, we need to make this node pointed by the head pointer of the list. This will be done by using the following Statements.

1. `Head = ptr`

In the second case,

- The condition **Head = NULL** would fail, since Head is not null. Now, we need to declare a temporary pointer **temp** in order to traverse through the list. **temp** is made to point the first node of the list.

1. `Temp = head`

- Then, traverse through the entire linked list using the statements:

1. `while (temp -> next != NULL)`

2. `temp = temp -> next;`

- At the end of the loop, the temp will be pointing to the last node of the list.

Now, allocate the space for the new node, and assign the item to its data part.

Since, the new node is going to be the last node of the list hence, the next part of this node needs to be pointing to the **null**. We need to make the next part of the temp node (which is currently the last node of the list) point to the new node (ptr) .

1. `temp = head;`

2. `while (temp -> next != NULL)`

3. `{`

4. `temp = temp -> next;`

5. `}`

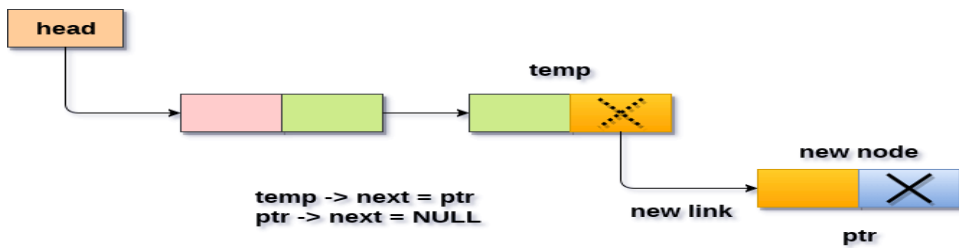
6. `temp->next = ptr;`

7. `ptr->next = NULL;`

Algorithm

- **Step 1:** IF PTR = NULL Write OVERFLOW
Go to Step 1
[END OF IF]
- **Step 2:** SET NEW_NODE = PTR
- **Step 3:** SET PTR = PTR - > NEXT
- **Step 4:** SET NEW_NODE - > DATA = VAL
- **Step 5:** SET NEW_NODE - > NEXT = NULL
- **Step 6:** SET PTR = HEAD
- **Step 7:** Repeat Step 8 while PTR - > NEXT != NULL

- **Step 8:** SET PTR = PTR - > NEXT
[END OF LOOP]
- **Step 9:** SET PTR - > NEXT = NEW_NODE
- **Step 10:** EXIT



Inserting node at the last into a non-empty list

Example

```
#include <iostream>
```

```
#include <stdio.h>
```

```
using namespace std;
```

```
struct Node
```

```
{
    int data;
    Node* next;
};
```

```
void addatend(Node* &head, int val)
```

```
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = nullptr;

    if(head == nullptr)
    {
        head = newNode;
        return;
    }
    Node* temp = head;
    while(temp->next != nullptr)
    {
        temp = temp->next;
    }
    temp->next = newNode;
}
```

```
void display(Node* head)
```

```
{
    if (head == nullptr)
    {
        cout << "List is empty.\n";
        return;
    }
    Node* temp = head;
    while(temp != nullptr)
    {
```

```

        cout << temp->data << "->";
        temp = temp->next;
    }
    cout << "NULL\n";
}

int main()
{
    Node* head = nullptr;
    int n;
    cout << "Enter the size of linked list:";
    cin >> n;
    int val;
    for(int i=0; i<n; i++)
    {
        cin >> val;
        addatend(head, val);
    }
    display(head);
    return 0;
}

```

Output

./add_at_tail_ll

Enter the size of linked list:5

1 2 3 4 5

1->2->3->4->5->NULL

Insertion in singly linked list after specified Node

- In order to insert an element after the specified number of nodes into the linked list, we need to skip the desired number of elements in the list to move the pointer at the position after which the node will be inserted. This will be done by using the following statements.

```

1. emp=head;
2.     for(i=0;i<loc;i++)
3.     {
4.         temp = temp->next;
5.         if(temp == NULL)
6.         {
7.             return;
8.         }
9.
10.    }

```

- Allocate the space for the new node and add the item to the data part of it. This will be done by using the following statements.

```

1. ptr = (struct node *) malloc (sizeof(struct node));
2.     ptr->data = item;

```

- Now, we just need to make a few more link adjustments and our node at will be inserted at the specified position. Since, at the end of the loop, the loop pointer temp would be pointing to the node after which the new node will be inserted. Therefore, the next part of the new node ptr must contain the address of the next part of the temp (since, ptr will be in between temp and the next of the temp). This will be done by using the following statements.

```

1. ptr → next = temp → next

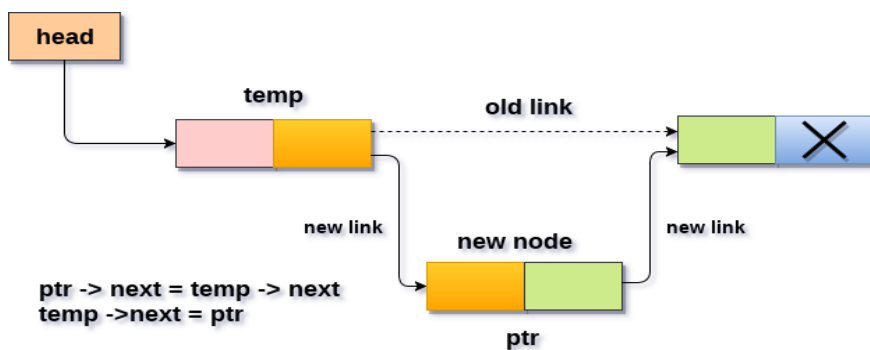
```

now, we just need to make the next part of the temp, point to the new node ptr. This will insert the new node ptr, at the specified position.

1. temp ->next = ptr;

Algorithm

- **STEP 1:** IF PTR = NULL
WRITE OVERFLOW
GOTO STEP 12
END OF IF
- **STEP 2:** SET NEW_NODE = PTR
- **STEP 3:** NEW_NODE → DATA = VAL
- **STEP 4:** SET TEMP = HEAD
- **STEP 5:** SET I = 0
- **STEP 6:** REPEAT STEP 5 AND 6 UNTIL I
- **STEP 7:** TEMP = TEMP → NEXT
- **STEP 8:** IF TEMP = NULL
WRITE "DESIRED NODE NOT PRESENT"
GOTO STEP 12
END OF IF
- **STEP 9:** PTR → NEXT = TEMP → NEXT
- **STEP 10:** TEMP → NEXT = PTR
- **STEP 11:** SET PTR = NEW_NODE
- **STEP 12:** EXIT



Example

```
#include <iostream>
#include <stdio.h>
```

```
using namespace std;
```

```
struct Node {
    int data;
    Node* next;
};
```

```
void addathead(Node* &head, int val)
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = head;
    head = newNode;
}
```

```
void display(Node* head)
{
```

```

if(head == nullptr)
{
    cout << "List is empty.\n";
    return;
}
Node* temp = head;
while(temp != nullptr)
{
    cout << temp->data << "->";
    temp = temp->next;
}
cout << "NULL\n";
}

void addatspecific(Node* &head, int pos, int val)
{
    if(pos < 0)
    {
        cout << "Invalid position.\n";
        return;
    }
    if(pos == 0)
    {
        addathead(head, val);
        return;
    }
    int cnt = 0;
    Node* temp = head;
    while(temp != nullptr && cnt < pos - 1)
    {
        temp = temp->next;
        cnt++;
    }
    if(temp == nullptr)
    {
        cout << "Position out of range.\n";
        return;
    }
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = temp->next;
    temp->next = newNode;
}

int main()
{
    Node* head = nullptr;
    int n;
    cout << "Enter the size of linked list: ";
    cin >> n;
    int val;
    for(int i=0;i<n;i++)
    {
        cin >> val;

```

```

        addathead(head, val);
    }
    display(head);
    int pos, add;
    cout << "Enter position and element to add in linked list: ";
    cin >> pos >> add;
    addatspecific(head, pos, add);
    display(head);
    return 0;
}

```

Output

```
./add_at_specific_ll
```

Enter the size of linked list: 5

1 2 3 4 5

5->4->3->2->1->NULL

Enter position and element to add in linked list: 3 9

5->4->3->9->2->1->NULL

Deletion in singly linked list at beginning

Deleting a node from the beginning of the list is the simplest operation of all. It just need a few adjustments in the node pointers. Since the first node of the list is to be deleted, therefore, we just need to make the head, point to the next of the head. This will be done by using the following statements.

1. ptr = head;
2. head = ptr->next;

Now, free the pointer ptr which was pointing to the head node of the list. This will be done by using the following statement.

1. free(ptr)

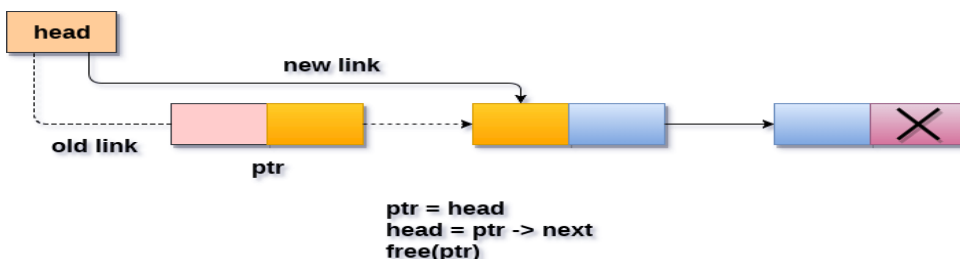
Algorithm

Write UNDERFLOW

Go to Step 5

[END OF IF]

- **Step 1:** IF HEAD = NULL
- **Step 2:** SET PTR = HEAD
- **Step 3:** SET HEAD = HEAD -> NEXT
- **Step 4:** FREE PTR
- **Step 5:** EXIT



Deleting a node from the beginning

Example

```
#include <iostream>
```

```
#include <stdio.h>
```

```
using namespace std;
```

```
struct Node {  
    int data;  
    Node* next;  
};
```

```
void addathead(Node* &head, int val)  
{  
    Node* newNode = new Node();  
    newNode->data = val;  
    newNode->next = head;  
    head = newNode;  
}
```

```
void deleteatbeg(Node* &head)  
{  
    if(head == nullptr)  
    {  
        cout << "List is Empty.\n";  
        return ;  
    }  
    Node* temp = head;  
    head = head->next;  
    delete temp;  
}
```

```
void displayList(Node* head)  
{  
    if(head == nullptr)  
    {  
        cout << "List is empty.\n";  
        return;  
    }  
    Node* temp = head;  
    while(temp != nullptr)  
    {  
        cout << temp->data << "->";  
        temp = temp->next;  
    }  
    cout << "NULL\n";  
}
```

```
int main()  
{  
    Node* head = nullptr;  
    int n;  
    cout << "Enter the size of the linked list: ";  
    cin >> n;  
    int val;  
    for(int i=0;i<n;i++)  
    {  
        cin >> val;  
        addathead(head, val);  
    }
```

```

    }
    displayList(head);
    deleteatbeg(head);
    displayList(head);
    return 0;
}

```

Output

```
./delete_at_beg_ll
```

Enter the size of the linked list: 5

```
1 2 3 4 5
```

```
5->4->3->2->1->NULL
```

```
4->3->2->1->NULL
```

Deletion in singly linked list at the end

There are two scenarios in which, a node is deleted from the end of the linked list.

1. There is only one node in the list and that needs to be deleted.
2. There are more than one node in the list and the last node of the list will be deleted.

In the first scenario,

the condition `head → next = NULL` will survive and therefore, the only node head of the list will be assigned to null. This will be done by using the following statements.

1. `ptr = head`
2. `head = NULL`
3. `free(ptr)`

In the second scenario,

The condition `head → next = NULL` would fail and therefore, we have to traverse the node in order to reach the last node of the list.

For this purpose, just declare a temporary pointer `temp` and assign it to head of the list. We also need to keep track of the second last node of the list. For this purpose, two pointers `ptr` and `ptr1` will be used where `ptr` will point to the last node and `ptr1` will point to the second last node of the list.

this all will be done by using the following statements.

1. `ptr = head;`
2. `while(ptr->next != NULL)`
3. `{`
4. `ptr1 = ptr;`
5. `ptr = ptr->next;`
6. `}`

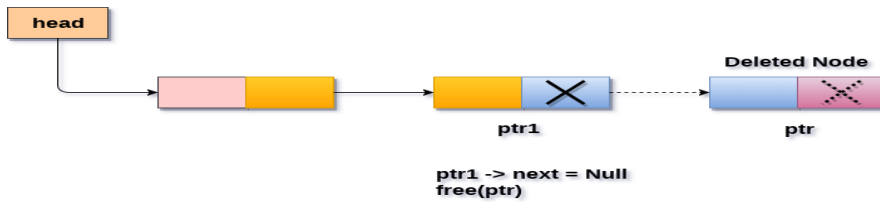
Now, we just need to make the pointer `ptr1` point to the NULL and the last node of the list that is pointed by `ptr` will become free. It will be done by using the following statements.

1. `ptr1->next = NULL;`
2. `free(ptr);`

Algorithm

- **Step 1:** IF HEAD = NULL
Write UNDERFLOW
Go to Step 8
[END OF IF]
- **Step 2:** SET PTR = HEAD
- **Step 3:** Repeat Steps 4 and 5 while PTR -> NEXT!= NULL
- **Step 4:** SET PREPTR = PTR
- **Step 5:** SET PTR = PTR -> NEXT
[END OF LOOP]
- **Step 6:** SET PREPTR -> NEXT = NULL

- **Step 7: FREE PTR**
- **Step 8: EXIT**



Deleting a node from the last

Example

```
#include <iostream>
#include <stdio.h>
```

```
using namespace std;
```

```
struct Node {
    int data;
    Node* next;
};
```

```
void addathead(Node* &head, int val)
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = head;
    head = newNode;
}
```

```
void deleteatend(Node* head)
{
    if(head == nullptr)
    {
        cout << "List is empty.\n";
        return;
    }
    if(head->next == nullptr)
    {
        delete head;
        head = nullptr;
        return;
    }
    Node* temp = head;
    while(temp->next->next != nullptr)
    {
        temp = temp->next;
    }
    delete temp->next;
    temp->next = nullptr;
}
```

```
void displayList(Node* head)
{
    if(head == nullptr)
```

```

    {
        cout << "List is empty.\n";
        return;
    }
    Node* temp = head;
    while(temp != nullptr)
    {
        cout << temp->data << "->";
        temp = temp->next;
    }
    cout << "NULL\n";
}

int main()
{
    Node* head = nullptr;
    int n;
    cout << "Enter the size of the linked list: ";
    cin >> n;
    int val;
    for(int i=0;i<n;i++)
    {
        cin >> val;
        addathead(head, val);
    }
    displayList(head);
    deleteatend(head);
    displayList(head);

    return 0;
}

```

Output

```
./delete_at_end_ll
```

```
Enter the size of the linked list: 5
```

```
1 2 3 4 5
```

```
5->4->3->2->1->NULL
```

```
5->4->3->2->NULL
```

Deletion in singly linked list after the specified node :

In order to delete the node, which is present after the specified node, we need to skip the desired number of nodes to reach the node after which the node will be deleted.

We need to keep track of the two nodes. The one which is to be deleted the other one if the node which is present before that node. For this purpose, two pointers are used: ptr and ptr1.

Use the following statements to do so.

1. ptr=head;
2. for(i=0;i<loc;i++)
3. {
4. ptr1 = ptr;
5. ptr = ptr->next;
- 6.
7. if(ptr == NULL)
8. {
9. printf("\nThere are less than %d elements in the list..",loc);

```

10.         return;
11.     }
12. }

```

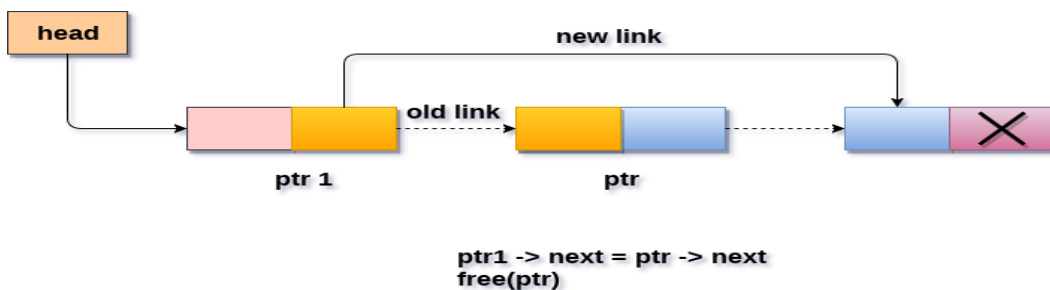
Now, our task is almost done, we just need to make a few pointer adjustments. Make the next of ptr1 (points to the specified node) point to the next of ptr (the node which is to be deleted).

This will be done by using the following statements.

1. ptr1 ->next = ptr ->next;
2. free(ptr);

Algorithm

- **STEP 1:** IF HEAD = NULL
WRITE UNDERFLOW
GOTO STEP 10
END OF IF
- **STEP 2:** SET TEMP = HEAD
- **STEP 3:** SET I = 0
- **STEP 4:** REPEAT STEP 5 TO 8 UNTIL I
- **STEP 5:** TEMP1 = TEMP
- **STEP 6:** TEMP = TEMP → NEXT
- **STEP 7:** IF TEMP = NULL
WRITE "DESIRED NODE NOT PRESENT"
GOTO STEP 12
END OF IF
- **STEP 8:** I = I+1
END OF LOOP
- **STEP 9:** TEMP1 → NEXT = TEMP → NEXT
- **STEP 10:** FREE TEMP
- **STEP 11:** EXIT



Deletion a node from specified position

Example

Output

Traversing in singly linked list

Traversing is the most common operation that is performed in almost every scenario of singly linked list. Traversing means visiting each node of the list once in order to perform some operation on that. This will be done by using the following statements.

1. ptr = head;
2. while (ptr!=NULL)
3. {
4. ptr = ptr -> next;
5. }

Algorithm

- **STEP 1:** SET PTR = HEAD
- **STEP 2:** IF PTR = NULL
 WRITE "EMPTY LIST"
 GOTO STEP 7
 END OF IF
- **STEP 4:** REPEAT STEP 5 AND 6 UNTIL PTR != NULL
- **STEP 5:** PRINT PTR → DATA
- **STEP 6:** PTR = PTR → NEXT
 [END OF LOOP]
- **STEP 7:** EXIT

Example

```
#include <iostream>
#include <stdio.h>
using namespace std;
```

```
struct Node {
    int data;
    Node* next;
};
```

```
void addathead(Node* &head, int val)
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = head;
    head = newNode;
}
```

```
void deleteatspecific(Node* head, int pos)
{
    if(head == nullptr)
    {
        cout << "List is empty.\n";
        return;
    }
    if(pos < 0)
    {
        cout << "Invalid position.\n";
        return;
    }
    if(pos == 0)
    {
        Node* temp = head;
        head = head->next;
        delete temp;
        return;
    }
    Node* curr = head;
    int cnt = 0;
    while(curr != nullptr && cnt < pos - 1)
    {
        curr = curr->next;
        cnt++;
    }
```

```

    }
    if(curr == nullptr || curr->next == nullptr)
    {
        cout << "Position out of range.\n";
        return;
    }
    Node* temp = curr->next;
    curr->next = temp->next;
    delete temp;
}

```

```

void displayList(Node* head)
{
    if(head == nullptr)
    {
        cout << "List is empty.\n";
        return;
    }
    Node* temp = head;
    while(temp != nullptr)
    {
        cout << temp->data << "->";
        temp = temp->next;
    }
    cout << "NULL\n";
}

```

```

int main()
{
    Node* head = nullptr;
    int n;
    cout << "Enter the size of linked list: ";
    cin >> n;
    int val;
    for(int i=0;i<n;i++)
    {
        cin >> val;
        addathead(head, val);
    }
    displayList(head);
    int pos;
    cout << "Enter the position to delete: ";
    cin >> pos;
    deleteatspecific(head, pos);
    displayList(head);
    return 0;
}

```

Output

```

./delete_at_specific_ll
Enter the size of linked list: 5
1 2 3 4 5
5->4->3->2->1->NULL
Enter the position to delete: 2
5->4->2->1->NULL

```

Searching in singly linked list

Searching is performed in order to find the location of a particular element in the list. Searching any element in the list needs traversing through the list and make the comparison of every element of the list with the specified element. If the element is matched with any of the list element then the location of the element is returned from the function.

Algorithm

- **Step 1:** SET PTR = HEAD
- **Step 2:** Set I = 0
- **STEP 3:** IF PTR = NULL
 WRITE "EMPTY LIST"
 GOTO STEP 8
 END OF IF
- **STEP 4:** REPEAT STEP 5 TO 7 UNTIL PTR != NULL
- **STEP 5:** if ptr → data = item
 write i+1
 End of IF
- **STEP 6:** I = I + 1
- **STEP 7:** PTR = PTR → NEXT
 [END OF LOOP]
- **STEP 8:** EXIT

Example

```
#include <iostream>
```

```
#include <stdio.h>
```

```
using namespace std;
```

```
struct Node {  
    int data;  
    Node* next;  
};
```

```
void addathead(Node* &head, int val)  
{  
    Node* newNode = new Node();  
    newNode->data = val;  
    newNode->next = head;  
    head = newNode;  
}
```

```
int searchElement(Node* &head, int find)  
{  
    if(head == NULL)  
    {  
        cout << "List is empty.\n";  
        return -1;  
    }  
    int pos = 1;  
    Node* temp = head;  
    while(temp != NULL)  
    {  
        if(temp->data == find)  
        {
```

```

        return pos;
    }
    else
    {
        temp = temp->next;
        pos++;
    }
}
return -1;
}

```

```

void displayList(Node* head)
{
    if(head == NULL)
    {
        cout << "List is Empty.\n";
        return;
    }
    Node* temp = head;
    while(temp != NULL)
    {
        cout << temp->data << "->";
        temp = temp->next;
    }
    cout << "NULL\n";
}

```

```

int main()
{
    Node* head = NULL;
    int n;
    cout << "Enter the size of linked list: ";
    cin >> n;
    int val;
    for(int i=0;i<n;i++)
    {
        cin >> val;
        addathead(head, val);
    }
    displayList(head);
    int match;
    cout << "Enter the element to search: ";
    cin >> match;
    int res = searchElement(head, match);
    cout << "Result: " << res << endl;
    return 0;
}

```

Output

./search_ll

Enter the size of linked list: 5

1 2 3 4 5

5->4->3->2->1->NULL

Enter the element to search: 4

Result: 2

Reverse Liked List

Example

```
#include <iostream>
```

```
#include <stdio.h>
```

```
using namespace std;
```

```
struct Node {  
    int data;  
    Node* next;  
};
```

```
void addathead(Node* &head, int val)  
{  
    Node* newNode = new Node();  
    newNode->data = val;  
    newNode->next = head;  
    head = newNode;  
}
```

```
Node* reverse_ll(Node* head)  
{  
    Node* prev = NULL;  
    Node* temp = head;  
    while(temp != NULL)  
    {  
        Node* newNode = temp->next;  
        temp->next = prev;  
        prev = temp;  
        temp = newNode;  
    }  
    return prev;  
}
```

```
void displayList(Node* head)  
{  
    if(head == NULL)  
    {  
        cout << "List is Empty.\n";  
        return;  
    }  
    Node* temp = head;  
    while(temp != NULL)  
    {  
        cout << temp->data << "->";  
        temp = temp->next;  
    }  
    cout << "NULL\n";  
}
```

```
int main()  
{  
    Node* head = NULL;  
    int n, val;
```

```

    cout << "Enter the size of the linked list: ";
    cin >> n;
    for(int i=0;i<n;i++)
    {
        cin >> val;
        addathead(head, val);
    }
    cout << "Linked List before reverse: ";
    displayList(head);
    head = reverse_ll(head);
    cout << "Linked List after reverse: ";
    displayList(head);
    return 0;
}

```

Output

```

./reverse_ll
Enter the size of the linked list: 5
1 2 3 4 5
Linked List before reverse: 5->4->3->2->1->NULL
Linked List after reverse: 1->2->3->4->5->NULL

```

Delete by Value

Example

```

#include <iostream>
#include <stdio.h>

using namespace std;

struct Node {
    int data;
    Node* next;
};

void addathead(Node* &head, int val)
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = head;
    head = newNode;
}

void deleteMatchElement(Node* &head, int find)
{
    if(head == nullptr)
    {
        cout << "List is Empty.\n";
        return;
    }
    while(head != NULL && head->data == find)
    {
        Node* toDelete = head;
        head = head->next;
        delete toDelete;
    }
}

```

```

Node* temp = head;
while(temp != NULL && temp->next != NULL)
{
    if(temp->next->data == find)
    {
        Node* toDelete = temp->next;
        temp->next = temp->next->next;
        delete toDelete;
    }
    else
    {
        temp = temp->next;
    }
}

void displayList(Node* head)
{
    if(head == NULL)
    {
        cout << "List is empty.\n";
        return;
    }
    Node* temp = head;
    while(temp != NULL)
    {
        cout << temp->data << "->";
        temp = temp->next;
    }
    cout << "NULL\n";
}

int main()
{
    Node* head = NULL;
    int n;
    cout << "Enter the size of the linked list: ";
    cin >> n;
    int val;
    for(int i=0;i<n;i++)
    {
        cin >> val;
        addathead(head, val);
    }
    displayList(head);
    int match;
    cout << "Enter element to delete from linked list: ";
    cin >> match;
    deleteMatchElement(head, match);
    displayList(head);
    return 0;
}

```

Output

./delete_match_data_ll

Enter the size of the linked list: 5

1 2 3 4 5

5->4->3->2->1->NULL

Enter element to delete from linked list: 3

5->4->2->1->NULL

Swap by position from both ends

Example

```
#include <iostream>
```

```
#include <stdio.h>
```

```
using namespace std;
```

```
struct Node {  
    int data;  
    Node* next;  
};
```

```
void addAtHead(Node* &head, int val)  
{  
    Node* newNode = new Node();  
    newNode->data = val;  
    newNode->next = head;  
    head = newNode;  
}
```

```
void displayList(Node* head)  
{  
    if(head == NULL)  
    {  
        cout << "List is Empty.\n";  
        return;  
    }  
    Node* temp = head;  
    while(temp != NULL)  
    {  
        cout << temp->data << "->";  
        temp = temp->next;  
    }  
    cout << "NULL\n";  
}
```

```
int getlength(Node* head)  
{  
    int count = 0;  
    Node* temp = head;  
    while(temp != NULL)  
    {  
        count++;  
        temp = temp->next;  
    }  
    return count;  
}
```

```

void replace_of_k(Node* head, int k)
{
    if(head == NULL)
    {
        cout << "List is Empty.\n";
        return;
    }
    int len = getlength(head);
    if(k < 1 || k > len)
    {
        cout << "Invalid position.\n";
        return;
    }
    Node* fromStart = head;
    for(int i=1;i<k;i++)
    {
        fromStart = fromStart->next;
    }
    Node* fromEnd = head;
    for(int i=1;i<len-k+1;i++)
    {
        fromEnd = fromEnd->next;
    }

    fromStart->data = fromStart->data + fromEnd->data;
    fromEnd->data = fromStart->data - fromEnd->data;
    fromStart->data = fromStart->data - fromEnd->data;
}

```

```

int main()
{
    Node* head = NULL;
    int n;
    cout << "Enter the size of Liked List: ";
    cin >> n;
    int val;
    for(int i=0;i<n;i++)
    {
        cin >> val;
        addAtHead(head, val);
    }
    displayList(head);
    int k;
    cout << "Place: ";
    cin >> k;
    replace_of_k(head, k);
    displayList(head);
    return 0;
}

```

Output

./data_replace_by_k

Enter the size of Liked List: 10

1 3 5 7 9 11 13 15 17 19

19->17->15->13->11->9->7->5->3->1->NULL

Place: 3

19->17->5->13->11->9->7->15->3->1->NULL

Complete Linked List

Example

```
#include <iostream>
```

```
using namespace std;
```

```
struct Node {  
    int data;  
    Node* next;  
};
```

```
/* Insert at beginning */
```

```
void insertAtStart(Node*& head, int val) {  
    Node* temp = new Node();  
    temp->data = val;  
    temp->next = head;  
    head = temp;  
}
```

```
/* Insert at end */
```

```
void insertAtEnd(Node*& head, int val) {  
    Node* temp = new Node();  
    temp->data = val;  
    temp->next = NULL;
```

```
    if (head == NULL) {  
        head = temp;  
        return;  
    }
```

```
    Node* curr = head;  
    while (curr->next != NULL)  
        curr = curr->next;
```

```
    curr->next = temp;  
}
```

```
/* Insert at specific position (1-based index) */
```

```
void insertAtPosition(Node*& head, int val, int pos) {  
    if (pos == 1) {  
        insertAtStart(head, val);  
        return;  
    }
```

```
    Node* temp = new Node();  
    temp->data = val;
```

```
    Node* curr = head;  
    for (int i = 1; i < pos - 1 && curr != NULL; i++)  
        curr = curr->next;
```

```
    if (curr == NULL) {
```

```

        cout << "Position out of range\n";
        return;
    }

    temp->next = curr->next;
    curr->next = temp;
}

/* Delete from beginning */
void deleteAtStart(Node*& head) {
    if (head == NULL) return;

    Node* temp = head;
    head = head->next;
    delete temp;
}

/* Delete from end */
void deleteAtEnd(Node*& head) {
    if (head == NULL) return;

    if (head->next == NULL) {
        delete head;
        head = NULL;
        return;
    }

    Node* curr = head;
    while (curr->next->next != NULL)
        curr = curr->next;

    delete curr->next;
    curr->next = NULL;
}

/* Delete by value */
void deleteByValue(Node*& head, int val) {
    if (head == NULL) return;

    if (head->data == val) {
        deleteAtStart(head);
        return;
    }

    Node* curr = head;
    while (curr->next != NULL && curr->next->data != val)
        curr = curr->next;

    if (curr->next == NULL) {
        cout << "Value not found\n";
        return;
    }

    Node* temp = curr->next;

```

```

    curr->next = temp->next;
    delete temp;
}

/* Display list */
void display(Node* head) {
    while (head != NULL) {
        cout << head->data << " -> ";
        head = head->next;
    }
    cout << "NULL\n";
}

int main() {
    Node* head = NULL;

    insertAtEnd(head, 10);
    insertAtEnd(head, 20);
    insertAtEnd(head, 30);
    insertAtStart(head, 5);
    insertAtPosition(head, 15, 3);

    cout << "Original List:\n";
    display(head);

    /* Reverse linked list (ONLY in main) */
    Node *prev = NULL, *curr = head, *next = NULL;

    while (curr != NULL) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    head = prev;

    cout << "Reversed List:\n";
    display(head);

    deleteAtStart(head);
    deleteAtEnd(head);
    deleteByValue(head, 15);

    cout << "After Deletions:\n";
    display(head);

    return 0;
}

```

Reverse Linked List from k to Last

Example

```

#include <iostream>
#include <stdio.h>

```

```
using namespace std;
```

```
struct Node {  
    int data;  
    Node* next;  
};
```

```
void addAtHead(Node* &head, int val)  
{  
    Node* newNode = new Node();  
    newNode->data = val;  
    newNode->next = head;  
    head = newNode;  
}
```

```
void addAtTail(Node* &head, int val)  
{  
    Node* newNode = new Node();  
    newNode->data = val;  
    newNode->next = NULL;  
  
    if(head == NULL)  
    {  
        head = newNode;  
        return;  
    }  
  
    Node* temp = head;  
    while(temp->next != NULL)  
    {  
        temp = temp->next;  
    }  
    temp->next = newNode;  
}
```

```
void printList(Node* head)  
{  
    if(head == NULL)  
    {  
        cout << "List is Empty.\n";  
        return;  
    }  
  
    Node* temp = head;  
    while(temp != NULL)  
    {  
        cout << temp->data << "->";  
        temp = temp->next;  
    }  
    cout << "NULL\n";  
}
```

```
int getLength(Node* head)  
{
```

```

int len = 0;
Node* temp = head;
while(temp != NULL)
{
    len++;
    temp = temp->next;
}
return len;
}

Node* reverse_from_k_pos(Node* head, int k)
{
    if (head == NULL || k <= 1)
        return head;

    Node* prev = NULL;
    Node* curr = head;
    for(int i=1; i<k && curr != NULL; i++)
    {
        prev = curr;
        curr = curr->next;
    }
    if(curr == NULL)
    {
        return head;
    }

    Node* lastNodebefK = prev;
    Node* kthNode = curr;

    Node* nextNode = NULL;
    prev = NULL;

    while(curr != NULL)
    {
        nextNode = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextNode;
    }
    lastNodebefK->next = prev;
    kthNode->next = NULL;

    return head;
}

int main()
{
    Node* head = NULL;
    int n, val;
    cout << "Enter the size of linked list: ";
    cin >> n;
    for(int i=0; i<n; i++)
    {

```

```

        cin >> val;
        if(val % 2 == 0)
            addAtHead(head, val);
        else
            addAtTail(head, val);
    }
    printList(head);
    int len = getLength(head);
    cout << "Length of List: " << len << endl;
    int k;
    cout << "Enter the Position: ";
    cin >> k;
    head = reverse_from_k_pos(head, k);
    printList(head);
    return 0;
}

```

Output

```

./reverse_from_k_pos
Enter the size of linked list: 10
1 2 3 4 5 6 7 8 9 10
10->8->6->4->2->1->3->5->7->9->NULL
Length of List: 10
Enter the Position: 4
10->8->6->9->7->5->3->1->2->4->NULL

```

Reverse Linked List from k to m

Example

```

#include <iostream>
#include <stdio.h>

```

```

using namespace std;

```

```

struct Node {
    int data;
    Node* next;
};

```

```

void addAtHead(Node* &head, int val)
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = head;
    head = newNode;
}

```

```

void addAtTail(Node* &head, int val)
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = NULL;

    if(head == NULL)
    {
        head = newNode;
    }
}

```

```

        return;
    }

    Node* temp = head;
    while(temp->next != NULL)
    {
        temp = temp->next;
    }
    temp->next = newNode;
}

void printList(Node* head)
{
    if(head == NULL)
    {
        cout << "List is Empty.\n";
        return;
    }

    Node* temp = head;
    while(temp != NULL)
    {
        cout << temp->data << "->";
        temp = temp->next;
    }
    cout << "NULL\n";
}

Node* reverse_from_k_to_m_pos(Node* head, int k, int m)
{
    if (head == NULL || k == m)
        return head;

    Node dummy;
    dummy.next = head;
    Node* prev = &dummy;
    for (int i = 1; i < k; i++)
    {
        if (prev->next == NULL)
            return head;
        prev = prev->next;
    }
    Node* curr = prev->next;
    Node* next = NULL;

    for (int i = 0; i < m - k; i++) {
        next = curr->next;
        curr->next = next->next;
        next->next = prev->next;
        prev->next = next;
    }
    return dummy.next;
}

```

```

int main()
{
    Node* head = NULL;
    int n, val;
    cout << "Enter the size of linked list: ";
    cin >> n;
    for(int i=0;i<n;i++)
    {
        cin >> val;
        if(val % 2 == 0)
            addAtHead(head, val);
        else
            addAtTail(head, val);
    }
    printList(head);
    int k, m;
    cout << "Enter the Position from reverse: ";
    cin >> k;
    cout << "Enter the Position till reverse: ";
    cin >> m;
    head = reverse_from_k_to_m_pos(head, k, m);
    printList(head);
    return 0;
}

```

Output

```

./reverse_from_k_to_m_pos
Enter the size of linked list: 10
1 2 3 4 5 6 7 8 9 10
10->8->6->4->2->1->3->5->7->9->NULL
Enter the Position from reverse: 3
Enter the Position till reverse: 7
10->8->3->1->2->4->6->5->7->9->NULL

```

Update Value By Position

Example

```

#include <iostream>
#include <stdio.h>

```

```

using namespace std;

```

```

struct Node {
    int data;
    Node* next;
};

```

```

void addAtHead(Node* &head, int val)
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = head;
    head = newNode;
}

```

```

void addAtTail(Node* &head, int val)

```

```

{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = NULL;

    if(head == NULL)
    {
        head = newNode;
        return;
    }

    Node* temp = head;
    while(temp->next != NULL)
    {
        temp = temp->next;
    }
    temp->next = newNode;
}

void printList(Node* head)
{
    if(head == NULL)
    {
        cout << "List is Empty.\n";
        return;
    }

    Node* temp = head;
    while(temp != NULL)
    {
        cout << temp->data << "->";
        temp = temp->next;
    }
    cout << "NULL\n";
}

void update_val_by_pos(Node* head, int pos, int newVal)
{
    if(head == NULL)
    {
        cout << "List is Empty.\n";
        return;
    }

    int cnt = 0;
    Node* temp = head;
    while(temp != NULL && cnt < pos-1)
    {
        cnt++;
        temp = temp->next;
    }
    if(temp == NULL)
    {

```

```

        cout << "Position out of range.\n";
        return;
    }
    temp->data = newVal;
}

int main()
{
    Node* head = NULL;
    int n, val;
    cout << "Enter the size of Linked List: ";
    cin >> n;
    for(int i=0;i<n;i++)
    {
        cin >> val;
        if(val % 2 == 0)
            addAtTail(head, val);
        else
            addAtHead(head, val);
    }
    printList(head);
    int pos;
    cout << "Enter the position where to update value: ";
    cin >> pos;
    int newVal;
    cout << "Enter the new Element: ";
    cin >> newVal;
    update_val_by_pos(head, pos, newVal);
    printList(head);
    return 0;
}

```

Output

```

./update_value_by_pos
Enter the size of Linked List: 7
1 2 3 4 5 6 7
7->5->3->1->2->4->6->NULL
Enter the position where to update value: 3
Enter the new Element: 100
7->5->100->1->2->4->6->NULL

```

Update Value By Value

Example

```

#include <iostream>
#include <stdio.h>

```

```

using namespace std;

```

```

struct Node {
    int data;
    Node* next;
};

```

```

void addAtHead(Node* &head, int val)
{

```

```
Node* newNode = new Node();
newNode->data = val;
newNode->next = head;
head = newNode;
}
```

```
void addAtTail(Node* &head, int val)
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = NULL;

    if(head == NULL)
    {
        head = newNode;
        return;
    }

    Node* temp = head;
    while(temp->next != NULL)
    {
        temp = temp->next;
    }
    temp->next = newNode;
}
```

```
void printList(Node* head)
{
    if(head == NULL)
    {
        cout << "List is Empty.\n";
        return;
    }

    Node* temp = head;
    while(temp != NULL)
    {
        cout << temp->data << "->";
        temp = temp->next;
    }
    cout << "NULL\n";

}
```

```
void update_val_by_val(Node* head, int oldVal, int newVal)
{
    if(head == NULL)
    {
        cout << "List is Empty.\n";
        return;
    }

    Node* temp = head;
    while(temp != NULL && temp->data != oldVal)
```

```

    {
        temp = temp->next;
    }
    if(temp == NULL)
    {
        cout << "Position out of range.\n";
        return;
    }
    temp->data = newVal;
}

int main()
{
    Node* head = NULL;
    int n, val;
    cout << "Enter the size of Linked List: ";
    cin >> n;
    for(int i=0;i<n;i++)
    {
        cin >> val;
        if(val % 2 == 0)
            addAtTail(head, val);
        else
            addAtHead(head, val);
    }
    printList(head);
    int oldVal;
    cout << "Enter the old value to update value: ";
    cin >> oldVal;
    int newVal;
    cout << "Enter the new Element: ";
    cin >> newVal;
    update_val_by_val(head, oldVal, newVal);
    printList(head);
    return 0;
}

```

Output

```

./update_value_by_val
Enter the size of Linked List: 7
1 2 3 4 5 6 7
7->5->3->1->2->4->6->NULL
Enter the old value to update value: 1
Enter the new Element: 50
7->5->3->50->2->4->6->NULL

```

Combine Function

```

#include <iostream>
#include <climits>
#include <stack>
#include <unordered_map>
#include <unordered_set>
using namespace std;
struct Node {
    int data;

```

```

    Node* next;
};

// ----- Basic Setup -----

//Create Node
Node* createNode(int val)
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = NULL;
    return newNode;
}

//Insert at Start
void insertAtStart(Node*& head, int val)
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = head;
    head = newNode;
}

//Insert at End
void insertAtEnd(Node*& head, int val)
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = NULL;
    if(!head)
    {
        head = newNode;
        return;
    }
    Node* temp = head;
    while(temp->next != NULL)
    {
        temp = temp->next;
    }
    temp->next = newNode;
}

//Insert at given position
void insertAtPosition(Node*& head, int val, int pos)
{
    if(pos == 1)
    {
        insertAtStart(head, val);
        return;
    }
    Node* newNode = new Node();
    newNode->data = val;
    Node* temp = head;
    for(int i=1;i<pos-1 && temp;i++)

```

```

{
    temp = temp->next;
}
if(!temp)
{
    cout << "Position out of Range.\n";
    delete newNode;
    return;
}
newNode->next = temp->next;
temp->next = newNode;
}

```

//Insert value after target value

```

/*
void insertNewValueAfterValue(Node*& head, int tar, int val)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return;
    }
    Node* newNode = new Node();
    newNode->data = val;
    Node* temp = head;
    while(temp != NULL)
    {
        if(temp->data == tar)
        {
            newNode->next = temp->next;
            temp->next = newNode;
            return;
        }
        temp = temp->next;
    }
    cout << "Target Value not found.\n";
}
*/

```

```

void insertNewValueAfterValue(Node*& head, int tar, int val)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return;
    }
    Node* temp = head;
    while(temp)
    {
        if(temp->data == tar)
        {
            Node* newNode = createNode(val);
            newNode->next = temp->next;
            temp->next = newNode;
            return;
        }
        temp = temp->next;
    }
}

```

```

    }
    temp = temp->next;
}
cout << "Target Value not found.\n";
}

```

//Print Linked list

```
void showLL(Node* head)
```

```

{
    if(head == NULL)
    {
        cout << "Linked List is empty.\n";
        return;
    }
    Node* temp = head;
    while(temp != NULL)
    {
        cout << temp->data << "->";
        temp = temp->next;
    }
    cout << "NULL\n";
}

```

// ----- Deletion -----

//Delete From beginning

```
void deleteFromBeg(Node*& head)
```

```

{
    if(!head)
        return;
    Node* temp = head;
    head = head->next;
    delete temp;
}

```

//Delete from end

```
void deleteFromEnd(Node*& head)
```

```

{
    if(!head)
        return;
    if(!head->next)
    {
        delete head;
        head = NULL;
        return;
    }
    Node* temp = head;
    while(temp->next->next)
    {
        temp = temp->next;
    }
    delete temp->next;
    temp->next = NULL;
}

```

```
}
```

```
//Delete from position
```

```
void deleteFromPos(Node*& head, int pos)
```

```
{
```

```
    if(!head)
```

```
        return;
```

```
    if(pos == 1)
```

```
    {
```

```
        deleteFromBeg(head);
```

```
        return;
```

```
    }
```

```
    Node* temp = head;
```

```
    for(int i=1;i<pos-1 && temp; i++)
```

```
    {
```

```
        temp = temp->next;
```

```
    }
```

```
    if(!temp || !temp->next)
```

```
    {
```

```
        cout << "Position out of range.\n";
```

```
        return;
```

```
    }
```

```
    Node* delNode = temp->next;
```

```
    temp->next = delNode->next;
```

```
    delete delNode;
```

```
}
```

```
//Delete By Value
```

```
void deleteByValue(Node*& head, int val)
```

```
{
```

```
    if(!head)
```

```
    {
```

```
        cout << "Linked List is Empty.\n";
```

```
        return;
```

```
    }
```

```
    if(head->data == val)
```

```
    {
```

```
        Node* temp = head;
```

```
        head = head->next;
```

```
        delete temp;
```

```
        return;
```

```
    }
```

```
    Node* curr = head;
```

```
    while(curr->next != NULL && curr->next->data != val)
```

```
    {
```

```
        curr = curr->next;
```

```
    }
```

```
    if(curr->next == NULL)
```

```
    {
```

```
        cout << "Value not found in list.\n";
```

```
        return;
```

```
    }
```

```
    Node* temp = curr->next;
```

```

    curr->next = curr->next->next;
    delete temp;
    return;
}

```

//Delete All Occurance(Value)

```

/*
void deleteAllOccurance(Node*& head, int val)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return;
    }
    if(head != NULL && head->data == val)
    {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
    Node* curr = head;
    while(curr != NULL && curr->next != NULL)
    {
        if(curr->next->data == val)
        {
            Node* temp = curr->next;
            curr->next = curr->next->next;
            delete temp;
        }
        else
        {
            curr = curr->next;
        }
    }
}
*/

```

```

void deleteAllOccurance(Node*& head, int val)
{
    while(head && head->data == val)
    {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
    Node* curr = head;
    while(curr && curr->next)
    {
        if(curr->next->data == val)
        {
            Node* temp = curr->next;
            curr->next = temp->next;
            delete temp;
        }
        else
    }
}

```

```

    {
        curr = curr->next;
    }
}
}

```

// ----- Utilities -----

//Count Node

```

int countNode(Node*& head)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return 0;
    }
    int cnt = 0;
    Node* temp = head;
    while(temp != NULL)
    {
        cnt++;
        temp = temp->next;
    }
    return cnt;
}

```

//Search element

```

int idxSearch(Node*& head, int val)
{
    if(head == NULL)
    {
        cout << "Linked List is Empty.\n";
        return -1;
    }
    int idx = 1;
    Node* temp = head;
    while(temp != NULL)
    {
        if(temp->data == val)
        {
            return idx;
        }
        temp = temp->next;
        idx++;
    }
    return -1;
}

```

//Find Max Node

```

int maxNode(Node*& head)
{
    if(!head)
        return INT_MIN;
}

```

```

int maxVal = head->data;
Node* temp = head->next;
while(temp)
{
    if(temp->data > maxVal)
        maxVal = temp->data;
    temp = temp->next;
}
return maxVal;
}

```

//Find Min Node

```

int minNode(Node*& head)
{
    if(!head)
        return INT_MAX;
    int minVal = head->data;
    Node* temp = head->next;
    while(temp)
    {
        if(temp->data < minVal)
            minVal = temp->data;
        temp = temp->next;
    }
    return minVal;
}

```

//Sum of all Nodes

```

int sumNode(Node*& head)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return -1;
    }
    int sum = 0;
    Node* temp = head;
    while(temp)
    {
        sum += temp->data;
        temp = temp->next;
    }
    return sum;
}

```

// ----- Reversal & Display -----

// Reverse Linked List

```

void reverseList(Node*& head)
{
    Node* prev = NULL;
    Node* curr = head;
    Node* next = NULL;

```

```

while(curr)
{
    next = curr->next;
    curr->next = prev;
    prev = curr;
    curr = next;
}
head = prev;
}

//Print in reverse without modifying
//1. Recursive Approach
void reverse_recursive(Node* head)
{
    if(!head)
    {
        //cout << "Linked List is Empty.\n";
        return;
    }
    reverse_recursive(head->next);
    cout << head->data << "->";
}

```

```

//2. Iterative Approach using a Stack
void reverse_stack(Node* head)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return;
    }
    stack<int> s;
    Node* temp = head;
    while(temp)
    {
        s.push(temp->data);
        temp = temp->next;
    }
    while(!s.empty())
    {
        cout << s.top();
        s.pop();
        if(!s.empty())
            cout << "->";
        else
            cout << "->NULL\n";
    }
}

```

```

// Reverse in Pair
void reverseInPair(Node*& head)
{
    if(!head || !head->next)

```

```

{
    cout << "Linked List is Empty or Single node.\n";
    return;
}
Node* curr = head;
Node* prev = NULL;
head = head->next;
while(curr && curr->next)
{
    Node* nextPair = curr->next->next;
    Node* second = curr->next;

    second->next = curr;
    curr->next = nextPair;

    if(prev)
        prev->next = second;
    prev = curr;
    curr = nextPair;
}
}

// Reverse sublist (between m and n)
void reverse_sublist(Node*& head, int m, int n)
{
    int len = countNode(head);
    if(!head || n > len || n < m)
    {
        cout << "Linked List is Empty or last value is more then list length.\n";
        return;
    }
    Node* temp = head;
    for(int i=1;i<m;i++)
    {
        temp = temp->next;
    }
    cout << "temp->data(m): " << temp->data << endl;

    temp = head;
    for(int i=1;i<n;i++)
    {
        temp = temp->next;
    }
    cout << "temp->data(n): " << temp->data << endl;

    Node* prev = NULL;
    Node* curr = head;
    Node* next = NULL;
    int pos = 1;
    while(pos < m)
    {
        prev = curr;
        curr = curr->next;
        pos++;
    }

```

```

}
Node* sub = prev;
Node* tail = curr;
for(int i=0;i<=n-m;i++)
{
    next = curr->next;
    curr->next = prev;
    prev = curr;
    curr = next;
}
if(sub)
    sub->next = prev;
else
    head = prev;
tail->next = curr;
return;
}

```

// ----- Analysis -----

// Occurance Count

```

void cntOccure(Node* head)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return;
    }
    unordered_map<int, int> freq;
    Node* temp = head;
    while(temp)
    {
        freq[temp->data]++;
        temp = temp->next;
    }
    for(auto& f : freq)
        cout << f.first << " " << f.second << endl;
}

```

//Remove Duplicates (Sorted or Unsorted)

```

void removeDub(Node*& head)
{
    unordered_set<int> seen;
    Node* curr = head;
    Node* prev = NULL;
    while(curr)
    {
        if(seen.find(curr->data) != seen.end())
        {
            prev->next = curr->next;
            delete curr;
            curr = prev->next;
        }
    }
}

```

```

        else
        {
            seen.insert(curr->data);
            prev = curr;
            curr = curr->next;
        }
    }
}

```

```

// Find middle element
//Using Length of linked list
/*

```

```

    int midEle(Node* head)
    {
        if(!head || !head->next)
        {
            cout << "Linked List is Empty or Single node.\n";
            return -1;
        }
        int l = countNode(head);
        if(l%2 == 0)
            l = l-1;
        Node* temp = head;
        int i = 0;
        for(i=0;i<l/2;i++)
        {
            temp = temp->next;
        }
        return temp->data;
    }
*/

```

```

int midEle(Node* head)
{
    if(!head)
        return -1;
    int len = countNode(head);
    Node* temp = head;
    for(int i=0;i<len/2;i++)
        temp = temp->next;
    return temp->data;
}

```

```

//Using slow/fast of linked list
/*

```

```

    int middleEle(Node* head)
    {
        if(!head || !head->next)
        {
            cout << "Linked List is Empty or Single node.\n";
            return -1;
        }
        Node* slow = head;
        Node* fast = head->next;
        while(fast && fast->next)

```

```

{
slow = slow->next;
fast = fast->next->next;
}
return slow->data;
}
*/
int middleEle(Node* head)
{
    if(!head)
        return -1;
    Node* slow = head;
    Node* fast = head;
    while(fast && fast->next)
    {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow->data;
}

```

```

// Find nth node from start or end
pair<int, int> nthNode(Node* head, int pos)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return {-1, -1};
    }
    int len = countNode(head);
    if(len < pos)
    {
        cout << "Position is Wrong.\n";
        return {-1, -1};
    }
    int start = INT_MIN;
    int end = INT_MIN;
    int l = 1;
    Node* temp = head;
    while(temp != NULL && l < pos)
    {
        l++;
        temp = temp->next;
    }
    start = temp->data;
    int j = 0;
    Node* temp1 = head;
    while(j < (len-pos))
    {
        j++;
        temp1 = temp1->next;
    }
    end = temp1->data;
}

```

```

        return {start, end};
    }

// Check palindrome – using stack
bool checkPalindrome(Node* head)
{
    if(!head)
        return false;
    stack<int> s;
    Node* temp = head;
    while(temp != NULL)
    {
        s.push(temp->data);
        temp = temp->next;
    }
    temp = head;
    while(temp != NULL)
    {
        if(temp->data != s.top())
            return false;
        else
            s.pop();
        temp = temp->next;
    }
    return true;
}

```

```

// Check palindrome – using fast/slow pointer
/*
bool palindromeCheck(Node* head)
{
    if(!head) return false;
    Node* slow = head;
    Node* fast = head;
    while(fast && fast->next)
    {
        slow = slow->next;
        fast = fast->next->next;
    }
    //cout << "slow->data:- " << slow->data << endl;
    Node* prev = NULL;
    Node* curr = slow;
    while(curr)
    {
        Node* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    Node* first = head;
    Node* second = prev;
    while(second)
    {
        if(first->data != second->data)

```

```

return false;
first = first->next;
second = second->next;
}
return true;
}
*/
bool palindromeCheck(Node* head)
{
    if(head == nullptr || head->next == nullptr)
        return true;
    Node* slow = head;
    Node* fast = head;
    while(fast && fast->next)
    {
        slow = slow->next;
        fast = fast->next->next;
    }
    // Skip middle node for odd length
    if(fast)
        slow = slow->next;
    // Reverse second half
    Node* prev = nullptr;
    Node* curr = slow;
    while(curr)
    {
        Node* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    Node* secondHalfHead = prev;
    // Compare
    Node* first = head;
    Node* second = secondHalfHead;
    bool isPalindrome = true;
    while(second)
    {
        if(first->data != second->data)
        {
            isPalindrome = false;
            break;
        }
        first = first->next;
        second = second->next;
    }
    // Restore second half
    curr = secondHalfHead;
    prev = nullptr;
    while(curr)
    {
        Node* next = curr->next;
        curr->next = prev;
        prev = curr;
    }
}

```

```

        curr = next;
    }
    return isPalindrome;
}

```

// ----- Cycle / Loop -----

// create cycle in linke list by position
void createCycle(Node* head, int pos)

```

{
    if (pos < 0 || head == nullptr)
        return;
    Node* temp = head;
    Node* cycleStartNode = nullptr;
    int index = 0;
    while (temp->next != nullptr)
    {
        if (index == pos)
        {
            cycleStartNode = temp;
        }
        temp = temp->next;
        index++;
    }
    if (index == pos)
        cycleStartNode = temp;
    if (cycleStartNode != nullptr)
    {
        temp->next = cycleStartNode;
    }
}

```

// create cycle in linked list by value

/* // Value Present Confirm

void createloop(Node* head, int val)

```

{
    Node* temp = head;
    while(temp->data != val)
    {
        temp = temp->next;
    }
    Node* last = head;
    while(last->next != NULL)
    {
        last = last->next;
    }
    last->next = temp;
}

```

*/

void createloop(Node* head, int val)

```

{
    if(!head)
        return;

```

```

Node* target = nullptr;
Node* temp = head;
while(temp)
{
    if(temp->data == val)
    {
        target = temp;
        break;
    }
    temp = temp->next;
}
if(!target)
{
    cout << "Value not found.\n";
    return;
}
Node* last = head;
while(last->next)
    last = last->next;
last->next = target;
}

//Cycle / loop detection
bool hasCycle(Node* head)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return false;
    }
    Node* slow = head;
    Node* fast = head;
    while(fast && fast->next)
    {
        slow = slow->next;
        fast = fast->next->next;
        if(slow == fast)
        {
            return true;
        }
    }
    return false;
}

// Find start of loop
int loopStart(Node* head)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return INT_MIN;
    }
    Node* slow = head;
    Node* fast = head;

```

```

while(fast && fast->next)
{
    slow = slow->next;
    fast = fast->next->next;
    if(slow == fast)
    {
        slow = head;
        while(slow != fast)
        {
            slow = slow->next;
            fast = fast->next;
        }
        return slow->data;
    }
}
return INT_MIN;
}

```

```

// Find length of loop
int loopLen(Node* head)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return 0;
    }
    Node* slow = head;
    Node* fast = head;
    while(fast && fast->next)
    {
        slow = slow->next;
        fast = fast->next->next;
        if(slow == fast)
        {
            int len = 1;
            Node* temp = slow->next;
            while(temp != slow)
            {
                len++;
                temp = temp->next;
            }
            return len;
        }
    }
    return 0;
}

```

```

// Detect loop and delete
void delLoop(Node*& head)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return;
    }
}

```

```

}
Node* slow = head;
Node* fast = head;
bool isloop = false;
while(fast && fast->next)
{
    slow = slow->next;
    fast = fast->next->next;
    if(slow == fast)
    {
        cout << "Loop Detected in linked List.\n";
        isloop = true;
        break;
    }
}
if(!isloop)
    return;
slow = head;
if(slow == fast)
{
    while(fast->next != slow)
        fast = fast->next;
    fast->next = NULL;
    return;
}
while(slow->next != fast->next)
{
    slow = slow->next;
    fast = fast->next;
}
fast->next = NULL;
return;
}

```

// ----- Advanced -----

// Merge 2 Sorted Lists

```

Node* mergeSort(Node* head, Node* head1)
{
    if(head == NULL)
        return head1;
    if(head1 == NULL)
        return head;
    if(head->data <= head1->data)
    {
        head->next = mergeSort(head->next, head1);
        return head;
    }
    else
    {
        head1->next = mergeSort(head, head1->next);
        return head1;
    }
}

```

```
}
```

```
// Remove N-th node from end – two-pass method
```

```
void removeFromEnd(Node* head, int pos)
```

```
{
    if(!head)
        return;
    Node* temp = head;
    int len = 0;
    while(temp != NULL)
    {
        len++;
        temp = temp->next;
    }
    Node* r = head;
    int cnt = len-pos-1;
    while(cnt)
    {
        r = r->next;
        cnt--;
    }
    Node *Del = r->next;
    r->next = r->next->next;
    delete Del;
}
```

```
// Remove N-th node from end – two-pointer method
```

```
void removeFromEnd(Node*& head, int pos)
```

```
{
    if(head == NULL)
        return;
    Node* dummy = new Node();
    dummy->next = head;
    Node* slow = dummy;
    Node* fast = dummy;
    for(int i=0; i<=pos; i++)
    {
        if(fast == NULL) return;
        fast = fast->next;
    }
    while(fast)
    {
        fast = fast->next;
        slow = slow->next;
    }
    Node* del = slow->next;
    slow->next = slow->next->next;
    delete del;
    head = dummy->next;
    delete dummy;
}
```

```
// Create intersection point
```

```
void createIntersection(Node* head1, Node* head2, int position)
```

```

{
    if(!head1 || !head2 || position <= 0) return;
    Node* temp1 = head1;
    int count = 1;
    while(temp1 != NULL && count < position)
    {
        temp1 = temp1->next;
        count++;
    }
    if(temp1 == NULL) return;
    Node* temp2 = head2;
    while(temp2->next != NULL)
    {
        temp2 = temp2->next;
    }
    temp2->next = temp1;
}

```

// Find intersection of two lists

// 1. Using 2 Pointer

bool checkIntersection(Node* head, Node* head1, int& value)

```

{
    if(!head || !head1) return false;

    Node* p1 = head;
    Node* p2 = head1;

    while(p1 != p2)
    {
        p1 = (p1 == NULL) ? head1 : p1->next;
        p2 = (p2 == NULL) ? head : p2->next;
    }
    if(p1 != NULL)
    {
        value = p1->data;
        return true;
    }
    return false;
}

```

// 2. Using Length

bool CheckIntersection(Node* head, Node* head1, int& value)

```

{
    int head_len = countNode(head);
    int head1_len = countNode(head1);
    int d = abs(head_len - head1_len);
    Node* h1 = head;
    Node* h2 = head1;
    if(head_len > head1_len)
    {
        for(int i=0;i<d;i++)
            h1 = h1->next;
    }
    else

```

```

{
    for(int i=0;i<d;i++)
        h2 = h2->next;
}
while(h1 != NULL && h2 != NULL)
{
    if(h1 == h2)
    {
        value = h1->data;
        return true;
    }
    h1 = h1->next;
    h2 = h2->next;
}
return false;
}

```

// Rotate list (by k positions) – right rotation
/*

```

void rightRotation(Node*& head, int pos)
{
    if(!head || !head->next || (pos == 0))
        return;
    int len = countNode(head);
    Node* temp = head;
    while(temp->next != NULL)
        temp = temp->next;
    if(pos % len == 0)
        return;
    temp->next = head;
    Node* newTail = head;
    for(int i=0;i<len-pos-1;i++)
        newTail = newTail->next;
    Node* newHead = newTail->next;
    newTail->next = nullptr;
    head = newHead;
}
*/

```

void rightRotation(Node*& head, int k)

```

{
    if(!head || !head->next || k == 0)
        return;
    int len = countNode(head);
    k = k % len;
    if(k == 0)
        return;
    Node* tail = head;
    while(tail->next)
        tail = tail->next;
    tail->next = head;
    Node* newTail = head;
    for(int i=1;i<len-k;i++)
        newTail = newTail->next;
    Node* newHead = newTail->next;
}

```

```

    newTail->next = nullptr;
    head = newHead;
}

// Rotate list (by k positions) – left rotation
/*

```

```

void leftRotation(Node*& head, int pos)
{
    if(!head || !head->next || (pos == 0))
        return;
    int len = countNode(head);
    Node* temp = head;
    while(temp->next != NULL)
        temp = temp->next;
    if(pos % len == 0)
        return;
    temp->next = head;
    Node* newTail = head;
    for(int i=0;i<pos-1;i++)
        newTail = newTail->next;
    Node* newHead = newTail->next;
    newTail->next = nullptr;
    head = newHead;
}
*/

```

```

void leftRotation(Node*& head, int k)
{
    if(!head || !head->next || k == 0)
        return;
    int len = countNode(head);
    k = k % len;
    if(k == 0)
        return;
    Node* tail = head;
    while(tail->next)
        tail = tail->next;
    tail->next = head;
    Node* newTail = head;
    for(int i=1;i<k;i++)
        newTail = newTail->next;
    Node* newHead = newTail->next;
    newTail->next = nullptr;
    head = newHead;
}

```

```

// Split list using 2 pointer (By Half)
void split_into_half(Node* head, Node*& first, Node*& second)
{
    if(!head)
    {
        first = second = nullptr;
        return;
    }
    Node* slow = head;

```

```

Node* fast = head;
Node* prev = nullptr;
while(fast && fast->next)
{
    prev = slow;
    slow = slow->next;
    fast = fast->next->next;
}
first = head;
second = slow;
if(prev)
    prev->next = nullptr;
return;
}

```

//Split list using position

```

void split_by_pos(Node* head, Node*& first, Node*& second, int pos)
{
    if(head == NULL)
    {
        first = second = nullptr;
        return;
    }
    Node* temp = head;
    Node* prev = NULL;
    for(int i=0;i<pos && temp;i++)
    {
        prev = temp;
        temp = temp->next;
    }
    first = head;
    second = temp;
    if(prev)
        prev->next = nullptr;
}

```

//Split list using value

```

void split_by_value(Node* head, Node*& first, Node*& second, int val)
{
    if(head == nullptr)
    {
        first = second = nullptr;
        return;
    }
    Node* temp = head;
    Node* prev = NULL;
    while(temp != nullptr && temp->data != val)
    {
        prev = temp;
        temp = temp->next;
    }
    if (!temp)
    {
        first = head;

```

```

        second = nullptr;
        return;
    }
    first = head;
    second = temp;
    if(prev)
        prev->next = nullptr;
}

```

// Add 2 numbers

```
void add2Nos(Node* head, Node* head1, Node*& out)
```

```

{
    out = NULL;
    Node* tail = NULL;
    int carry = 0;
    while(head || head1 || carry)
    {
        int sum = carry;
        if(head)
        {
            sum+= head->data;
            head = head->next;
        }
        if(head1)
        {
            sum+=head1->data;
            head1 = head1->next;
        }
        Node* newNode = new Node();
        newNode->data = (sum%10);
        newNode->next = nullptr;
        carry = sum/10;
        if(!out)
        {
            out = tail = newNode;
        }
        else
        {
            tail->next = newNode;
            tail = tail->next;
        }
    }
}

```

// Odd-even reordering

```
void odd_even_reordering(Node*& head)
```

```

{
    if(!head || !head->next) return;
    Node* odd = head;
    Node* even = head->next;
    Node* evenhead = even;
    while(even && even->next)
    {
        odd->next = even->next;

```

```

        odd = odd->next;
        even->next = odd->next;
        even = even->next;
    }
    odd->next = evenhead;
}

```

// Skip M nodes and delete N nodes

```

/*
void skipMdeleteN(Node*& head, int M, int N)
{
    Node* curr = head;
    Node* temp;
    while(curr)
    {
        for(int i=1;i<M && curr;i++)
        {
            curr = curr->next;
        }
        if(curr == NULL) break;
        Node* toDelete = curr->next;
        for(int i=0;i<N && toDelete;i++)
        {
            temp = toDelete;
            toDelete = toDelete->next;
            delete temp;
        }
        curr->next = toDelete;
        curr = toDelete;
    }
}
*/

```

```

void skipMdeleteN(Node*& head, int M, int N)
{
    if(M <= 0)
    {
        return;
    }
    Node* curr = head;
    while(curr)
    {
        for(int i=1;i<M && curr;i++)
            curr = curr->next;
        if(!curr)
            break;
        Node* temp = curr->next;
        for(int i=0;i<N && temp;i++)
        {
            Node* del = temp;
            temp = temp->next;
            delete del;
        }
        curr->next = temp;
        curr = temp;
    }
}

```

```

    }
}

// Delete M nodes and skip N nodes
void deleteMskipN(Node*& head, int M, int N)
{
    Node* curr = head;
    Node* prev = NULL;
    while(curr)
    {
        for(int i=0;i<M && curr;i++)
        {
            Node* temp = curr;
            curr = curr->next;
            delete temp;
        }
        if(prev == NULL)
            head = curr;
        else
            prev->next = curr;
        for(int i=0;i<N && curr;i++)
        {
            prev = curr;
            curr = curr->next;
        }
    }
}

```

```

// Segregate 0s, 1s, 2s
void segregate_0s_1s_2s(Node* head)
{
    int cnt0 = 0, cnt1 = 0, cnt2 = 0;
    Node* temp = head;
    while(temp)
    {
        if(temp->data == 0) cnt0++;
        else if(temp->data == 1) cnt1++;
        else cnt2++;
        temp = temp->next;
    }
    temp = head;
    while(temp)
    {
        if(cnt0 > 0)
        {
            temp->data = 0;
            cnt0--;
        }
        else if(cnt1 > 0)
        {
            temp->data = 1;
            cnt1--;
        }
        else

```

```

    {
        temp->data = 2;
        cnt2--;
    }
    temp = temp->next;
}
}

```

// ----- Cleanup -----

// Free List

```

void freeList(Node*& head)
{
    while(head)
    {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
}

```

int main()

```

{
    Node* head = NULL;
    int n, v;

    cout << "Enter initial Linked List size: ";
    cin >> n;
    /*for(int i = 0; i < n; i++)
    {
        cin >> v;
        insertAtEnd(head, v);
    }*/
    int i = 1;
    while(i<=n)
    {
        insertAtEnd(head, i);
        i++;
    }
    cout << "Initial list: ";
    showLL(head);

    int choice;
    do {
        cout << "\n===== LINKED LIST MENU =====\n";
        cout << "\t--- Basic Setup ---\n";
        cout << " 1. createNode\n";
        cout << " 2. insertAtStart\n";
        cout << " 3. insertAtEnd\n";
        cout << " 4. insertAtPosition\n";
        cout << " 5. showLL\n";
        cout << "\t--- Deletion ---\n";
    } while (choice != 0);
}

```

```

cout << " 6. deleteFromBeg\n";
cout << " 7. deleteFromEnd\n";
cout << " 8. deleteFromPos\n";
cout << " 9. deleteByValue\n";
cout << "10. deleteAllOccurance\n";
cout << "\t--- Utilities ---\n";
cout << "11. countNode\n";
cout << "12. idxSearch\n";
cout << "13. maxNode\n";
cout << "14. minNode\n";
cout << "15. sumNode\n";
cout << "\t--- Reversal & Display ---\n";
cout << "16. reverseList\n";
cout << "17. reverse_recursive\n";
cout << "18. reverse_stack\n";
cout << "19. reverseInPair\n";
cout << "20. reverse_sublist\n";
cout << "\t--- Analysis ---\n";
cout << "21. cntOccure\n";
cout << "22. removeDub\n";
cout << "23. midEle (length method)\n";
cout << "24. middleEle (slow/fast method)\n";
cout << "25. nthNode\n";
cout << "26. checkPalindrome (stack)\n";
cout << "27. palindromeCheck (slow/fast)\n";
cout << "\t--- Cycle / Loop ---\n";
cout << "28. createCycle (by position)\n";
cout << "29. createloop (by value)\n";
cout << "30. hasCycle\n";
cout << "31. loopStart\n";
cout << "32. loopLen\n";
cout << "33. delLoop\n";
cout << "\t--- Advanced ---\n";
cout << "34. mergeSort (merge 2 sorted lists)\n";
cout << "35. removeFromEnd (2-pass)\n";
cout << "36. removefromEnd (2-pointer)\n";
cout << "37. insertNewValueAfterValue\n";
cout << "38. checkIntersection (2-pointer)\n";
cout << "39. CheckIntersection (length)\n";
cout << "40. rightRotation (By k position)\n";
cout << "41. leftRotation (By k position)\n";
cout << "42. split list (By 2-pointer into 2 parts)\n";
cout << "43. split list (By k position into 2 parts)\n";
cout << "44. split list (By k value into 2 parts)\n";
cout << "45. add 2 Numbers\n";
cout << "46. Odd-even reordering\n";
cout << "47. skip M delete N\n";
cout << "48. Delete M skip N\n";
cout << "49. segerate 0s 1s 2s\n";
cout << "\t--- Cleanup ---\n";
cout << "50. freeList\n";
cout << " 0. Exit\n";
cout << "=====\n";
cout << "Enter choice: ";

```

```
cin >> choice;
```

```
switch(choice)
```

```
{
```

```
    case 1:
```

```
    {
```

```
        int val;
```

```
        cout << "Enter value for new node: ";
```

```
        cin >> val;
```

```
        Node* node = createNode(val);
```

```
        cout << "Created node with value: " << node->data << endl;
```

```
        delete node;
```

```
        break;
```

```
    }
```

```
    case 2:
```

```
    {
```

```
        int val;
```

```
        cout << "Enter value to insert at start: ";
```

```
        cin >> val;
```

```
        insertAtStart(head, val);
```

```
        cout << "After insertAtStart: ";
```

```
        showLL(head);
```

```
        break;
```

```
    }
```

```
    case 3:
```

```
    {
```

```
        int val;
```

```
        cout << "Enter value to insert at end: ";
```

```
        cin >> val;
```

```
        insertAtEnd(head, val);
```

```
        cout << "After insertAtEnd: ";
```

```
        showLL(head);
```

```
        break;
```

```
    }
```

```
    case 4:
```

```
    {
```

```
        int val, pos;
```

```
        cout << "Enter value and position: ";
```

```
        cin >> val >> pos;
```

```
        insertAtPosition(head, val, pos);
```

```
        cout << "After insertAtPosition: ";
```

```
        showLL(head);
```

```
        break;
```

```
    }
```

```
    case 5:
```

```
    {
```

```
        cout << "Current list: ";
```

```
        showLL(head);
```

```
        break;
```

```
    }
```

```
    case 6:
```

```
    {
```

```
        deleteFromBeg(head);
```

```
        cout << "After deleteFromBeg: ";
```

```

        showLL(head);
        break;
    }
case 7:
    {
        deleteFromEnd(head);
        cout << "After deleteFromEnd: ";
        showLL(head);
        break;
    }
case 8:
    {
        int pos;
        cout << "Enter position to delete: ";
        cin >> pos;
        deleteFromPos(head, pos);
        cout << "After deleteFromPos: ";
        showLL(head);
        break;
    }
case 9:
    {
        int val;
        cout << "Enter value to delete: ";
        cin >> val;
        deleteByValue(head, val);
        cout << "After deleteByValue: ";
        showLL(head);
        break;
    }
case 10:
    {
        int val;
        cout << "Enter value to delete all occurrences: ";
        cin >> val;
        deleteAllOccurance(head, val);
        cout << "After deleteAllOccurance: ";
        showLL(head);
        break;
    }
case 11:
    {
        cout << "Node count: " << countNode(head) << endl;
        break;
    }
case 12:
    {
        int val;
        cout << "Enter value to search: ";
        cin >> val;
        int idx = idxSearch(head, val);
        if(idx != -1)
            cout << "Found at index: " << idx << endl;
        else

```

```

        cout << "Value not found.\n";
        break;
    }
case 13:
    {
        cout << "Max value: " << maxNode(head) << endl;
        break;
    }
case 14:
    {
        cout << "Min value: " << minNode(head) << endl;
        break;
    }
case 15:
    {
        cout << "Sum of nodes: " << sumNode(head) << endl;
        break;
    }
case 16:
    {
        reverseList(head);
        cout << "After reverseList: ";
        showLL(head);
        break;
    }
case 17:
    {
        cout << "Reverse (recursive): ";
        reverse_recursive(head);
        cout << "NULL\n";
        break;
    }
case 18:
    {
        cout << "Reverse (stack): ";
        reverse_stack(head);
        break;
    }
case 19:
    {
        reverseInPair(head);
        cout << "After reverseInPair: ";
        showLL(head);
        break;
    }
case 20:
    {
        int m, n;
        cout << "Enter start (m) and end (n) positions: ";
        cin >> m >> n;
        reverse_sublist(head, m, n);
        cout << "After reverse_sublist: ";
        showLL(head);
        break;
    }

```

```

    }
case 21:
{
    cout << "Occurrence counts:\n";
    cntOccure(head);
    break;
}
case 22:
{
    removeDub(head);
    cout << "After removeDub: ";
    showLL(head);
    break;
}
case 23:
{
    cout << "Middle element (length): " << midEle(head) << endl;
    break;
}
case 24:
{
    cout << "Middle element (slow/fast): " << middleEle(head) << endl;
    break;
}
case 25:
{
    int pos;
    cout << "Enter position: ";
    cin >> pos;
    auto res = nthNode(head, pos);
    if(res.first != INT_MIN)
        cout << "From start: " << res.first << " From end: " << res.second
<< endl;
    break;
}
case 26:
{
    cout << "Palindrome (stack): " << (checkPalindrome(head) ? "Yes" :
"No") << endl;
    break;
}
case 27:
{
    cout << "Palindrome (slow/fast): " << (palindromeCheck(head) ?
"Yes" : "No") << endl;
    break;
}
case 28:
{
    int pos;
    cout << "Enter position to create cycle: ";
    cin >> pos;
    createCycle(head, pos);
    cout << "Cycle created at position " << pos << ".\n";
}

```

```

        break;
    }
case 29:
    {
        int val;
        cout << "Enter value to start loop at: ";
        cin >> val;
        createloop(head, val);
        cout << "Loop created starting at value " << val << ".\n";
        break;
    }
case 30:
    {
        cout << "Has cycle: " << (hasCycle(head) ? "Yes" : "No") << endl;
        break;
    }
case 31:
    {
        int s = loopStart(head);
        if(s != INT_MIN)
            cout << "Loop starts at value: " << s << endl;
        else
            cout << "No loop detected.\n";
        break;
    }
case 32:
    {
        int len = loopLen(head);
        if(len)
            cout << "Loop length: " << len << endl;
        else
            cout << "No loop detected.\n";
        break;
    }
case 33:
    {
        delLoop(head);
        cout << "Has cycle after delLoop: " << (hasCycle(head) ? "Yes" :
"No") << endl;
        showLL(head);
        break;
    }
case 34:
    {
        Node* head1 = NULL;
        int n2, v2;
        cout << "Enter second sorted list size: ";
        cin >> n2;
        for(int i = 0; i < n2; i++)
        {
            cin >> v2;
            insertAtEnd(head1, v2);
        }
        cout << "List 1: "; showLL(head);
    }

```

```

        cout << "List 2: "; showLL(head1);
        Node* merged = mergeSort(head, head1);
        head = merged;
        cout << "Merged sorted list: ";
        showLL(head);
        break;
    }
case 35:
    {
        int pos;
        cout << "Enter position from end: ";
        cin >> pos;
        removeFromEnd(head, pos);
        cout << "After removeFromEnd (2-pass): ";
        showLL(head);
        break;
    }
case 36:
    {
        int pos;
        cout << "Enter position from end: ";
        cin >> pos;
        removefromEnd(head, pos);
        cout << "After removefromEnd (2-pointer): ";
        showLL(head);
        break;
    }
case 37:
    {
        int tar, val;
        cout << "Enter value to add after value: ";
        cin >> tar >> val;
        insertNewValueAfterValue(head, tar, val);
        cout << "After insertNewValueAfterValue: ";
        showLL(head);
        break;
    }
case 38:
    {
        Node* head1 = new Node();
        head1->data = 10;
        head1->next = nullptr;
        int pos, out;
        cout << "Enter position to create intersection: ";
        cin >> pos;
        createIntersection(head, head1, pos);
        showLL(head1);
        bool res = checkIntersection(head, head1, out);
        if(res)
        {
            cout << "Point of Intersection is Present.\n";
            cout << "    Point of Intersection is " << out << endl;
        }
        else

```

```

        cout << "No point of Intersection.\n";
        break;
    }
case 39:
    {
        Node* head1 = new Node();
        head1->data = 1000;
        head1->next = nullptr;
        int pos, out;
        cout << "Enter position to create intersection: ";
        cin >> pos;
        createIntersection(head, head1, pos);
        showLL(head1);
        bool res = CheckIntersection(head, head1, out);
        if(res)
        {
            cout << "Point of Intersection is Present.\n";
            cout << "    Point of Intersection is " << out << endl;
        }
        else
            cout << "No point of Intersection.\n";
        break;
    }
case 40:
    {
        int k;
        cout << "Enter value to right rotate: ";
        cin >> k;
        cout << "Linked List Before Right rotation: ";
        showLL(head);
        cout << "Linked List After Right rotation: ";
        rightRotation(head, k);
        showLL(head);
        break;
    }
case 41:
    {
        int k;
        cout << "Enter value to leftate: ";
        cin >> k;
        cout << "Linked List Before Left rotation: ";
        showLL(head);
        cout << "Linked List After Left rotation: ";
        leftRotation(head, k);
        showLL(head);
        break;
    }
case 42:
    {
        cout << "Linked list Before split: ";
        showLL(head);
        cout << "Linked list After split: \n";
        Node* first;
        Node* second;
    }

```

```

        split_into_half(head, first, second);
        cout << "First part:- ";
        showLL(first);
        cout << "Second part:- ";
        showLL(second);
        break;
    }
case 43:
    {
        int pos;
        cout << "Enter position want to split: ";
        cin >> pos;
        cout << "Linked list Before split: ";
        showLL(head);
        cout << "Linked list After split: \n";
        Node* first;
        Node* second;
        split_by_pos(head, first, second, pos);
        cout << "First part:- ";
        showLL(first);
        cout << "Second part:- ";
        showLL(second);
        break;
    }
case 44:
    {
        int val;
        cout << "Enter Value want to split: ";
        cin >> val;
        cout << "Linked list Before split: ";
        showLL(head);
        cout << "Linked list After split: \n";
        Node* first;
        Node* second;
        split_by_value(head, first, second, val);
        cout << "First part:- ";
        showLL(first);
        cout << "Second part:- ";
        showLL(second);
        break;
    }
case 45:
    {
        Node* head1 = NULL;
        Node* result = NULL;
        int n1, v1;
        cout << "Enter size of Linked List2: ";
        cin >> n1;
        for(int i=0;i<n1;i++)
        {
            cin >> v1;
            insertAtEnd(head1, v1);
        }
        cout << "List 1: ";
    }

```

```

        showLL(head);
        cout << "List 2: ";
        showLL(head1);
        add2Nos(head, head1, result);
        showLL(result);
        break;
    }
case 46:
    {
        cout << "List before odd-even reordering: ";
        showLL(head);
        odd_even_reordering(head);
        cout << "List after odd-even reordering: ";
        showLL(head);
        break;
    }
case 47:
    {
        int x, y;
        cout << "Enter 2 Value for skip & delete: ";
        cin >> x >> y;
        cout << "List before skip M delete N: ";
        showLL(head);
        skipMdeleteN(head, x, y);
        cout << "List after skip M delete N: ";
        showLL(head);
        break;
    }
case 48:
    {
        int x, y;
        cout << "Enter 2 Value for delete & skip: ";
        cin >> x >> y;
        cout << "List before delete M skip N: ";
        showLL(head);
        deleteMskipN(head, x, y);
        cout << "List after delete M skip N: ";
        showLL(head);
        break;
    }
case 49:
    {
        cout << "List before segregate 0s 1s 2s: ";
        showLL(head);
        segerate_0s_1s_2s(head);
        cout << "List after segregate 0s 1s 2s: ";
        showLL(head);
        break;
    }
case 50:
    {
        freeList(head);
        cout << "After freeList: ";
        showLL(head);
    }

```

```

        break;
    }
    case 0:
    {
        cout << "Exiting...\n";
        freeList(head);
        break;
    }
    default:
    {
        cout << "Invalid choice. Try again.\n";
    }
}

} while(choice != 0);

return 0;
}

```

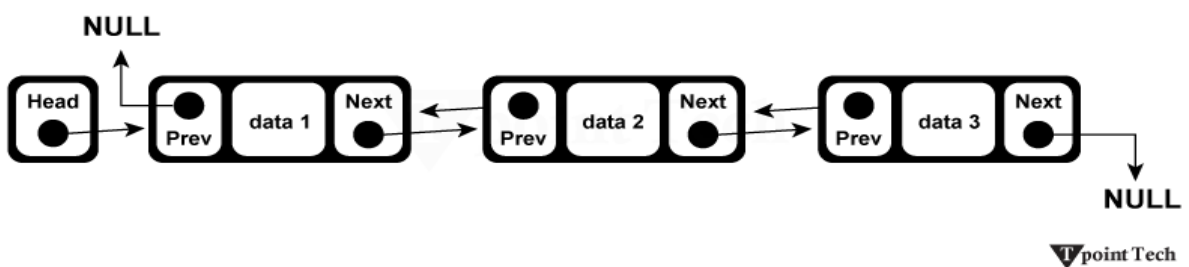
Doubly Linked List (Data Structure)

The doubly linked list is an essential data structure that allows efficient forward and backward navigation. For example, in a music playlist application, a doubly linked list allows smooth switching between songs.

Each node in a doubly linked list contains the actual data and two references (or pointers): one pointing to the previous node and the other to the next node. This two-way linking enables efficient bidirectional traversal.

A doubly linked list is a useful data structure in which each node contains three parts:

- Data Part: It stores the actual information.
- Next Pointer: It holds the reference to the following node.
- Prev Pointer: It includes the reference to the preceding node.



How it works?

Head and Tail: The head links to the first data of the list and the tail links to the last data. These links enhance operations such as adding or removing elements at the ends of the list.

Traversal: It can start from the Head and follow the next pointers to move forward, or start from the Tail and follow the prev pointers to move backward.

Insertion and Deletion: Adding or deleting nodes can be done efficiently at any point in the list.

Structure of a Doubly Linked List

Here, the doubly linked list is represented in C programming:

```

struct node {
    int data;
    struct node *next;

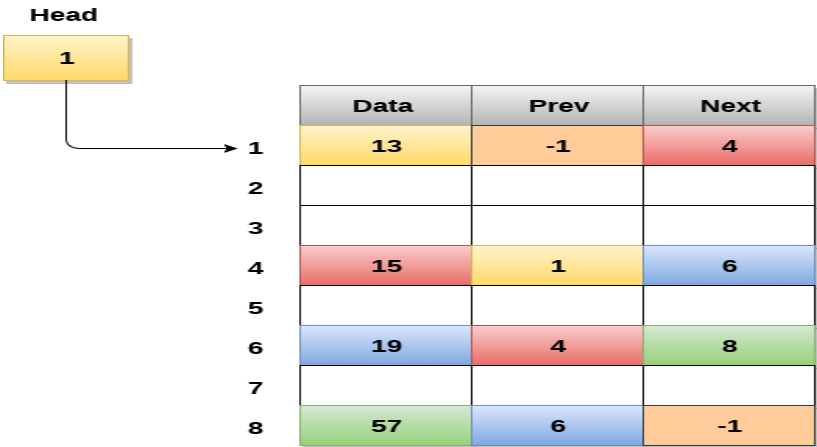
```

```
struct node *prev;
}
```

Memory Representation of a Doubly Linked List

Generally, doubly linked list consumes more space for every node and therefore, causes more expansive basic operations such as insertion and deletion. However, we can easily manipulate the elements of the list since the list maintains pointers in both the directions (forward and backward).

In the following image, the first element of the list that is i.e. 13 stored at address 1. The head pointer points to the starting address 1. Since this is the first element being added to the list therefore the **prev** of the list **contains** null. The next node of the list resides at address 4 therefore the first node contains 4 in its next pointer. We can traverse the list in this way until we find any node containing null or -1 in its next part.



Memory Representation of a Doubly linked list

Doubly Linked List Operations and Execution Trace

1. Common Setup for Insertion/Deletion:

- **Create Node:** Create a newNode with data; set prev and next to null.
- **Handle Empty List (Insertion):** If head==null, set both head and tail to newNode.
- **Handle Single Node (Deletion):** If head==tail, set head=tail=null.